

嵌入式人工智能实验指南

基于 EEP-TPU 的算法与软件开发手册

V0.94n

2020.06

目录

PART I. 深度学习基础.....	3
1.1 深度学习发展历程.....	3
1.2 卷积神经网络.....	6
1.3 深度学习模型.....	20
1.4 深度学习框架.....	29
PART II. EEP-TPU 张量处理器.....	39
2.1 处理器架构.....	39
2.2 EEP-TPU 张量处理器架构.....	42
2.3 EEP-TPU 神经网络处理器架构.....	44
2.4 EEP-TPU 异构计算.....	45
PART III. EEP-TPU 开发流程.....	48
3.1 开发流程介绍.....	48
3.2 基于 Caffe 框架的深度学习分类算法实验.....	52
3.3 基于 Caffe 框架的深度学习目标检测算法实验.....	87
PART IV. EEP-TPU 创新应用.....	113
4.1 应用开发介绍.....	113
4.2 基于 USB 摄像头的人脸检测实验.....	114
4.3 基于 USB 摄像头的客流统计实验.....	122
4.4 基于 USB 摄像头的 HDMI 显示.....	132
附录 A. NNAPI.....	135
附录 B. EEP-TPU Compiler.....	149

I. 深度学习基础

1.1 深度学习发展历程

1.1.1 传统机器学习的局限性

传统机器学习技术在处理原始形态的自然数据方面有很大的局限性。

几十年来，构建模式识别或机器学习系统需要技艺高超的工程师和经验丰富的领域专家来设计特征提取器（Feature Extractor），将原始数据（如图像的像素值）转化为合适的中间表示形式或者特征向量（Feature Vector），再利用学习子系统（通常为分类器）对输入模式进行检测或分类，如图 1 所示。

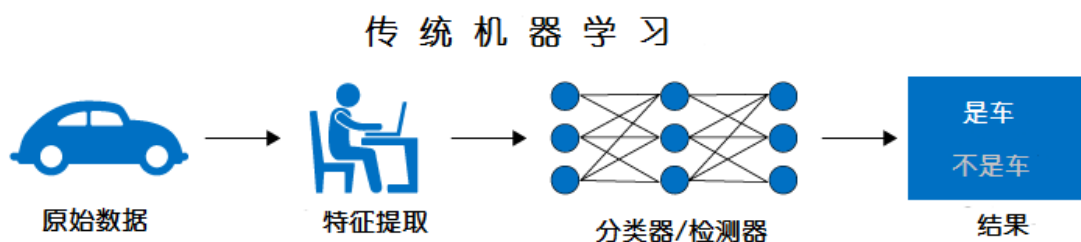


图 1 传统机器学习处理模式

深度学习方法则不需要人工设计特征提取器，而是由机器自动学习获得，特别适用于变化多端的自然数据，具有非常优良的泛化能力和鲁棒性。

1.1.2 从表示学习到深度学习

在表示学习（Representation Learning）系统中，直接以原始数据形式提供机器输入，自动发现用于检测和分类的表示（Representation）；深度学习是一种多层的表示学习方法，由简单的非线性模块构建而成，这些模块将上一层表示（从原始数据开始）转化为更高层、更抽象的表示。当一个学习系统由足够多这样的简单非线性模块构建时，可以学习非常复杂的特征，从而实现各种复杂功能。

对于分类问题，高层表示可以强调重要的类别信息，同时抑制无关的背景信息。一幅图像总是以像素值数组的形式提供网络输入，第一层学习到的特征为边缘信息，即图像某个位置是否存在特定朝向的边缘；第二层检测边缘信息按特定方式排列组成的基本图案，而不关心边缘位置的变化；第三层将基本图案组合起来，对应典型物体的部件；后续层检测由部件组成的物体，如图 2 所示。深度学习的概念源于人工神经网络的研究，含多隐层的多层感知器就是一种深度学习结构。深度学习通过组合低层特征形成更加抽象的高层表示属性类别或特征，以发现数据的分布式特征表示，最关键的方面是这些特征层不是由专家设计的，而是使由通用学习方法自动从数据学习得到的。这些从低到高的“表示”是人类无法预估的，完全由机器决定哪些特征是自己需要的，哪些是可以抑制的。

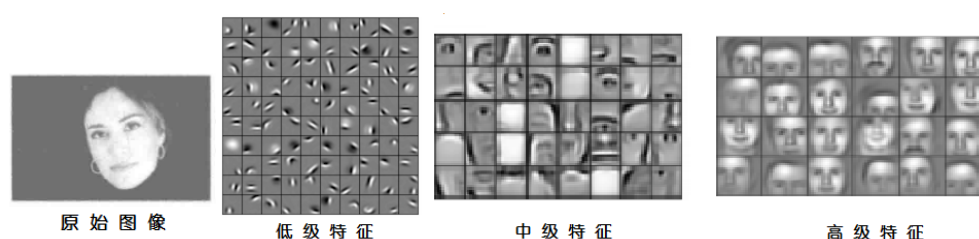


图 2 表示学习中间特征

深度学习十分擅长在高维数据中发现复杂结构，可应用于科学、商业和政务等很多领域，在图像识别、语音识别中打破多项记录。更惊人的是，在自然语言理解（特别是主题分类、句法分析、问答系统和语言翻译）中产生了大量有价值的结果。

深度学习只需要少量人工介入，非常适合当前大规模计算系统和海量数据。

1.1.3 生活中的深度学习

深度学习首先在语音识别领域取得了成功。微软研究院的邓立、俞栋和辛顿合作的产品于 2011 年发布，打败了传统的基于混合高斯模型（Gaussian Mixed Model）的办法，取得了不错的效果。在这些成果背后，深度学习，尤其是深度循环神经网络（Recurrent Neural Networks, RNN）在背后发挥着巨大的作用。后来 Google、Facebook 等大公司也通过研发或收购分别推出了基于深度学习的语音识别系统。国内也研发了基于深度学习的语音识别系统，比较有代表性的如科大讯飞、百度等。如

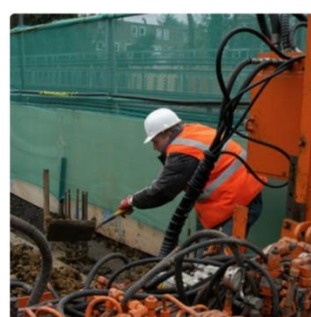
今无论是苹果的 Siri，微软的小娜，还是安卓手机上的语音输入助手，背后的技术都是深度学习。

当然提到语音输入助手就不能不提更上层的自然语言处理，如今我们只需要对微信说句话就能转化成文字消息发给朋友，对着地图说出要去的地方就能返回给我们规划好的路线。英文不好，没关系，在计算机上复制、粘贴就能翻译。这些让我们生活更便利的技术，几乎都是基于深度学习。

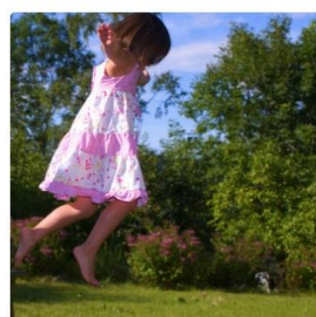
除了语言语音领域，另一个深度学习应用最广泛的就是图像和视觉。事实上图像和视觉一直以来就是机器学习领域所钟爱的方向，只不过在深度学习出现之前，能达到实用的应用并不多。而在深度学习出现后，尤其是 2012 年之后，基于深度学习图像和视觉应用迅速覆盖了生活的方方面面。当你走在街上看到一件好看的衣服，立刻拿出手机用淘宝或者京东的拍照购物功能，就能找到同款或者类似的款式。当你打开计算机，只需要刷一下指纹或者对着摄像头笑一笑，就能进入系统，既方便又安全。去银行办理公积金联名卡，费时又费力，如今银行工作人员带着 iPad 和便携设备，只需要按照办理系统的提示对着摄像头就能搞定。犯罪分子作案后潜逃，刑警们不需要熬夜看周边的监控视频，智能搜查系统就能搞定。还有现在各大科技公司和汽车企业都在大力发展的无人驾驶，背后的技术都少不了基于深度学习的计算机视觉。



"man in black shirt is playing guitar."



"construction worker in orange safety vest is working on road."



"girl in pink dress is jumping in air."



"black and white dog jumps over bar."

图 3 基于深度学习的图像识别

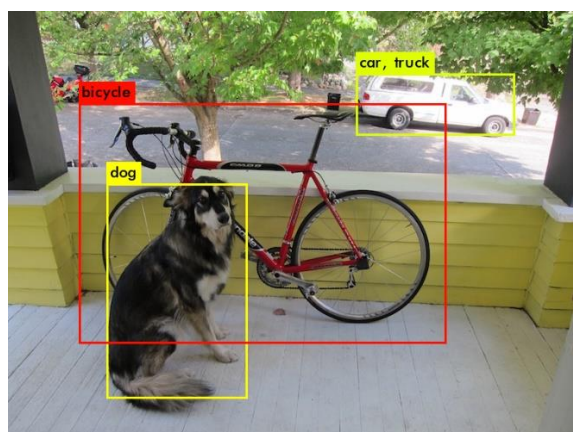


图 4 基于深度学习的图像目标检测及识别

大数据也是近年来一个热点，其背后的主要技术之一也是深度学习，如通过生物医药海量数据助力药品研发和疾病诊断、又如分析海量的用户行为和数据，帮助银行和 P2P 金融公司做出更加安全的放贷决定、还有分析海量的商品数据，帮助电商更好地销售商品，用户也更容易买到想要的东西。其他的还有推荐系统、机器人、游戏等，深度学习正在让人们的生活更加便利。

1.2 卷积神经网络

1.2.1 基本概念

早在 1989 年，LeCun 就发明了卷积神经网络，并且被广泛应用于美国的很多银行系统中，用来识别支票上的手写数字。

卷积神经网络是受到了动物视觉神经系统的启发，所以先来简单了解一下视觉神经系统。眼睛是一个成像系统，图像通过瞳孔、晶状体最终在视网膜上成像，这一部分是视觉的光学系统。视网膜上布满了大量的光感受细胞，可以把光刺激转换成神经冲动，从这些细胞开始，就进入了视觉神经系统。神经冲动经过视觉通路（视觉神经→视交叉→视束→外侧膝状体→视放射）传递进大脑的初级视觉皮层。从视觉皮层开始，对图像信息的处理也是分层的，这些层分别是 VI 到 V5（V 代表 Visual）。每一层处理一部分特定的信息，比如 VI 主要对一些边缘纹理响应，V4 则对一些简单的形状响应，而信号传递的大概顺序也是从 VI→V5，需要注意的是这种大概顺序并不是一层层的简单连接，比如 V2 和其他所有层都有连接。

上面只是大概提一下神经科学中的分层结构，真正对现在的计算机视觉尤其是对卷积神经网络启发很大的发现，是加拿大科学家大卫·休伯尔（David Hubel），以及瑞典科学家托斯坦·维厄瑟尔西（Torsten Wiesel）从 1958 年起对猫视觉皮层的研究。他们在 VI 皮层里发现了两种细胞，简单细胞（Simple Cells）和复杂细胞（Complex Cells），这两种细胞都有一个特点就是每个细胞只对特定方向的条形图样刺激有反应，也就是说，这些细胞是有方向性选择的。这两种细胞的主要区别是简单细胞对应的视网膜上的光感受细胞所在的区域很小，而复杂细胞则对应更大的区域，这个区域被称作感受野（Receptive Field）。这种结构如果简化一下形象理解如图 5 所示。

模拟这种概念的结构，早在 1980 年计算机界由日本人 Kunihiro Fukushima 提出了 Neocognitron。现在的卷积神经网络结构深受 Neocognitron 的影响，简单来说就是卷积层用来模拟对特定图案的响应，而池化层模拟感受野。基本的结构如图 4 所示。

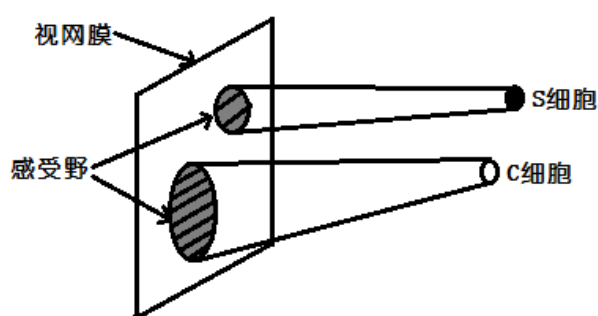


图 1 感受野视图

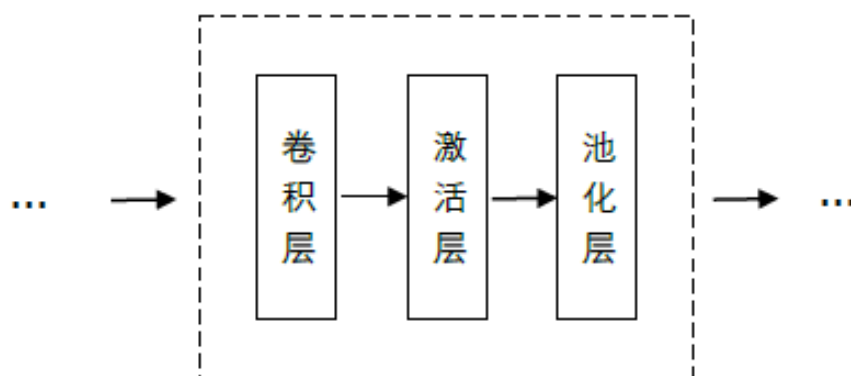


图 2 卷积神经网络中卷积层+池化层的基本结构

图 6 中所示的是卷积神经网络中最基础的一种结构，输入首先经过卷积层得到响应，然后经过激活层做非线性变换。卷积本身也可以看做是一种线性变换，再考虑到偏置项，这种仿射变换、非线性变换的操作其实就是一般神经网络中基本操作的一种特殊情况。卷积神经网络是一种特殊的深层神经网络模型，它的特殊性体现在两个方面，一方面它的神经元间的连接是非全连接的，另一方面同一层中某些神经元之间的连接的权重是共享的（即相同的）。它的非全连接和权值共享的网络结构使之更类似于生物神经网络，降低了网络模型的复杂度（对于很难学习的深层结构来说，这是非常重要的），减少了权值的数量。所以相对来说更特殊一点的是激活层后面的池化层。接下来的小节就稍微深入一些来分别讨论卷积层和池化层。

1.2.2 卷积

(1) 广义的卷积

卷积，很多时候是我们在各种工程领域、信号领域所看到的常用名词，例如：

- 统计学中：加权的滑动平均是一种卷积。
- 概率论中：两个统计独立变量 X 与 Y 的和的概率密度函数是 X 与 Y 的概率密度函数的卷积。
- 声学中：回声可以用源声与一个反映各种反射效应的函数的卷积表示。
- 电子工程与信号处理中：任一个线性系统的输出都可以通过将输入信号与系统函数（系统的冲激响应）做卷积获得。
- 物理学中：任何一个线性系统（符合叠加原理）都存在卷积。
- 计算机科学中：卷积神经网络（CNN）是深度学习算法中的一种，近年来被广泛用到模式识别、图像处理等领域中。

这六个领域中，卷积起到了至关重要的作用。在面对一些复杂情况时，作为一种强有力的处理方法，卷积给出了简单却有效的输出。对于机器学习领域，尤其是深度学习，最著名的 CNN 卷积神经网络(Convolutional Neural Network, CNN)，在图像领域取得了非常好的实际效果，从一出现便横扫各类算法，在后文我们将详细介绍。

(2) 神经网络中的卷积

卷积神经网络的名字就来自于其中的卷积操作。卷积的主要目的是为了从输入图像中提取特征。卷积可以通过从输入的一小块数据中学到图像的特征，并可以保留像素间的空间关系。卷积层输入特征图通过和卷积核局部连接进行卷积操作，将卷积结果累加并加一个偏置量后所得即为输出特征图。通过该过程从输入图像提取不同特征，

并且通过堆叠多个卷积层来提取更高级特征。一般地，卷积层可以用以下表达式 (1) 表示：

$$X_j^l = \sum_{i \in N_j} X_i^{l-1} * W_{i,j} + B_j^l \quad (1)$$

其中， X_j^l 表示第 l 层卷积层的第 j 个卷积核对应的输出特征图， N_j 表示当前卷积对输入特征图的选择， $W_{i,j}$ 表示第 l 层的第 j 个卷积核的第 i 个加权系数， B_j^l 表示第 l 层卷积层的第 j 个卷积核对应的偏置系数。

a. 二维图像上的卷积操作

每张图像都可以看作是像素值的矩阵。对于一个 5×5 的图像，它的像素值仅为 0 或者 1（注意对于灰度图像而言，像素值的范围是 0 到 255，下面像素值为 0 和 1 的绿色矩阵仅为特例）：

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

图 3 原始矩阵

同时，另一个 3×3 的矩阵，可以所示如下：

1	0	1
0	1	0
1	0	1

图 4 辅助矩阵

我们用橙色的矩阵在原始图像（绿色）上滑动，每次滑动一个像素（也叫做“步长”），在每个位置上，我们计算对应元素的乘积（两个矩阵间），并把乘积的和作为最后的结果，得到输出矩阵（粉色）中的每一个元素的值。注意， 3×3 的矩阵每次步长中仅可以“看到”输入图像的一部分。得到结果：

4	3	4
2	4	3
2	3	4

图 5 结果矩阵

在 CNN 的术语中，3x3 的矩阵叫做“滤波器 (filter) ”或者“核 (kernel) ”或者“特征检测器 (feature detector) ”，通过在图像上滑动滤波器并计算点乘得到矩阵叫做“卷积特征 (Convolved Feature) ”或者“激活图 (Activation Map) ”或者“特征映射 (Feature Map) ”。记住滤波器在原始输入图像上的作用是特征检测器。

由结果可以看出，对于同样的输入图像，不同值的滤波器将会生成不同的特征映射。

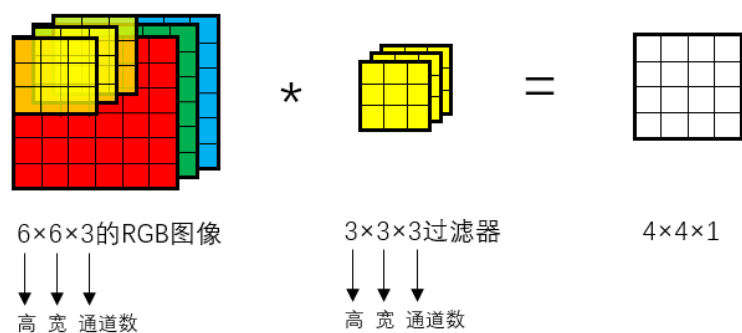
在实践中，CNN 会在训练过程中学习到这些滤波器的值（尽管我们依然需要在训练前指定诸如滤波器的个数、滤波器的大小、网络架构等参数）。我们使用的滤波器越多，提取到的图像特征就越多，网络所能在未知图像上识别的模式也就越好。

特征映射的大小（卷积特征）由下面三个参数控制，我们需要在卷积前确定它们：

- 深度 (Depth)：深度对应的是卷积操作所需的滤波器个数，也称为通道数。在接下来的网络中，我们使用三个不同的滤波器对原始图像进行卷积操作，这样就可以生成三个不同的特征映射。你可以把这三个特征映射看作是堆叠的 2d 矩阵，那么，特征映射的“深度”或“通道数”就是 3。
- 步长 (Stride)：步长是我们在输入矩阵上滑动滤波矩阵的像素数。当步长为 1 时，我们每次移动滤波器一个像素的位置。当步长为 2 时，我们每次移动滤波器会跳过 2 个像素。步长越大，将会得到更小的特征映射。
- 零填充 (Zero-padding)：有时，在输入矩阵的边缘使用零值进行填充，这样我们就可以对输入图像矩阵的边缘进行滤波。零填充的一大好处是可以帮助我们控制特征映射的大小。使用零填充的也叫做泛卷积，不适用零填充的叫做严格卷积。

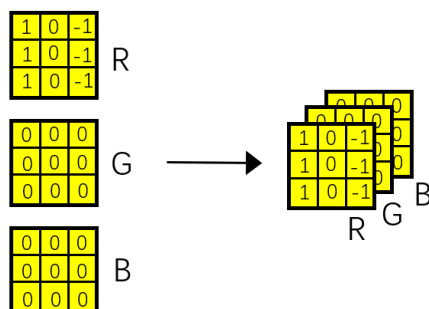
b. 三维图像上的卷积操作

输入图像为一个 6×6 的 RGB 彩色图像，其输入特征图的大小为 6×6×3，滤波器大小为 3×3×3，输出 4×4×1 的特征图。如下所示：

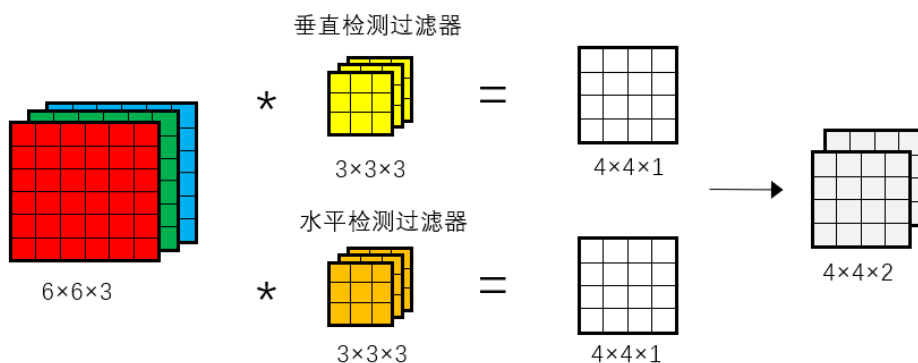


对于过滤器而言，总共有 27 个参数。将过滤器放在 RGB 图像的左上角，通道对应进行卷积（27 个参数和原输入图像的 27 个特征值的乘积和），得到一个输出特征值；将过滤器滑动一个单位（步长为 1），再与原 RGB 图像上的对应位置进行卷积，得到下一个输出；依次进行计算，得到其余的输出。

若只检测红色通道的垂直边缘信息，则可以将深度为 3 的过滤器进行如下设置：



也可以同时对三个通道都进行边缘检测，这取决于三个通道所对应的过滤器的参数设置。如果想要同时获取垂直边缘检测结果、水平边缘检测结果或其他特征等，则需要同时使用多个三通道过滤器。

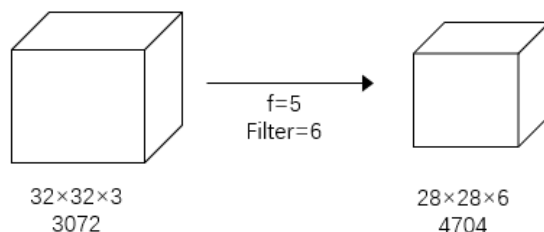


步幅为 1，无 padding 条件下，卷积维度总结如下：

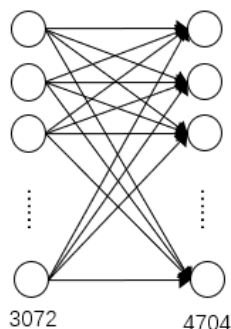
- 输入图像： $n \times n \times n_c$ （上例中为 $6 \times 6 \times 3$ ）
- 卷积滤波器： $f \times f \times n_c$ （上例中为 $3 \times 3 \times 3$ ）
- 卷积结果： $(n-f+1) \times (n-f+1) \times N$ (过滤器的个数) （上例中为 $4 \times 4 \times 1$ ）

(3) 卷积神经网络 vs. 全连接神经网络

卷积神经网络和全连接网络相比较而言，卷积网络所需要训练的参数远远小于全连接网络。如下例所示，卷积计算输入输出特征图的大小分别为 3072 和 4704。



如果对应在全连接层中，如下图所示，可以知道在全连接网络中所需要的权重数量为 3072×4704 ，大约 1400 万个。



尽管当前的技术可以实现 1400 万个参数的训练，考虑到这些参数是在输入图像的大小为 $32 \times 32 \times 3$ 的前提下的总数，如果输入图像变得更大，比如 1000×1000 ，那么权重矩阵会变得异常大。

再来看一下卷积层的参数数量，每个过滤器有 5×5 ，25 个参数，再加上偏差，26 个参数；一共 6 个过滤器，则总共是 156 个参数，相比于 1400 万个权重而言，数量非常小。

经过上面的比较，可以看出选择卷积神经网络是一种更为有效的方法，之所以卷积网络中参数数量会如此小，是因为卷积网络的两个特点：稀疏连接和权值共享。

a. 稀疏连接(Sparse Connectivity)

卷积网络通过在相邻两层之间强制使用局部连接模式来利用图像的空间局部特性，即，在第 m 层的隐层单元只与第 $m-1$ 层的输入单元的局部区域有连接，第 $m-1$ 层的这些局部区域被称为空间连续的接受域。我们可以将这种结构描述如下：

设第 $m-1$ 层为视网膜输入层，第 m 层的接受域宽度为 3，也就是说该层的每个单元与且仅与输入层 ($m-1$) 层的 3 个相邻的神经元相连，第 m 层与第 $m+1$ 层具有类似的链接规则，如下图所示。

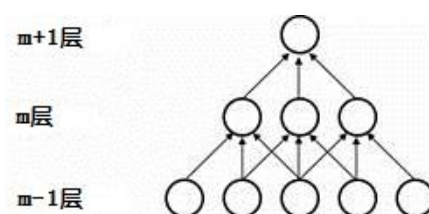


图 6 空间连续的接受域

可以看到 $m+1$ 层的神经元相对于第 m 层的接受域宽度也为 3，但相对于输入层的接受域为 5，这种结构将学习到的过滤器（或者卷积核）（对应于输入信号中可以被最大激活的单元）限制在局部空间模式（因为每个单元对它接受域外的 variation 不做反应）。从上图也可以看出，多个这样的层堆叠起来后，会使得过滤器（不再是线性的）逐渐成为全局的（也就是覆盖到了更大的视觉区域）。例如上图中第 $m+1$ 层的神经元可以对宽度为 5 的输入进行一个非线性的特征编码。

b. 权值共享(Shared Weights)

在卷积网络中，每个稀疏过滤器通过共享权值都会覆盖整个可视域，这些共享权值的单元构成一个特征映射，如下图所示。

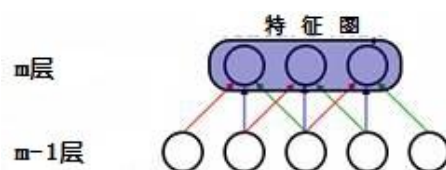
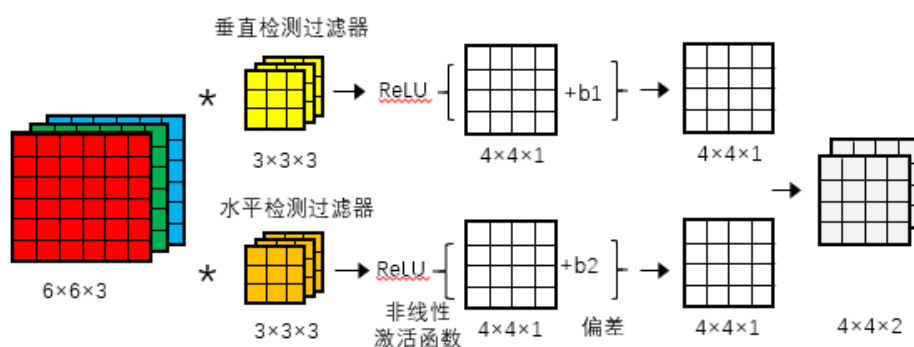


图 7 共享权值的单元构成的特征映射

在图中，有 3 个隐层单元，他们属于同一个特征映射。同种颜色的连接权值是相同的，我们仍然可以使用梯度下降的方法来学习这些权值，只需要对原始算法做一些小的改动，这里共享权值的梯度是所有共享参数的梯度总和。我们不禁会问为什么要权重共享呢？一方面，重复单元能够对特征进行识别，而不考虑它在可视域中的位置。另一方面，权值共享使得我们能更有效的进行特征抽取，因为它极大的减少了需要学习的自由变量的个数。通过控制模型的规模，卷积网络对视觉问题可以具有很好的泛化能力。

1.2.3 单层卷积网络

我们已经知道卷积神经网络的一个重要组成部分是卷积层，接下来讲解如何构建卷积层。在上一节中已经了解了卷积的概念，并同时使用两个过滤器进行卷积，获取两个 4×4 的卷积特征图，也就是说并行使用了两个卷积神经网络层。在卷积结果的基础上，为每个值加上偏差 b （是一个实数），然后使用非线性激活函数进行运算



在传统神经网络中，有： $z^{[1]} = w^{[1]}a^{[0]} + b^{[1]}$ ， $a^{[1]} = g(z^{[1]})$ ，在卷积神经网络中， $w^{[1]}$ 使用过滤器来实现，卷积实际上实现了 $w^{[1]}a^{[0]}$ 的操作。使用经过激活函数后输出

的特征图作为下一网络结构的输入，输出特征图的深度取决于过滤器的个数（也就是索要提取的特征数量，上图例子中为 2。）

单层卷积网络各种标记以及各个部分维度计算：

卷积网络所在的层数： l

第 l 层过滤器的尺寸： $f^{[l]}$

Padding 的数量： $p^{[l]}$

卷积步幅： $s^{[l]}$

l 层输入数据维度 (input)： $n_H^{[l-1]} \times n_W^{[l-1]} \times n_c^{[l-1]}$

l 层输出数据维度 (input)： $n_H^{[l]} \times n_W^{[l]} \times n_c^{[l]}$

$$n_H^{[l]} = \left\lfloor \frac{n_H^{[l-1]} + 2p^{[l]} - f^{[l]}}{s^{[l]}} + 1 \right\rfloor$$

$$n_W^{[l]} = \left\lfloor \frac{n_W^{[l-1]} + 2p^{[l]} - f^{[l]}}{s^{[l]}} + 1 \right\rfloor$$

l 层过滤器的数量： $n_c^{[l]}$

l 层每个过滤器参数的个数： $f^{[l]} \times f^{[l]} \times n_c^{[l-1]}$

激活函数输出 $a^{[l]}$ 维度： $n_H^{[l]} \times n_W^{[l]} \times n_c^{[l]}$

l 层所有过滤器参数的个数（总权重）： $f^{[l]} \times f^{[l]} \times n_c^{[l-1]} \times n_c^{[l]}$

l 层偏差 (bias) 参数的个数： $n_c^{[l]}$ —— (1, 1, 1, 1, $n_c^{[l]}$)

卷积过滤器 (filter) 参数的个数只与过滤器的大小有关，和输入特征图的大小无关；输出特征图深度（通道数）取决于本层卷积过滤器的个数。在实际的应用中分析卷积层参数维度时，可以参照以上标记及维度计算方法。

1.2.4 池化

空间池化 (Spatial Pooling)（也叫做亚采用或者下采样）降低了各个特征映射的维度，同时可以保持大部分重要的信息。空间池化有最大化、平均化、加和等方式。对于最大池化 (Max Pooling)，我们定义一个空间邻域（比如，2x2 的窗口），并从窗口内的修正特征映射中取出最大的元素。除了取最大元素，我们也可以取平均

(Average Pooling) 或者对窗口内的元素求和。在实际中，最大池化被证明效果更好一些。

下面的图展示了使用 2x2 窗口在修正特征映射（在卷积 + ReLU 操作后得到）上使用最大池化的例子。

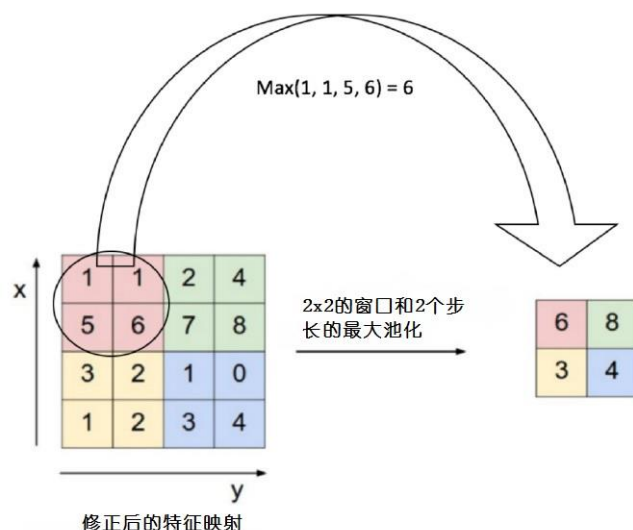


图 8 修正后的特征进行最大池化

我们以 2 个元素（也叫做“步长”）滑动我们 2x2 的窗口，并在每个区域内取最大值。如上图所示，这样操作可以降低我们特征映射的维度。

在下图展示的网络中，池化操作是分开应用到特征映射的各个通道上的（注意，因为这样的操作，我们可以从三个通道的特征映射中得到三个通道的池化结果）。



图 9 可分的池化操作

池化函数可以逐渐降低输入表示的空间尺度。特别地，池化：

- 使输入表示（特征维度）变得更小，并且网络中的参数和计算的数量更加可控的减小，因此，可以控制过拟合；
- 使网络对于输入图像中更小的变化、冗余和变换变得稳定（输入的微小冗余将不会改变池化的输出——因为我们在局部邻域中使用了最大化/平均值的操作；
- 帮助我们获取图像最大程度上的尺度不变性。它非常的强大，因为我们可以检测图像中的物体，无论它们位置在哪里。

1.2.5 深度卷积神经网络

卷积网络是为识别二维形状而特殊设计的一个多层感知器，这种网络结构对平移、比例缩放、倾斜或者其他形式的变形具有高度不变性。 这些良好的性能是网络在有监督方式下学会的，网络的结构主要有稀疏连接和权值共享两个特点，包括如下形式的功能：

- 特征提取。每一个神经元从上一层的局部接受域获取输入，因而迫使它提取局部特征。一旦一个特征被提取出来，只要它相对于其他特征的位置被近似地保留下来，它的精确位置就变得没有那么重要了。
- 特征映射。网络的每一个计算层都是由多个特征映射组成的，每个特征映射都是平面形式的。平面中单独的神经元在约束下共享相同的权值集，这种结构形式具有如下的有益效果：a.平移不变性。b.自由参数数量的缩减(通过权值共享实现)。
- 下采样。每个卷积层后面跟着一个实现局部平均和子抽样的计算层，由此特征映射的分辨率降低。这种操作具有使特征映射的输出对平移和其他形式的变形的敏感度下降的作用。

卷积神经网络是一个多层的神经网络，每层由多个二维平面组成，而每个平面由多个独立神经元组成。

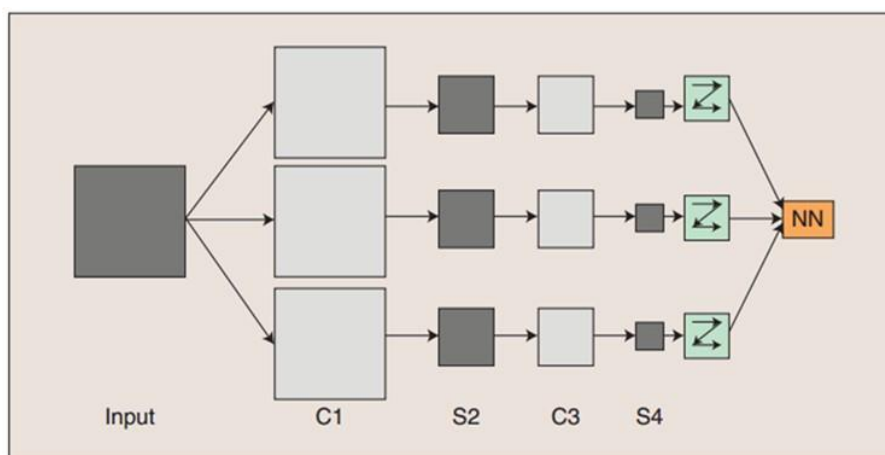


图 10 卷积神经网络的概念示范：

图 14 卷积神经网络的概念示范：输入图像通过和三个可训练的滤波器和可加偏置进行卷积，卷积后在 C1 层产生三个特征映射图，然后特征映射图中每组的四个像素再进行求和，加权值，加偏置，通过一个 Sigmoid 函数得到三个 S2 层的特征映射图。这些映射图再经过卷积得到 C3 层。这个层级结构再和 S2 一样产生 S4。最终，这些像素值被连接成一个向量输入到全连接神经网络，得到输出。

一般地，C 层为特征提取层，每个神经元的输入与前一层的局部感受野相连，并提取该局部的特征，一旦该局部特征被提取后，它与其他特征间的位置关系也随之确定下来；S 层是特征映射层，网络的每个计算层由多个特征映射组成，每个特征映射为一个平面，平面上所有神经元的权值相等。特征映射结构采用影响函数核小的 sigmoid 函数作为卷积网络的激活函数，使得特征映射具有位移不变性。

此外，由于一个映射面上的神经元共享权值，因而减少了网络自由参数的个数，降低了网络参数选择的复杂度。卷积神经网络中的每一个特征提取层（C-层）都紧跟着一个用来求局部平均与二次提取的计算层（S-层），这种特有的两次特征提取结构使网络在识别时对输入样本有较高的畸变容忍能力。

下面介绍卷积神经网络的一个完整模型

卷积神经网络是一个多层的神经网络，每层由多个二维平面组成，而每个平面由多个独立神经元组成。网络中包含一些简单元和复杂元，分别记为 S-元和 C-元。S-元聚合在一起组成 S-面，S-面聚合在一起组成 S-层，用 U_s 表示。C-元、C-面和 C-层 (U_s) 之间存在类似的关系。网络的任一中间级由 S-层与 C-层串接而成，而输入级只含

一层，它直接接受二维视觉模式，样本特征提取步骤已嵌入到卷积神经网络模型的互联结构中。

一般地， U_c 是特征映射层(卷积层)，每个神经元的输入与前一层的局部感受野相连，并提取该局部的特征，一旦该局部特征被提取后，它与其他特征间的位置关系也随之确定下来。

U_s 为特征提取层(下采样层)，网络的每个计算层由多个特征映射组成，每个特征映射为一个平面，平面上所有神经元的权值相等。特征映射结构采用影响函数核小的 sigmoid 函数作为卷积网络的激活函数，使得特征映射具有位移不变性。此外，由于一个映射面上的神经元共享权值，因而减少了网络自由参数的个数，降低了网络参数选择的复杂度。卷积神经网络中的每一个特征提取层(C-层)都紧跟着一个用来求局部平均与二次提取的计算层(S-层)，这种特有的两次特征提取结构使网络在识别时对输入样本有较高的畸变容忍能力。

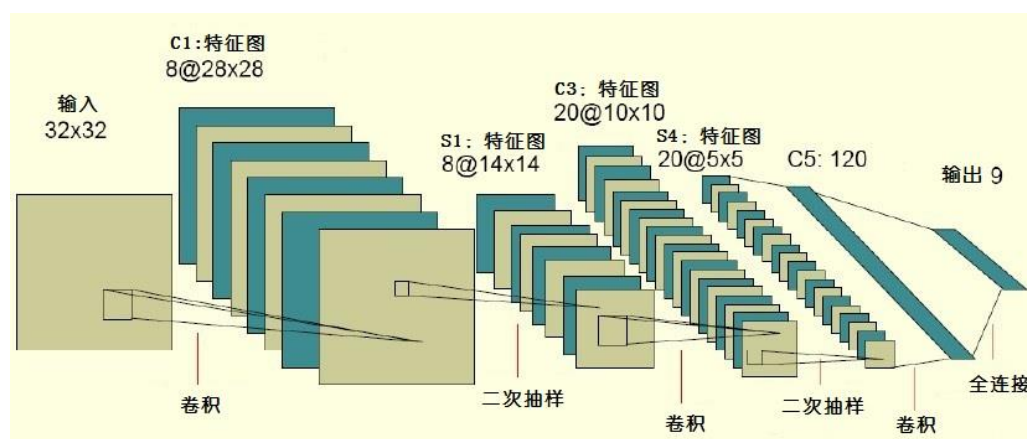


图 11 卷积网络实例

上图中的卷积网络工作流程如下，输入层由 32×32 个感知节点组成，接收原始图像。然后，计算流程在卷积和下采样之间交替进行，如下所述：

- 第一隐藏层进行卷积，它由 8 个特征映射组成，每个特征映射由 28×28 个神经元组成，每个神经元指定一个 5×5 的接受域。
- 第二隐藏层实现下采样和局部平均，它同样由 8 个特征映射组成，但其每个特征映射由 14×14 个神经元组成。每个神经元具有一个 2×2 的接受域，一个可训练系数，一个可训练偏置和一个 sigmoid 激活函数。可训练系数和偏置控制神经元的操作点。

- 第三隐藏层进行第二次卷积，它由 20 个特征映射组成，每个特征映射由 10×10 个神经元组成。该隐藏层中的每个神经元可能具有和下一个隐藏层几个特征映射相连的突触连接，它以与第一个卷积层相似的方式操作。
- 第四个隐藏层进行第二次下采样和局部平均计算。它由 20 个特征映射组成，但每个特征映射由 5×5 个神经元组成，它以与第一次抽样相似的方式操作。
- 第五个隐藏层实现卷积的最后阶段，它由 120 个神经元组成，每个神经元指定一个 5×5 的接受域。
- 最后是全连接层，得到输出向量。

相继的计算层在卷积和采样之间的连续交替，我们得到一个“双尖塔”的效果，也就是在每个卷积或抽样层，随着空间分辨率下降，与相应的前一层相比特征映射的数量增加。卷积之后进行下采样的思想是受到动物视觉系统中的“简单的”细胞后面跟着“复杂的”细胞的想法的启发而产生的。

图中所示的多层感知器包含近似 100000 个突触连接，但只有大约 2600 个自由参数(每个特征映射为一个平面，平面上所有神经元的权值相等)。自由参数在数量上显著地减少是通过权值共享获得的，学习机器的能力（以 VC 维的形式度量）下降，但提高了它的泛化能力。而且它对自由参数的调整通过反向传播学习的随机形式来实现。另一个显著的特点是使用权值共享使得以并行形式实现卷积网络变得可能。这是卷积网络对全连接的多层感知器而言的另一个优点。

1.3 深度学习模型

深度学习的概念源于人工神经网络的研究。含多隐层的多层感知器就是一种深度学习结构。深度学习通过组合低层特征形成更加抽象的高层表示属性类别或特征，以发现数据的分布式特征表示。

深度学习是一种技术类型，包含多个重要算法：

- Convolutional Neural Networks(CNN)卷积神经网络
- AutoEncoder 自动编码器
- Sparse Coding 稀疏编码
- Restricted Boltzmann Machine(RBM)限制波尔兹曼机
- Deep Belief Networks(DBN)深信度网络

- Recurrent neural Network(RNN)多层反馈循环神经网络神经网络

对于不同问题(图像, 语音, 文本), 需要选用不同网络模型才能达到更好效果。本章将介绍一些经典的卷积神经网络模型

1.3.1 LeNet-5

典型的卷积网络 LeNet-5 被用来识别数字。当年美国大多数银行用它来识别支票上面的手写数字。能够达到商用的地步, 它的准确性可想而知。

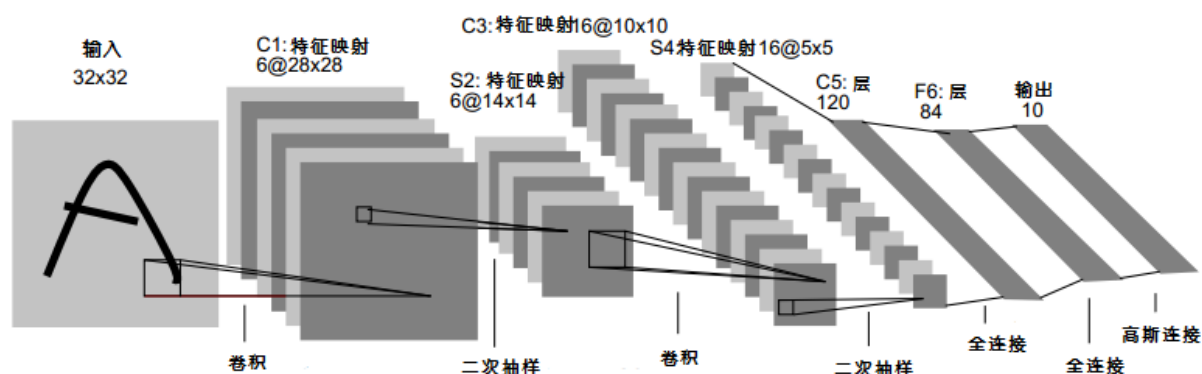


图 12 LeNet 示例图

LeNet-5 共有 7 层, 不包含输入, 每层都包含可训练参数(连接权重)。输入图像为 32*32 大小。这要比公认的手写数据库 Mnist 中最大的字母还大。这样做的原因是希望潜在的明显特征如笔画断点或角点能够出现在最高层特征监测子感受野的中心。

首先要明确一点: 每个层有多个特征映射, 每个特征映射通过一种卷积滤波器提取输入的一种特征, 然后每个特征映射有多个神经元。

C1 层是一个卷积层(通过卷积运算, 可以使原信号特征增强, 并且降低噪音), 由 6 个特征映射构成。特征映射中每个神经元与输入中 5*5 的邻域相连。特征映射的大小为 28*28, 这样能防止输入的连接掉到边界之外。C1 有 156 个可训练参数(每个滤波器包含 5*5=25 个 unit 参数和 1 个 bias 参数, 一共 6 个滤波器对应 6 个特征映射, 共(5*5+1)*6=156 个参数), 共 156*(28*28)=122,304 个连接。

S2 层是一个亚采样层（利用图像局部相关性的原理，对图像进行子抽样，可以减少数据处理量同时保留有用信息），有 6 个 14×14 的特征映射。特征映射中的每个单元与 C1 中相对应特征映射的 2×2 邻域相连接。S2 层每个单元对应的 4 个输入相加，乘以一个可训练参数，再加上一个可训练偏置，结果通过 sigmoid 函数计算。可训练系数和偏置控制着 sigmoid 函数的非线性程度。如果系数比较小，那么运算近似于线性运算，亚采样相当于模糊图像。如果系数比较大，根据偏置的大小亚采样可以被看成是有噪声的“或”运算或者有噪声的“与”运算。每个单元的 2×2 感受野并不重叠，因此 S2 中每个特征映射的大小是 C1 中特征映射大小的 $1/4$ （行和列各 $1/2$ ）。S2 层有 12 个可训练参数和 5880 个连接。

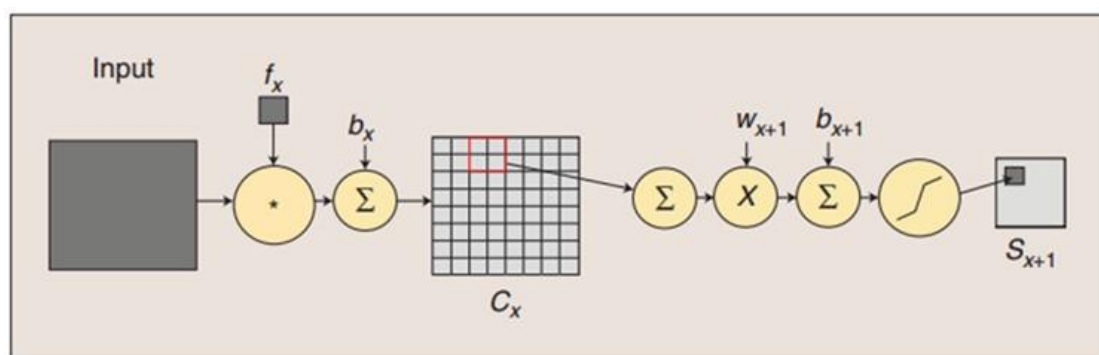


图 13 卷积和子采样过程

卷积过程包括：用一个可训练的滤波器 f_x 去卷积一个输入的图像（第一阶段是输入的图像，后面的阶段就是卷积特征映射了），然后加一个偏置 b_x ，得到卷积层 C_x 。子采样过程包括：每邻域四个像素求和变为一个像素，然后通过标量 w_{x+1} 加权，再增加偏置 b_{x+1} ，然后通过一个 sigmoid 激活函数，产生一个大概缩小四倍的特征映射图 S_{x+1} 。

所以从一个平面到下一个平面的映射可以看作是卷积运算，S-层可看作是模糊滤波器，起到二次特征提取的作用。隐层与隐层之间空间分辨率递减，而每层所含的平面数递增，这样可用于检测更多的特征信息。

C3 层也是一个卷积层，它同样通过 5×5 的卷积核去卷积层 S2，然后得到的特征映射就只有 10×10 个神经元，但是它有 16 种不同的卷积核，所以就存在 16 个特征映射了。这里需要注意的一点是：C3 中的每个特征映射是连接到 S2 中的所有 6 个或者几个特征映射的，表示本层的特征映射是上一层提取到的特征映射的不同组合。

刚才说 C3 中每个特征映射由 S2 中所有 6 个或者几个特征映射组合而成。为什么不把 S2 中的每个特征映射连接到每个 C3 的特征映射呢？原因有两点。第一：不完全的连接机制将连接的数量保持在合理的范围内。第二：也是最重要的，其破坏了网络的对称性。由于不同的特征映射有不同的输入，所以迫使他们抽取不同的特征。

例如，存在的一个方式是：C3 的前 6 个特征映射以 S2 中 3 个相邻的特征映射子集为输入。接下来 6 个特征映射以 S2 中 4 个相邻特征映射子集为输入。然后 3 个以不相邻的 4 个特征映射子集为输入。最后一个将 S2 中所有特征映射为输入。这样 C3 层有 1516 个可训练参数和 151600 个连接。

S4 层是一个下采样层，由 16 个 5×5 大小的特征映射构成。特征映射中的每个单元与 C3 中相应特征映射的 2×2 邻域相连接，跟 C1 和 S2 之间的连接一样。S4 层有 32 个可训练参数（每个特征映射 1 个因子和一个偏置）和 2000 个连接。

C5 层是一个卷积层，有 120 个特征映射。每个单元与 S4 层的全部 16 个单元的 5×5 邻域相连。由于 S4 层特征映射的大小也为 5×5 （同滤波器一样），故 C5 特征映射的大小为 1×1 ：这构成了 S4 和 C5 之间的全连接。之所以仍将 C5 标示为卷积层而非全连接层，是因为如果 LeNet-5 的输入变大，而其他的保持不变，那么此时特征映射的维数就会比 1×1 大。C5 层有 48120 个可训练连接。

F6 层有 84 个单元（选这个数字的原因来自于输出层的设计），与 C5 层全相连。有 10164 个可训练参数。如同经典神经网络，F6 层计算输入向量和权重向量之间的点积，再加上一个偏置。然后将其传递给 sigmoid 函数产生单元 i 的一个状态。

最后，输出层由欧式径向基函数（Euclidean Radial Basis Function）单元组成，每类一个单元，每个有 84 个输入。换句话说，每个输出 RBF 单元计算输入向量和参数向量之间的欧式距离。输入离参数向量越远，RBF 输出的越大。一个 RBF 输出可以被理解为衡量输入模式和与 RBF 相关联类的一个模型的匹配程度的惩罚项。用概率术语来说，RBF 输出可以被理解为 F6 层配置空间的高斯分布的负 log-likelihood。给定一个输入模式，损失函数应能使得 F6 的配置与 RBF 参数向量（即模式的期望分类）足够接近。这些单元的参数是人工选取并保持固定的（至少初始时候如此）。这些参数向量的成分被设为 -1 或 1。虽然这些参数可以以 -1 和 1 等概率的方式任选，或者构成一个纠错码，但是被设计成一个相应字符类的 7×12 大小（即 84）的格式化图片。这种表示对识别单独的数字不是很有用，但是对识别可打印 ASCII 集中的字符串发挥了巨大的作用。

1.3.2 VGG-Net

VGGNet 是牛津大学计算机视觉组（VisualGeometry Group）和 GoogleDeepMind 公司的研究员一起研发的深度卷积神经网络。VGGNet 探索了卷积神经网络的深度与其性能之间的关系，通过反复堆叠 3×3 的小型卷积核和 2×2 的最大池化层，VGGNet 成功地构筑了 16~19 层深的卷积神经网络。VGGNet 相比之前 state-of-the-art 的网络结构，错误率大幅下降，并取得了 ILSVRC 2014 比赛分类项目的第 2 名和定位项目的第 1 名。同时 VGGNet 的拓展性很强，迁移到其他图片数据上的泛化性非常好。VGGNet 的结构非常简洁，整个网络都使用了同样大小的卷积核尺寸（ 3×3 ）和最大池化尺寸（ 2×2 ）。到目前为止，VGGNet 依然经常被用来提取图像特征。VGGNet 训练后的模型参数在其官方网站上开源了，可用来在特定的图像分类任务上进行再训练（相当于提供了非常好的初始化权重），因此被用在了很多地方。

VGGNet 论文中全部使用了 3×3 的卷积核和 2×2 的池化核，通过不断加深网络结构来提升性能。图 11 所示为 VGGNet 各级别的网络结构图，从 11 层的网络一直到 19 层的网络都有详尽的性能测试。虽然从 A 到 E 每一级网络逐渐变深，但是网络的参数量并没有增长很多，这是因为参数量主要都消耗在最后 3 个全连接层。前面的卷积部分虽然很深，但是消耗的参数量不大，不过训练比较耗时的部分依然是卷积，因其计算量比较大。这其中的 D、E 也就是我们常说的 VGGNet-16 和 VGGNet-19。C 很有意思，相比 B 多了几个 1×1 的卷积层， 1×1 卷积的意义主要在于线性变换，而输入通道数和输出通道数不变，没有发生降维。训练时，输入是大小为 224×224 的 RGB 图像，预处理只有在训练集中的每个像素上减去 RGB 的均值。

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

图 14 VGGNet 各级别网络结构图

VGGNet 拥有 5 段卷积，每一段内有 2 至 3 个卷积层，同时每段尾部会连接一个最大池化层用来缩小图片尺寸。每段内的卷积核数量一样，越靠后的段的卷积核数量越多：64-128-256-512-512。其中经常出现多个完全一样的 3*3 的卷积层堆叠在一起的情况，这其实是非常有用的设计。如图 12 所示，两个 3*3 的卷积层串联相当于 1 个 5*5 的卷积层，即一个像素会跟周围 5*5 的像素产生关联，可以说感受野大小为 5*5。而 3 个 3*3 的卷积层串联的效果则相当于 1 个 7*7 的卷积层。除此之外，3 个串联的 3*3 的卷积层，拥有比 1 个 7*7 的卷积层更少的参数量，只有后者的 $(3*3*3)/(7*7)=55\%$ 。最重要的是，3 个 3*3 的卷积层拥有比 1 个 7*7 的卷积层更多的非线性变换（前者可以使用三次 ReLU 激活函数，而后者只有一次），使得 CNN 对特征的学习能力更强。

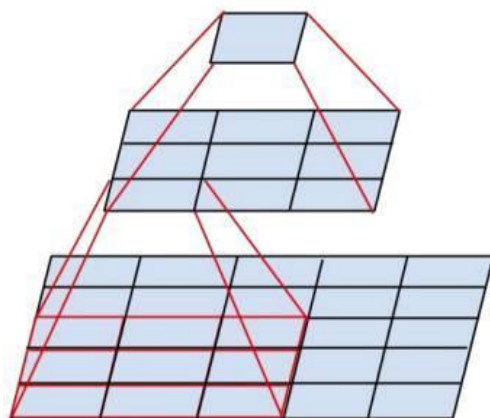


图 15 两个串联 3*3 的卷积层功能类似于一个 5*5 的卷积层

1.3.3 ResNet

深度残差网络 (Deep residual network, ResNet) 的提出是 CNN 图像史上的一件里程碑事件, ResNet 在 ILSVRC 和 COCO2015 上取得了 5 项第一, 并又一次刷新了 CNN 模型在 ImageNet 上的历史。ResNet 的作者何恺明 (<http://kaiminghe.com/>) 也因此摘得 CVPR2016 最佳论文奖。那么 ResNet 为什么会有如此优异的表现呢? 其实 ResNet 是解决了深度 CNN 模型难训练的问题, 从图中可以看到 14 年的 VGG 才 19 层, 而 15 年的 ResNet 多达 152 层, 这在网络深度完全不是一个量级上, 所以如果是第一眼看这个图的话, 肯定会觉得 ResNet 是靠深度取胜。事实当然是这样, 但是 ResNet 还有架构上的 trick, 这才使得网络的深度发挥出作用, 这个 trick 就是残差学习 (Residual learning)。

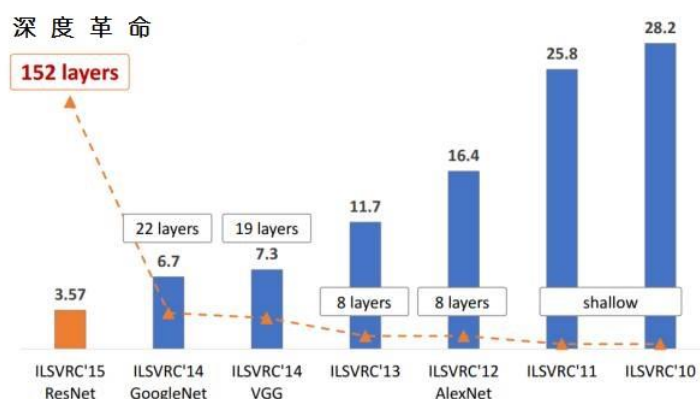


图 16 ImageNet 分类前五名误差

关于深层网络，在训练网络方面有一个重要的问题：退化问题。退化问题简单来说就是随着层数的加深到一定程度之后，越深的网络反而效果越差，并且并不是因为更深的网络造成了过拟合，也未必是因为梯度传播的衰减，因为已经有很多行之有效的方法来避免问题。关于这一点，原文并没能很好地论证，其实梯度的衰减仍然存在。

何恺明博士提出了残差学习来解决退化问题。对于一个堆积层结构（几层堆积而成）当输入为 x 时其学习到的特征记为 $\mathcal{F}(x)$ ，现在我们希望其可以学习到残差，这样其实原始的学习特征是 x 。之所以这样是因为残差学习相比原始特征直接学习更容易。当残差为 0 时，此时堆积层仅仅做了恒等映射，至少网络性能不会下降，实际上残差不会为 0，这也会使得堆积层在输入特征基础上学习到新的特征，从而拥有更好的性能。残差学习的结构如图所示。这有点类似与电路中的“短路”，所以是一种短路连接 (shortcut connection)

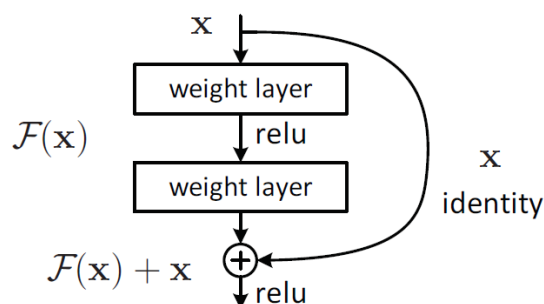


图 17 残差学习单元

从直观上看残差学习需要学习的内容少，因为残差一般会比较小，所以残差学习相对更容易。我们也可以自己从数学的角度来分析这个问题。

ResNet 网络是参考了 VGG19 网络，在其基础上进行了修改，并通过短路机制加入了残差单元，如图 15 所示。变化主要体现在 ResNet 直接使用 stride=2 的卷积做下采样，并且用 global average pool 层替换了全连接层。ResNet 的一个重要设计原则是：当 feature map 大小降低一半时，featuremap 的数量增加一倍，这保持了网络层的复杂度。从图 5 中可以看到，ResNet 相比普通网络每两层间增加了短路机制，这就形成了残差学习，其中虚线表示 featuremap 数量发生了改变。图 5 展示的 34-layer 的 ResNet，还可以构建更深的网络如表 1 所示。从表中可以看到，对于 18-layer 和 34-layer 的 ResNet，其进行的两层间的残差学习，当网络更深时，其进行的

是三层间的残差学习，三层卷积核分别是 1x1, 3x3 和 1x1，一个值得注意的是隐含层的特征映射数量是比较小的，并且是输出特征映射数量的 1/4。

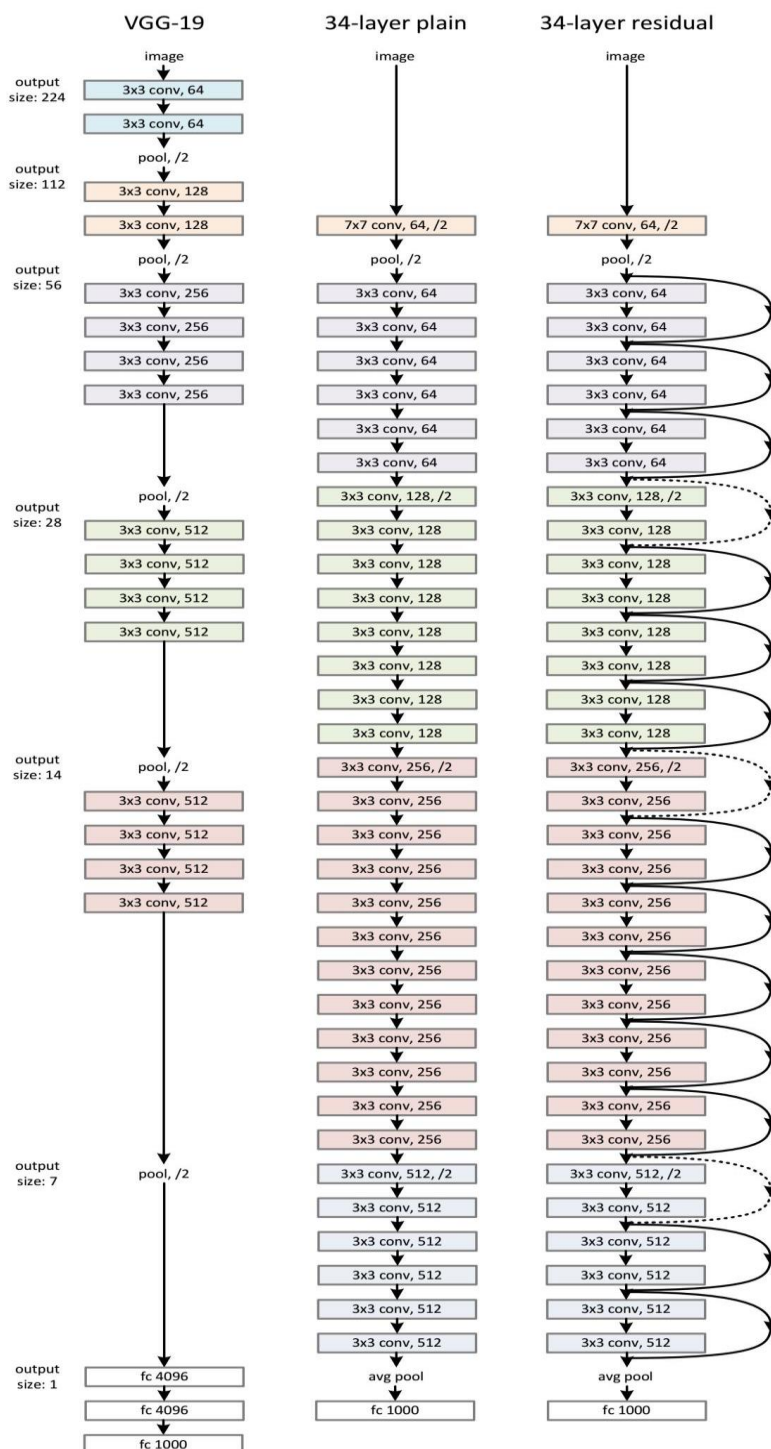


图 18 ResNet 网络结构图

1.4 深度学习框架

1.4.1 Caffe

Caffe 是作者贾扬清在 2013 年下半年利用空闲时间实现的。期初纯粹是一个业余的项目，但在 2013 年 12 月开源之后，由于其高效、简洁，最终成为了一个在深度学习领域有影响力的学习框架。

版本说明：

由于各个版本的不同，可能源码稍微有些不同，常用的版本有以下几个：

- 最原始的最开始版本：伯克利 BVLC 版 <https://github.com/BVLC/caffe>
- 贾扬清自己的版本：<https://github.com/Yangqing/caffe>
- caffe-for-windows 基本版：<https://github.com/niuzhiheng/caffe>
- caffe window 改进版运行在 VS2013 :
<https://github.com/initialneil/caffe-vs2013> 。 配 置 :
<https://initialneil.wordpress.com/>
- github 上面 happynear 的版本：<https://github.com/happynear/caffe-windows> 。 配 置 :
<http://blog.csdn.net/happynear/article/details/45372231>

设计思路

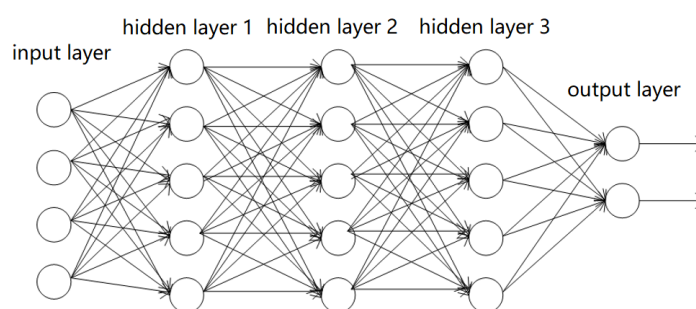


图 19 简单的神经网络示意图

上图是一个简单的神经网络示意图。神经网络由一层的神经元节点组成。训练数据从输入层 (input layer) 节点输入，中间的隐层 (hidden layer) 节点做一些运算，输出层 (output layer) 连接标注，就可以通过梯度下降法迭代更新网络权重，完成一次机器学习过程。在这个过程中，有几个要素，网络中的数据流，负责运算的

节点算子，层次化的网络结构，以及最后的优化算法。Caffe 把这 4 个部分进行抽象和封装，就形成了一套深度学习的计算框架。简单的说，Caffe = 数据流 + 节点算子 + 网络结构 + 优化算法。

实现架构和源码解析

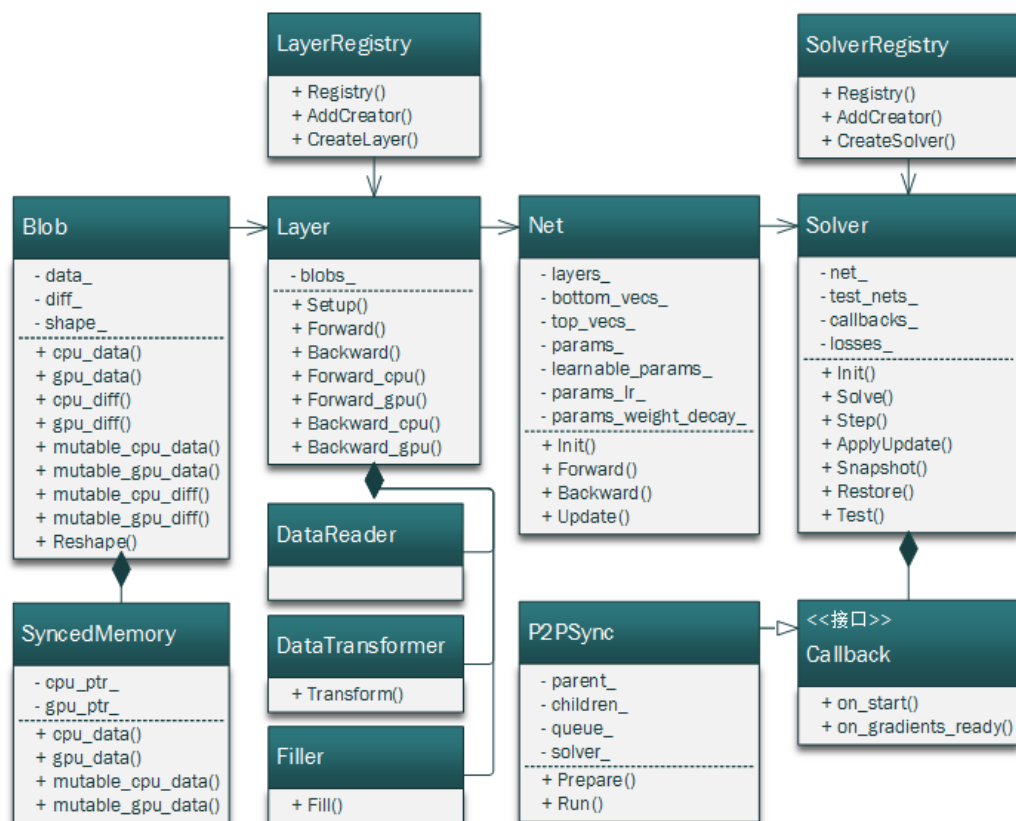


图 2 caffe 架构图

上图是分析 Caffe 源码之后总结出的架构图，罗列了 Caffe 中主要的几个类和接口。对照 Caffe 的官方文档，把这个图理清楚，基本就可以掌握 Caffe 的实现原理了。

Caffe 中用 Blob 表示数据，类似于 Numpy 中的 ndarray，可以理解为一个 n 维数组。深度学习计算过程中，数据、参数、残差等都用这个类型表示。data_ 字段是数据，diff_ 字段用来辅助存储残差，shape_ 字段是数据的维度形状。Blob 封装了 CPU 和 GPU 之间的数据同步问题，提供了 cpu_data/mutable_cpu_data 这样的接口。通过调用这些接口，开发者可以不用关心计算过程中数据到底是存在 CPU 上还是 GPU 上，两个设备上的数据是否同步等问题。

Caffe 中用 Layer 表示神经网络中的一层，通过定义不同的 Layer 来实现不同的算子。目前 Caffe 已经提供了大量内置的算子，例如数据读取，损失函数，卷积，激活函数等。如果需要实现自定义的运算，开发者可以实现自己的 Layer，通过 LayerRegistry 注册到 Caffe 中，就可以调用了。

Caffe 中用 Net 表示网络结构。Net 存储了网络中所有的层，以及层的输入输出，参数等等。Net 中提供了 Forward，Backward，Update 接口，对应神经网络优化过程中的前向传播，反向传播，参数更新操作。

Caffe 中用 Solver 表示优化算法。Solver 中的 Step 函数调用 Net 中的接口实现一轮参数迭代，Snapshot 和 Restore 提供了模型快照的存取功能。

使用方法

Caffe 提供了方便快捷的命令行调用方式，一般分为下面几个步骤：

- 准备数据，将数据转换为 Caffe 识别的格式
- 编写网络结构，可以参照 `caffe/src/caffe/proto/caffe.proto` 中对于 NetParameter 的定义
- 编写训练配置，可以参照 `caffe/src/caffe/proto/caffe.proto` 中对于 SolverParameter 的定义
- 训练，Caffe 针对常见的场景提供了三种命令
 - 开始训练：`caffe train -solver=solver.prototxt`
 - 继续被中断的训练：`caffe train -solver=solver.prototxt -snapshot=net.solverstate`
 - 使用预训练的模型初始化参数：`caffe train -solver=solver.prototxt -weights=net.caffemodel`
- 预测：`caffe test -model=solver.prototxt -weights=net.caffemodel`

除命令行之外，Caffe 也提供了 Python 接口。可以更自由的构建网络和控制训练过程，使用起来也很方便。具体的应用举例可以参考官方文档。

主要特点

- CPU 和 GPU。Caffe 既可以在 CPU 上运行，也可以在 GPU 上运行。
- 模块化。Caffe 设计之初就做到了尽可能的模块化，允许对数据格式、网络层和损失函数进行扩展。

- 表示和实现分离。Caffe 的模型定义是用 Protocol Buffer 语言写进配置文件的，以任意有向无环图的形式，Caffe 支持网络架构。Caffe 会根据网络的需要来正确占用内存。通过一个函数调用，实现 CPU 和 GPU 之间的切换。
- 测试覆盖。在 Caffe 中，每一个单一的模块都对应一个测试。
- Python 和 MATLAB 接口。同时提供 Python 和 MATLAB 接口。
- 预训练参考模型。针对视觉项目，Caffe 提供了一些参考模型，这些模型仅应用在学术和非商业领域，它们的 License 不是 BSD，而是 BSD 2-Clause。

优点:

- 很好地适用于前反馈网络。
- 很好地适用于 finetuning 网络。
- 不需要任何代码就可以训练模型。
- Python 接口使用便利。

缺点:

- 需要 C++/CUDA 写新的 GPU 层。

官方社区:

- 官方讨论: <https://groups.google.com/forum/#!forum/caffe-users>
- CaffeCN 深度学习社区_Caffe 中国用户社区: <http://www.caffecn.cn/>

使用平台:

Android 手机、IOS 手机、普通的 CPU 服务器、大规模 GPU 集群。

1.4.2 TensorFlow

TensorFlow 最初是由 Google Brain 团队（隶属于 Google 的 AI 部门）中的研究人员和工程师开发的，可为机器学习和深度学习提供强力支持，并且其灵活的数值计算核心广泛应用于许多其他科学领域。

版本说明:

TensorFlow 框架从 1.0 发展到了如今的 2.0。源码见：
<https://github.com/tensorflow/tensorflow/releases>。

概念思路

TensorFlow 是一个采用数据流图 (data flow graphs)，用于数值计算的开源软件库。节点 (Nodes) 在图中表示数学操作，图中的线 (edges) 则表示在节点间相互联系的多维数据数组，即张量 (tensor)。

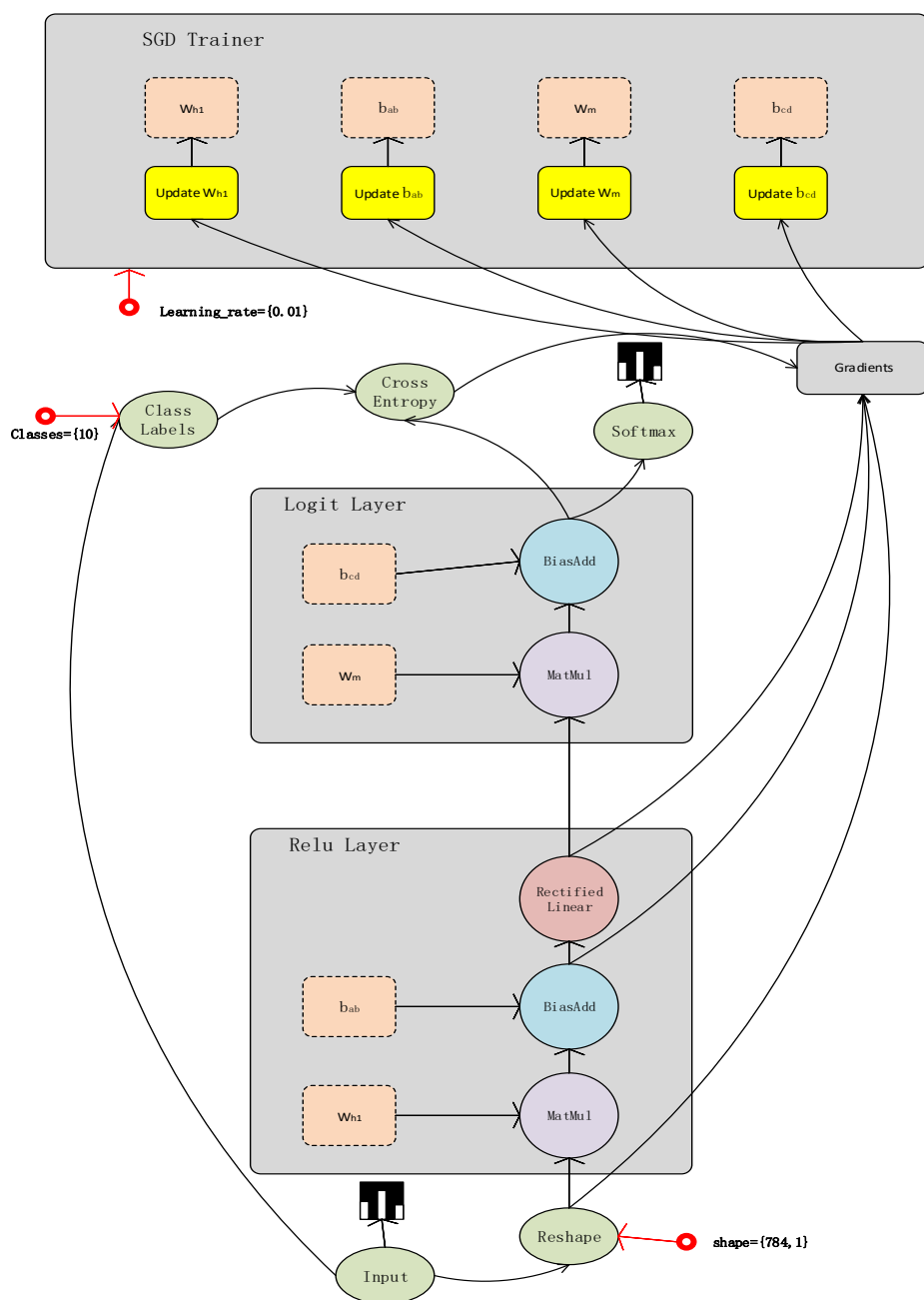


图 3 TensorFlow 数据流图

系统架构和设计理念

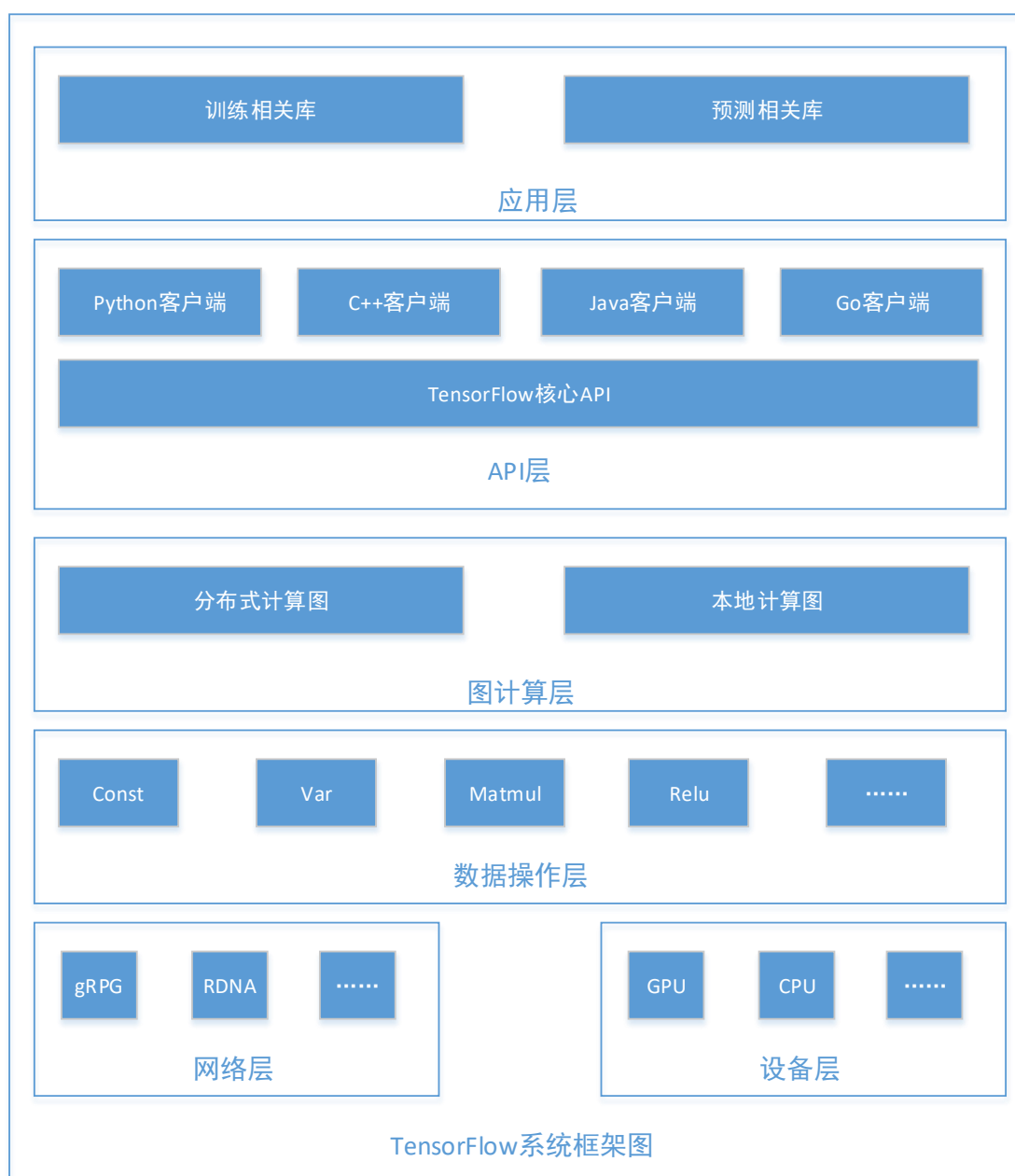


图 4 TensorFlow 架构图

设计理念:

将图的定义和图的运行完全分开，因此 Tensorflow 被认为是一个“符合主义”的库。

Tensorflow 中涉及的运算都要放在图中，而图的运行只发生在会话（session）中。开启会话后，就可以用数据去填充节点，进行运算。关闭会话后，就不能继续计算了。因此会话提供了操作运算和 Tensor 求值的环境主要特点及优缺点

主要特点:

- 高度的灵活性。TensorFlow 不是一个严格的“神经网络”库，只要能够将计算表示为一个数据流图，就可以使用 TensorFlow。
- 较强可移植。TensorFlow 可以在 CPU 和 GPU 上运行，也可以将模型在云端服务器或者 Docker 容器里运行。
- 自动求微分。TensorFlow 具有自动求微分的能力，只需要定义预测模型的结构，将结构和目标函数结合，并添加数据，TensorFlow 就能自动计算相关的微分导数。
- 多语言支持。TensorFlow 支持 C++、Python、Go、Java、Lua 和 JavaScript 等语言。
- 性能最优化。比如有一个 32 个 CPU 内核、4 个 GPU 显卡的工作站，由于 Tensorflow 给予了线程、队列、异步操作等以最佳的支持，Tensorflow 让你可以将你手边硬件的计算潜能全部发挥出来。你可以自由地将 Tensorflow 图中的计算元素分配到不同设备上，Tensorflow 可以帮你管理好这些不同副本。

优点:

- 可自行设计神经网络结构;
- 不需要通过反向传播求解梯度，Tensorflow 支持自动求导;
- 通过 C++ 编写核心代码，简化了线上部署的复杂度(通过 SWIG 实现 Python, Go 和 JAVA 接口);
- Tensorflow 中内置 TF.Learn 和 TF.Slim 等组件，并兼容 Scikit-learn estimator 接口(evaluate、grid、search、cross、validation);
- 数据流式图支持自由的算法表达，可实现深度学习以外的机器学习算法;
- 可写内层循环代码控制计算图分支的计算，可将相关的分支转化为子图并执行迭代计算;
- 可进行并行设计，充分利用硬件资源;
- 具有灵活的移植性，编译速度较快。

缺点:

- 过于复杂的系统设计。
- TensorFlow 在 GitHub 代码仓库的总代码量超过 100 万行。
- 频繁变动的接口。

- TensorFlow 的接口一直处于快速迭代之中，并且没有很好地考虑向后兼容性，这导致现在许多开源代码已经无法在新版的 TensorFlow 上运行，同时也间接导致了許多基于 TensorFlow 的第三方框架出现 BUG。
- 接口设计过于晦涩难懂。在设计 TensorFlow 时，创造了图、会话、命名空间、Placeholder 等诸多抽象概念，对普通用户来说难以理解。同一个功能，TensorFlow 提供了多种实现，这些实现良莠不齐，使用中还有细微的区别，很容易将用户带入坑中。
- 文档混乱脱节。TensorFlow 作为一个复杂的系统，文档和教程众多，但缺乏明显的条理和层次，虽然查找很方便，但用户却很难找到一个真正循序渐进的入门教程。

官方社区：

- Tensorflow 官方：<https://tensorflow.google.cn/>
- Tensorflow 中文社区：<http://www.tensorfly.cn/>

使用平台：

Android 手机、IOS 手机、普通的 CPU 服务器、大规模 GPU 集群。

1.4.3 Keras

Keras 是一个高层神经网络 API，Keras 由纯 Python 编写而成，并且是基于 TensorFlow、Theano 以及 CNTK 后端。

版本说明：

2015 年 Keras 1.0 发布，2017 年 Keras 2.0 发布，在 2.0 中增强了 Keras 与 TensorFlow 的逻辑一致性。

设计思路：

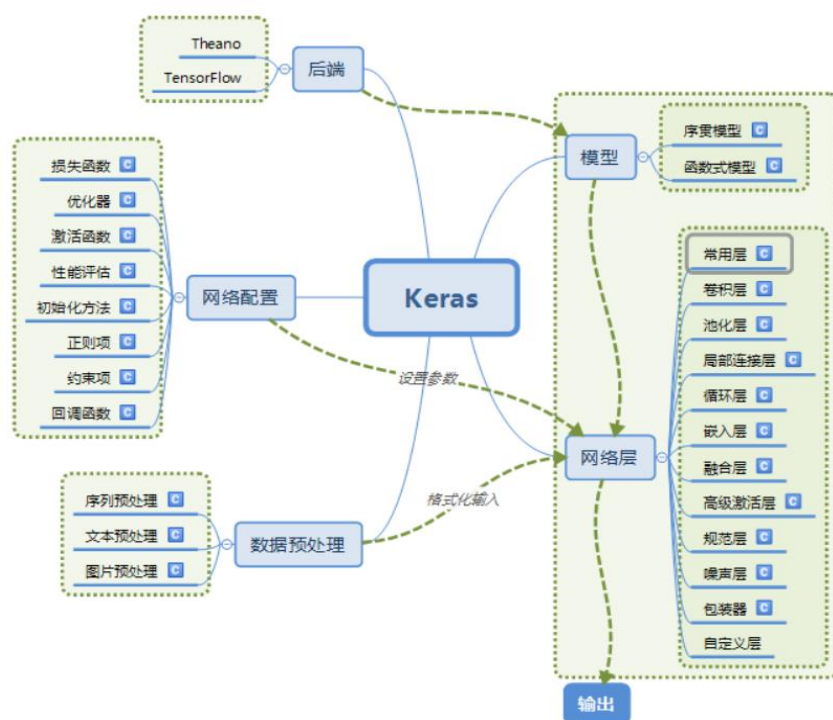


图 6 Keras 模块结构

主要特点:

- 使用简单，能够快速实现原理；
- 支持卷积网络和递归网络，以及两者的组合；
- 无缝运行在 CPU 和 GPU 上；支持任意连接方式，包括多输入多输出训练。

优点:

- 用户友好：Keras 遵循减少认知困难的最佳实践：Keras 提供一致而简洁的 API，能够极大减少一般应用下用户的工作量，同时，Keras 提供清晰和具有实践意义的 bug 反馈。
- 模块性：模型可理解为一个层的序列或数据的运算图，完全可配置的模块可以用最少的代价自由组合在一起。具体而言，网络层、损失函数、优化器、初始化策略、激活函数、正则化方法都是独立的模块，你可以使用它们来构建自己的模型。
- 易扩展性：添加新模块超级容易，只需要仿照现有的模块编写新的类或函数即可。创建新模块的便利性使得 Keras 更适用于先进的研究工作。
- 与 Python 协作：Keras 没有单独的配置模型文件类型，模型由 python 代码描述，使其更紧凑和更易 debug，并提供了扩展的便利性。

缺点:

- 速度慢。Keras 本身是一个中间层，通过它调用 Tensorflow 或 theano，会比单独使用 Tensorflow 或 theano 要慢。
- 程序占用 GPU 内存比较多。

官方社区:

使用中遇到的困惑或者问题可以提交到 github :
<https://github.com/fchollet/keras>

使用平台:

Android 手机、IOS 手机、普通的 CPU 服务器、大规模 GPU 集群。

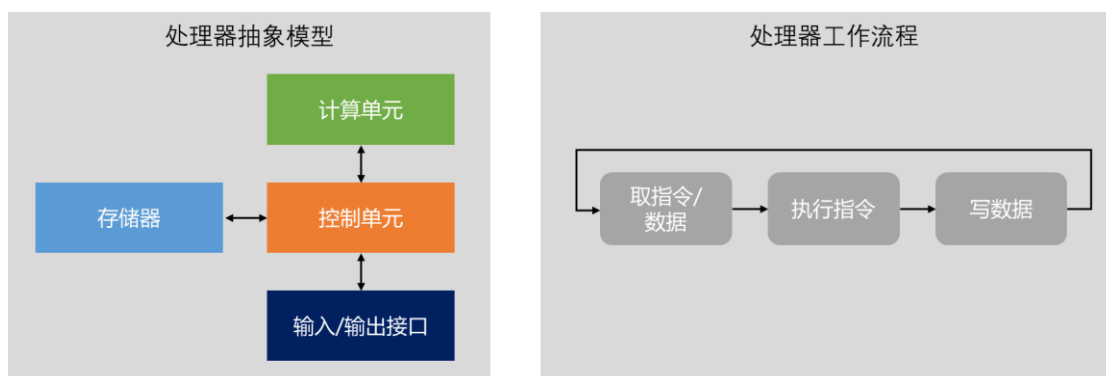
PART II. EEP-TPU 张量处理器

2.1 处理器架构

2.1.1、处理器

处理器技术十分复杂，是人类科技进步的重大体现之一。然而，处理器的抽象模型却十分简单：（1）处理器都由存储器、输入/输出接口、控制单元和计算单元组成；（2）处理器循环执行下列操作：“取指令/数据、执行指令、写数据”；（3）处理器的行为完全由指令和数据决定。无论处理器多复杂，无论是 CPU、GPU 或 DSP，上述模型全部适用。

这个处理器抽象模型就是著名的“冯诺依曼结构”，其核心是把用于控制的程序当作数据来存储，这种基于存储程序的计算模型一直沿用至今，无论半导体工艺多先进，处理器结构多复杂，存储程序型计算从未改变。



2.1.2、并行计算

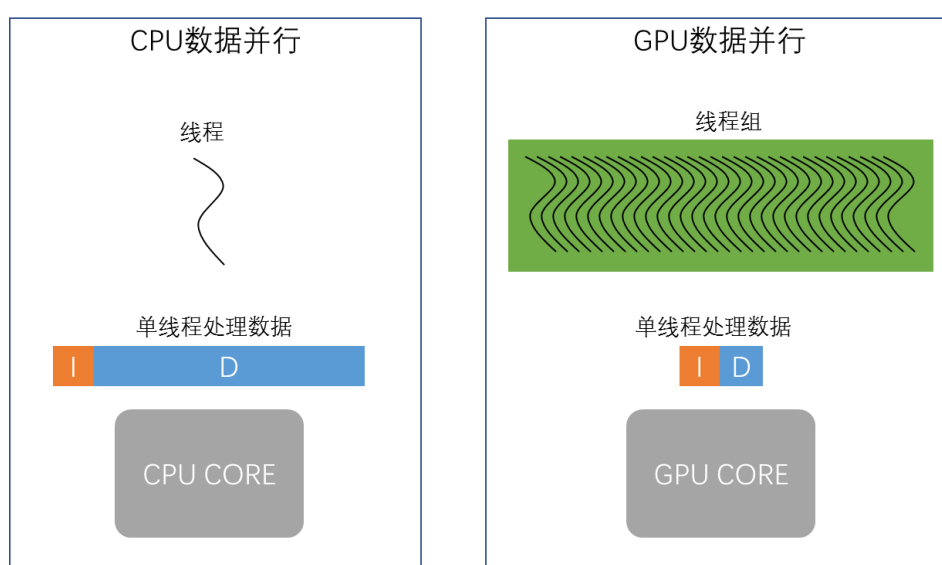
在存储程序计算中，指令和数据是所有操作的核心，直观的，按照指令和数据来划分，传统计算体系结构可以分为四类：

- SISD（单指令单数据）：最早的计算体系结构，任何时刻，只有一条指令执行，处理一个数据。

- SIMD（单指令多数据）：一种并行计算体系，任何时刻，只有一条指令执行，处理多个数据。大多数现代处理器都拥有这类体系结构扩展（例如 ARM NEON 扩展指令和 X86 MMX/SSE 扩展指令）。
- MISD（多指令单数据）：多条指令处理一个数据，目前未被普遍使用。
- MIMD（多指令多数据）：一种并行计算体系，多个核心（运行不同指令）处理多个数据，大多数 MIMD 体系实际由包含 SIMD 的多个核心组成。

然而，随着数据密集型任务的出现，数据并行成为计算性能的关键瓶颈。SIMD 架构是用于增加数据并行的直观选择，然而，把多个数据同步打包成一个向量数据并用一条指令执行，这极大的限制了数据并行度的发掘。

对此，英伟达提出了 SIMT（单指令多线程）架构。相比于 SIMD，SIMT 的数据由不同线程维护，数据之间是完全异步的关系，各自完全独立，可以实现大量异步数据的完全并行，也即线程级的数据并行。这样的架构极大的增加了数据的并行维度。典型的，1 个 16 核现代先进 CPU 通常只能同时执行 16 或 32 个线程，而一个现代先进 GPU 同时执行的线程数高达几千个。



显而易见的，在存储程序计算中，提高计算性能，就是提高指令和数据的执行性能，在过去的近 50 年发展历程中，以英特尔、英伟达为代表的美国企业引领了处理器技术的重要发展。根据计算任务的特点：指令密集型或数据密集型，处理器架构也按照指令优化和数据优化两大方向发展，并衍生出 CPU、GPU 两大处理器类型。

CPU 是最早的处理器，其技术发展主要面向指令执行效率的优化，包括更高的频率、更高效的指令集（RISC）、更多的指令级并行（超标量）、更多的任务级并行（超线程、多核）等。

GPU 是随着数据密集型任务的增多而逐渐发展起来的处理器，其技术发展主要面向数据执行效率的优化，包括更多核心数，更多线程（SIMT）、更高效的内存结构，更高效的编程模型等。

在通用并行计算这条路上，CPU\GPU 架构探索了近 50 年，拥有一系列复杂的“组合拳”来完成多种粒度的并行计算，从而实现最高能效比的高性能计算，软硬件技术壁垒之高，很难打破。

3、编程模型

从一开始，计算机编程就存在两种模型，一种模拟人类行为结果，一种模拟人类大脑。

- 模拟人类行为结果的编程模型（称为传统编程模型），本质是基于人类认知的数学抽象进行编程。在该模型下，计算机的一切行为由人类的抽象思维决定，人类编写的程序代码变成确定的执行序列，并被特定的硬件使用。
- 模拟人类大脑的编程模型（称为神经网络编程模型），本质是基于人类大脑的生物抽象进行编程。在该模型下，计算机的一切行为由神经网络结构和知识参数决定，训练获得的知识通过数据的形式存储，并被特定硬件使用。

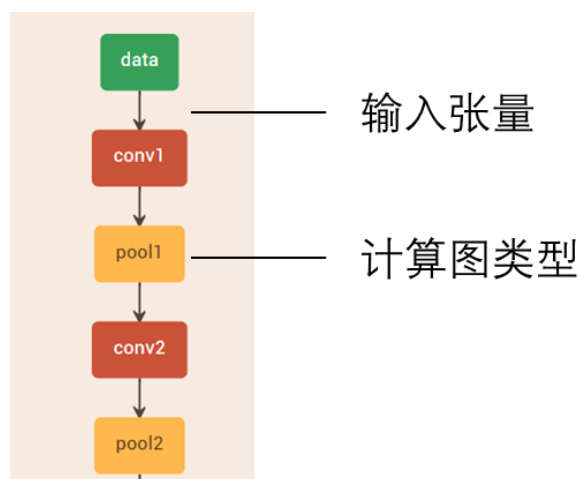
在过去的 70 年间，由于各种原因，模拟人类行为结果的编程模型得到蓬勃发展，并成为如今主流，目前几乎所有软件编程都属于此类。而模拟人类大脑的编程模型则历经几次浪潮与寒冬，进展缓慢，目前基于神经网络/深度学习技术的编程属于此类。

CPU/GPU 是基于传统编程模型打造的处理器。CPU/GPU 也可以运行神经网络算法，但这是通过把神经网络算法转换成传统编程模型后实现的。

大量事实证明，神经网络编程模型十分重要，是下一代智能计算体系的核心关键。如此重要的体系，难道不需要一种比 CPU、GPU 更高效的架构来执行吗？

2.2 EEP-TPU 张量处理器架构

神经网络编程模型的本质是计算图模型，如下图所示，计算图的输入/输出是张量数据，计算图的类型代表操作类型。因此，直观的，最适合于神经网络编程模型的计算体系结构，是 Graph/Tensor 计算体系，其中，处理器的功能由计算图类型决定，而数据则是计算图的输入/输出张量。

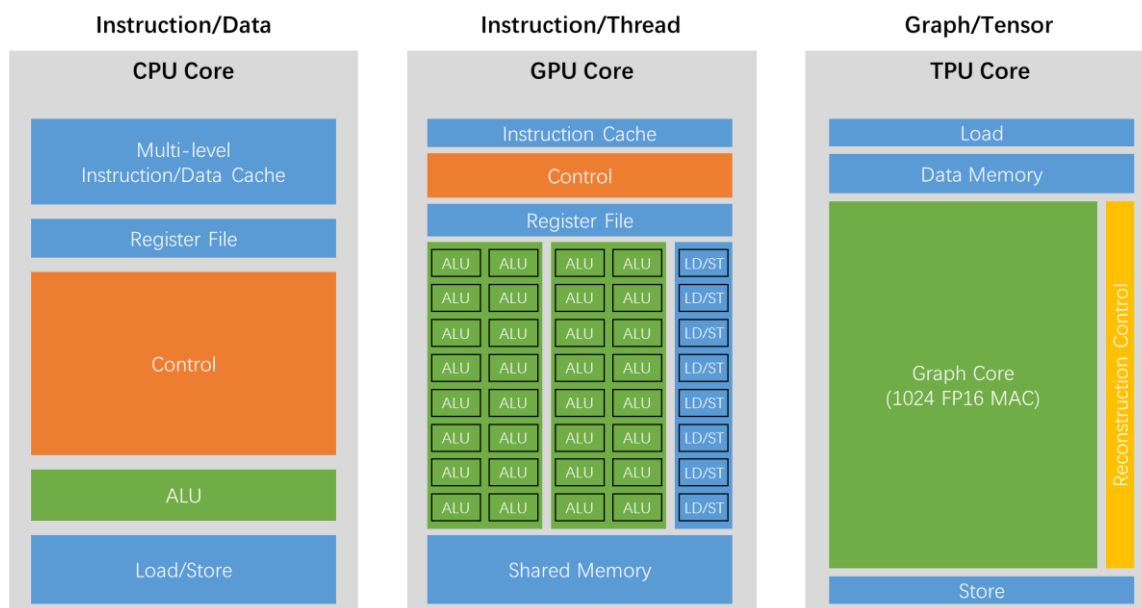


然而，计算图这一层级的粒度太粗，各类型间并没有太大的相关性，一个典型的神经网络计算由 Convolution、Pooling、BN、Scale、RELU 等组成，它们之间的行为差异巨大，如果处理器按照计算图操作的粒度去设计，这就意味着需要为每一个计算图操作（或某几个）设计专门的计算硬件（正如 NVIDIA DLA 那样，NVDLA 为卷积、池化和 BN 专门设计了不同的计算电路），这样的代价是巨大的，而且也不具备可扩展性。

EEP-TPU 所采用的张量运算集用一种巧妙的方式解决了 Graph/Tensor 体系结构中的核心问题：如何用一套标准、完善且最大粒度的操作（粒度远远高于传统 CPU/GPU 的加、减、乘、除，同时也是各计算图的子集）来定义基础运算类型。

在张量运算集中，张量运算被划分为 1 维、2 维、3 维和 4 维四大种类，每个种类又由各种不同类型的计算操作组成，从而形成一个完备的计算集合，通过这些张量运算操作的组合，便能在相同硬件架构下，完成如卷积、DW 卷积、Pooling、Resize、Relu 等计算图操作。

CPU、GPU、EEP-TPU 的处理器架构对比如下图所示：



相比于传统 CPU 的 Instruction/Data 和 GPU 的 Instruction/Thread 架构，EEP-TPU 的 Graph/Tensor 架构有如下明显优势：

- 编程模型直观且简单。直接面向神经网络编程模型，使用计算图模型作为原始输入，不存在模型转换和传统 CPU/GPU 软件并行编程的优化与匹配问题，在合理的编译器分配下，便可最大化利用硬件资源。
- 无指令（计算图类型的切换，最终通过电路重构来实现，实际计算过程中并无任何指令开销）。CPU 几十年架构演变的首要目标，就是增大指令执行效率，减少指令调度和切换的开销。尽管 GPU 架构对指令调度的依赖较小，但其数据计算仍然需要在指令的调度下统一工作。EEP-TPU 的无指令架构完全消除了由指令引入的计算性能（由指令调度和切换引入）和计算资源（由指令控制和存储引入）的开销。
- 内存带宽需求低。传统 CPU/GPU 架构是一种基于指令的计算体系，内存带宽需求高，例如，一个 32 位加法操作，需要多次指令和数据的内存访问才能完成。在 EEP-TPU 的 Graph/Tensor 架构下，每个数据只需从内存读出一次，便可完成它生命周期内的所有计算操作，存储器访问频次降低到每个数据一次，相比传统 CPU/GPU 完成相同工作要小得多，因此内存带宽的需求比传统 CPU/GPU 小得多。
- 片上缓存容量低。传统 CPU/GPU 架构需要引入多层级的存储结构来解决“存储墙”问题。由于 EEP-TPU 无指令，且内存带宽需求低，因此只需一个小容量的片上缓存便可满足计算过程中的数据缓存需求。

针对边缘、终端、传感器场景，通常一个 EEP-TPU Core 的算力就能满足绝大多数应用的需求（28nm 工艺下最高算力为 2T FP16 FLOPS）。如果需要更高性能，则可采用多核的方式实现性能扩展。特别的，在 Xilinx zynq 7020 上部署 EEP-TPU，算力约为 0.03T FP16 FLOPS。在这样的算力水平下，已经可以实现最高 30FPS 的实时目标检测任务（基于 YOLOV3）。而在多核架构下，算力可以轻松达到 32T FP16 FLOPS（英伟达 P100 算力为 18.7T FP16 FLOPS），如下图所示：

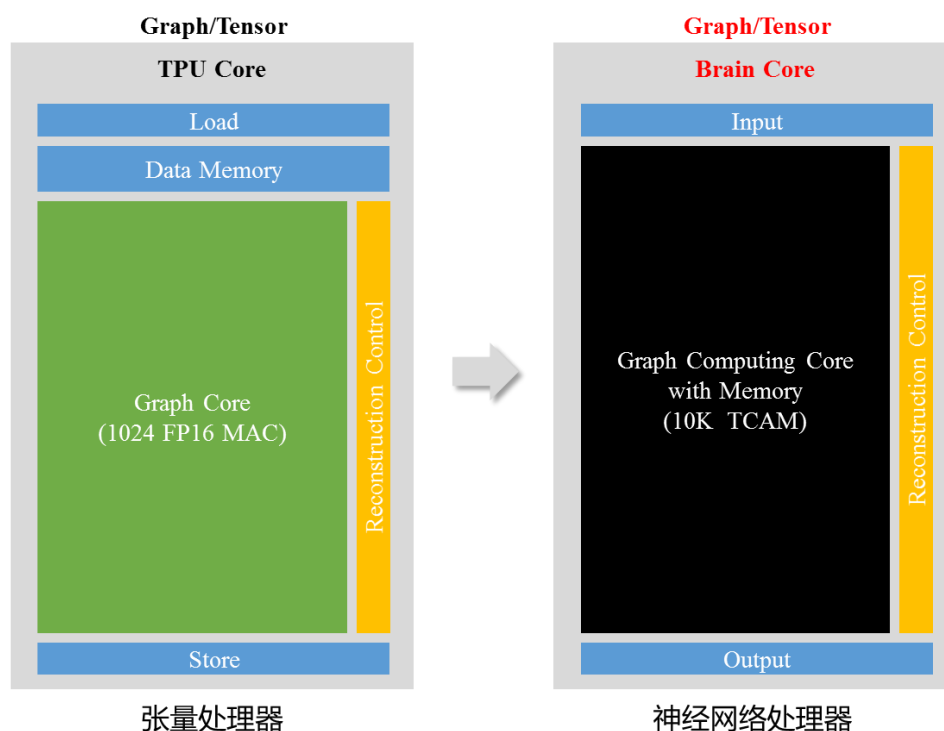


2.3 EEP-TPU 神经网络处理器架构

上一节所介绍的张量处理器，从集成电路设计与集成电路工艺的角度出发，仍然采用与 CPU、GPU、DSP 等传统芯片几乎相同的计算内核：乘法器电路。尽管体系结构的优化可以使得乘法器计算内核的使用效率增加，但其仍然无法摆脱乘法器单元所带来的“性能天花板”。神经网络处理器就是为了突破这种限制而产生的全新电路结构。

简单来讲，神经网络处理器就是希望使用全新的模拟电路形态的乘法器来代替传统的数字电路形态的乘法器，并且把乘法器的计算单元与存储单元进行叠加，实现存储与计算的一体化，从而达到 1~2 个数量级的性能提升。

一种神经网络处理的架构演变如下图所示。



EEP-TPU 神经网络处理器属于全新的下一代产品，可以做到现有软件体系架构下对张量处理器的全新升级与替换，目前仍处于验证阶段。

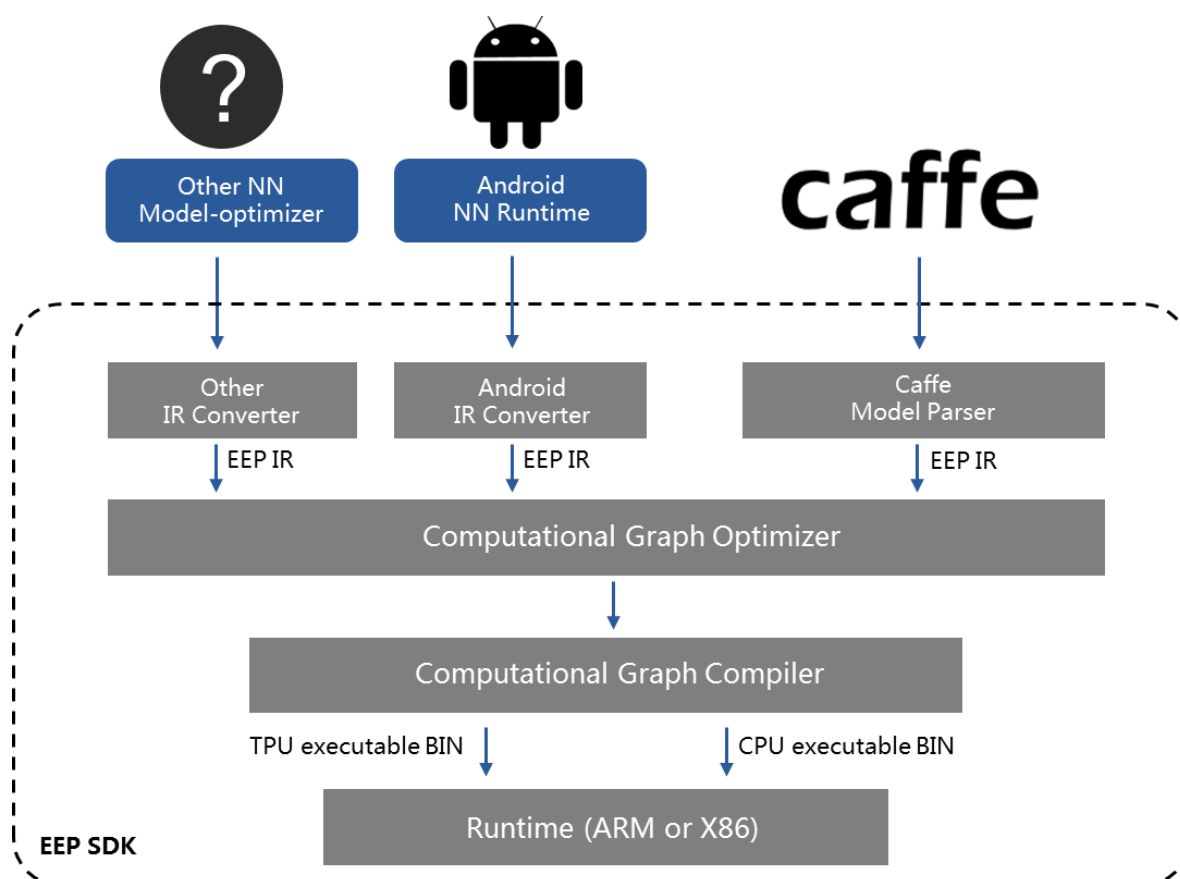
2.4 EEP-TPU 异构计算

在一个完整的人工智能芯片产品中，通常包含多种不同架构的处理器核心，最基本的，至少包含一个 CPU 和一个 TPU。

对于一个包含 CPU 与 TPU 的人工智能系统来讲，CPU 与 TPU 可以完成相同的计算任务，唯一的区别是能耗、时延、资源占用等性能的不同。在这里特别要注意的是，尽管 TPU 是专门针对人工智能计算而设计的处理器，对于深度学习算法中的某种特定计算类型来说，TPU 的计算并不一定有绝对的性能优势。例如，TPU 更擅长于处理数据流计算类型的任务，而并不擅长仅通过坐标变换就能实现的控制类型的任务，而这类控制类型的任务交由 CPU 实现具有更高的效率。

因此，由不同类型的处理器组成，并且各处理器发挥不同的优势共同完成某种计算，成为了一个高效率人工智能计算系统的关键，这就是异构计算系统开发的初衷。

EEP-TPU 拥有一套完整的异构计算支撑体系。该支撑体系由编译器+异构计算框架组成，可以支持由 CPU+TPU 组成的异构计算系统的高效计算。其中，编译器的作用是分析来自用户输入的深度学习算法，并根据某种选择和规则，把深度学习算法进行 CPU 和 TPU 计算的切割，并分别进行优化与编译，最终生成 CPU 可执行文件和 TPU 可执行文件。EEP-TPU 编译器框图如下。

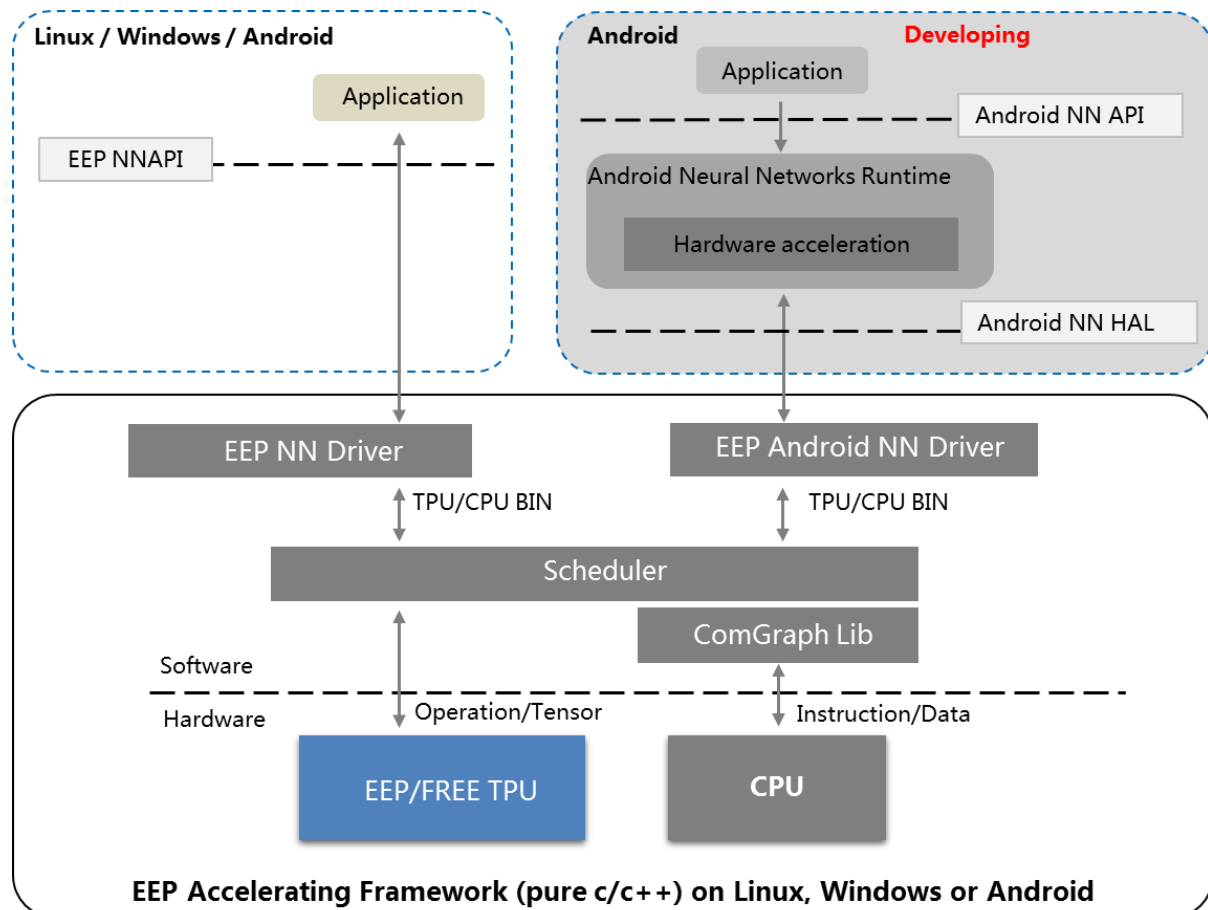


TPU 可执行文件和 CPU 可执行文件在实际工程中，将被打包成一个独立的 BIN 文件，由运行在终端目标系统的异构加速框架读取并执行。

EEP-TPU 的异构计算框架是一套由 C/C++ 语言编写的，运行在目标操作系统之上的底层软件系统，该软件系统读入来自编译器的可执行文件，并根据可执行文件的内容，自动完成 CPU 计算与 TPU 计算的资源分配、任务调度、数据转换、数据传输等工作，从而实现异构计算。

EEP-TPU 的异构计算框架封装了各种类型的深度学习计算层，极端情况下，可以不使用 TPU，仅使用 CPU 便可完成所有的深度学习计算（以极低的效率）。因此，

从用户的角度来看，该异构计算框架几乎可以承载所有类型层的计算，从而实现绝大多数的深度学习算法，并通过使用 EEP-TPU，自适应的完成深度学习计算的高效加速。在这过程中，用户对于该异构计算框架的使用几乎是无感的。



III. EEP-TPU 开发流程

3.1 开发流程介绍

3.1.1 背景知识

EEP-TPU 的开发流程与传统的嵌入式开发流程类似，拥有 Host（主机）和 Device（从机）两类设备。EEP-TPU 的开发流程通常也被分为两部分：**神经网络算法开发流程**和**嵌入式应用开发流程**。

神经网络算法的开发通常在 Host 主机上使用 GPU 加速完成，其简要的流程如下：开发人员在 Host 主机上进行算法开发工作（训练）；开发（训练）完成后，通过交叉编译的方式，在主机环境下使用交叉编译工具（针对 Device 从机的编译工具）完成神经网络算法的(从机)可执行文件编译工作；最终通过某种通讯方式完成该可执行文件的下载，从而实现神经网络算法在嵌入式设备端的部署。典型的，Host 主机是一台包含 GPU 的服务器或高性能 PC，Device 从机是一个 ZYNQ 开发板。

另一方面，通常情况下，当落地到具体场景解决某个具体问题时，神经网络算法通常仅是整体解决方案的一小部分，其他的算法和流程需要传统编程模型来实现。因此，当 EEP-TPU 匹配某种 CPU（如 ZYNQ 7020 的 ARM）及其操作系统时（如 Linux），该从设备的应用开发就需要按照该 CPU 及操作系统的环境来执行。一个典型的案例，当 EEP-TPU 配合运行 Linux 操作系统的 ARM 处理器时（如 ZYNQ 7020 开发板），从设备的应用开发环境需要按照 Linux 应用开发的环境来部署；另一个典型的案例，当 EEP-TPU 配合运行 Free-RTOS 操作系统的 RISC-V 处理器时，从设备的应用开发环境需要按照 Free-RTOS 应用开发的环境来部署。

因此，当 EEP-TPU 配合某个 CPU 成为嵌入式 SoC 方案时，EEP-TPU 的应用开发流程实际上便成为了基于某个 CPU 架构在某种操作系统环境下的嵌入式开发流程。目前，EEP-TPU 可支持 Linux、Free-RTOS、Bare-metal（裸机，无操作系统）三种应用开发环境。嵌入式软件的开发流程在这里就不再详细阐述，具体请参考相关书籍和文档。需要注意的是，EEP-TPU 的开发流程与嵌入式从机的 CPU 架构无关，无论是 ARM 还是 RISC-V，其流程都是相同的。

3.1.2 人工智能应用开发流程

基于 EEP-TPU 的人工智能应用开发可归纳为以下几个步骤：

(1) 准备数据集

在传统的有监督的神经网络算法中，数据集对最终的算法性能起着决定性的关键作用。因此，神经网络算法训练的首要任务，就是实现数据集的搜集工作。

在机器视觉任务中，有监督神经网络算法的数据集由图片及其对应的标注信息组成。其中，图片通常来自实际场景，而对应的标注信息则根据不同的机器视觉任务大致分为三类：（1）分类标注，即该图片属于 N 类中的哪一类；（2）定位标注，即给出图片中 K 个物体的标注信息，该信息通常包含坐标（两个顶点的坐标 x_1, y_1, x_2, y_2 ）、类别（N 类中的哪一类）等；（3）像素分割标注，即给出每个像素点属于 N 类中的哪一类。

一个合格的数据集，每个种类的数量通常在 1k~100k 之间。著名的 ImageNet 数据集包含 1000 类，每类约 1000 张，该数据集的图片总数量高达百万。因此，给每张图进行“人工标注”，成为了数据集准备中最为繁杂的一项工作，通常需要耗费大量的人力、时间和金钱。

需要注意的是，在这里，数据集所包含的图片是神经网络算法的输入，而数据集中的标注则是神经网络算法的输出。“人工标注”的过程可以看作一种人“教”机器的过程，也即通过人类的先验知识，把事物在图像中所反映出来的正确属性（如类别、坐标等）变成机器可识别的文本形式，从而使得机器可以在后续的训练过程中使用。

(2) 训练

数据集准备完毕后，便可开始算法训练工作，通常情况下，算法训练在 Linux 环境下使用带 GPU 的 X86 架构服务器或高性能 PC 来完成。

在机器视觉任务中，神经网络算法可以解决的问题大致分为三类：**分类**、**检测**、**分割**。如今，在全球科研机构和大企业的共同努力下，针对这三类机器视觉任务有无数优秀的开源项目可以参考和使用。例如谷歌所开源的用于分类的 Mobilenet 算法，Darknet 所开源的用于目标检测的 YOLO 算法，香港中文大学所开源的用于像素级分割的 ICNet 算法等。

选定参考算法后，另一个重要的任务，就是选择深度学习框架。深度学习框架是一种用于神经网络算法开发的工具，其主要作用，是根据神经网络结构，以数据集中的图片和标注为输入，计算得到与之对应的权重参数。神经网络结构+对应参数便构

成了神经网络算法，在应用阶段（部署阶段），输入图片经过神经网络算法的计算便可最终得到与训练集所标注类似的信息，例如该图片输入哪一类物体，或者该图片内含什么物体且每个物体的坐标位置。

当数据集准备完毕（根据深度学习框架的要求转换成对应格式）、神经网络结构准备完毕（以文本形式按照某种语法描述），神经网络算法的训练过程几乎是自动的（约等于执行某个训练命令）。算法开发人员需要根据训练过程中所反映出来的各种数据来决定如何优化和改进神经网络结构，最终达到应用的需求。

当训练完毕后，通常会得到神经网络结构和权重参数两个文件（也可能合成一个文件），这两个文件将会被后续的算法部署阶段使用。

(3) EEP-TPU 算法编译

神经网络算法的开发工作通常在 X86 架构的服务器上完成，而 EEP-TPU 则是一种与 X86 完全不同的计算架构。因此，上述训练所得的神经网络结构和权重参数文件，需要按照交叉编译的方式，在 X86 架构服务器上使用 EEP-TPU 编译器，把这两个文件编译成为 EEP-TPU 架构可以识别和使用的文件，这就是 EEP-TPU 算法编译过程。

EEP-TPU 编译器仅支持 CAFFE 框架所输出的格式，其中神经网络结构由 .prototxt 为扩展名的文件描述，权重参数由 .caffemodel 为扩展名的文件描述。EEP-TPU 编译器可以直接使用 CAFFE 输出的 .prototxt 和 .caffemodel 两个文件作为输入，从而编译得到 EEP-TPU 架构所使用的可执行文件（BIN 文件），该可执行文件可在后续的嵌入式应用开发中所调用。

(4) EEP-TPU 嵌入式应用开发

EEP-TPU 张量处理器以知识产权 IP 的形式可以存在于多种不同的系统中。一个典型案例，在 EEP-TPU + ARM CPU 组成的 SoC 系统中，EEP-TPU 以加速器的形式存在，专用于执行神经网络算法，而 ARM CPU 则可以运行 Linux 操作系统、执行网络任务、维护用户界面等。在这样的基于 ARM 的 CPU 系统中，EEP-TPU 的应用开发完全遵循 ARM 嵌入式应用开发的流程和准则。

另一个典型案例，是仅由 EEP-TPU 组成的 ASIC 系统，其中 EEP-TPU 以主控处理器的形式存在，专用于基于某个特定神经网络算法的特定应用。在这样的 ASIC 系统中，EEP-TPU 的应用将按照专用 ASIC 系统的方式，将特定流程的任务以软件+硬件结合的方式实现。

在特定的嵌入式系统中，EEP-TPU 的调用通过特定的 API（Application Program Interface）接口来实现，这些 API 接口通常以 C/C++ 函数的形式存在，并通过静态链接库整合到用户的应用程序中。

(5) 嵌入式应用程序的编译

通过调用EEP-TPU的API接口，用户可以在其开发的应用程序中，随时调用EEP-TPU 进行神经网络计算，并快速获得结果。当应用程序开发完成后（通常是一个C/C++软件程序），EEP-TPU 的调用及底层库都包含在该应用程序中。此时，该C/C++的软件程序需要通过CPU的编译器（如ARM GNU编译器）来完成编译工作，从而获得可以在嵌入式CPU处理器运行的可执行文件。

在这里，需要特别注意的是，在本文所介绍的EEP-TPU开发流程中，有两种不同的编译器，生成两种不同的可执行文件。EEP-TPU编译器用于把神经网络算法转变成嵌入式端神经网络算法可执行文件，CPU编译器用于把C/C++软件转变成嵌入式端应用程序可执行文件。他们之间是相互独立而存在的，也即可在应用程序不变的情况下，通过更换神经网络算法可执行文件来实现算法更新与迭代；也可在神经网络算法不变的情况下，通过更换应用程序可执行文件来实现应用功能的更新与迭代。

(6) 下载

到这里为止，我们所有的开发工作都是在Host主机上完成的，并最终得到两个可执行文件：嵌入式应用程序可执行文件（CPU）和神经网络算法可执行文件（EEP-TPU）。这些可执行文件只能在从设备上（如ZYNQ 7020）运行，因此，需要通过某种方式把该文件拷贝到从设备的特定位置，这就是下载。

根据从设备系统所提供的功能不同，可使用的下载方式多种多样，可通过以太网、USB、UART、SPI等多种形式传输，具体可根据从设备所提供的通讯方式中选择最方便的即可。下载完成后，就可以在从设备上运行该程序，从而实现特定的AI任务了。

3.2 基于 Caffe 框架的深度学习分类算法实验

3.2.1 实验目的

1. 掌握 Ubuntu 系统下基于 Caffe 框架的深度学习算法开发方法和流程；
2. 掌握深度学习分类算法开发方法和流程
2. 掌握嵌入式人工智能推理计算的开发方法和流程；
3. 掌握 IMAGENET 基本知识及数据集制作步骤；
4. 掌握 EEP-TPU 算法编译与部署流程；
5. 掌握 EEP-TPU 嵌入式应用程序的基本开发方法和流程；

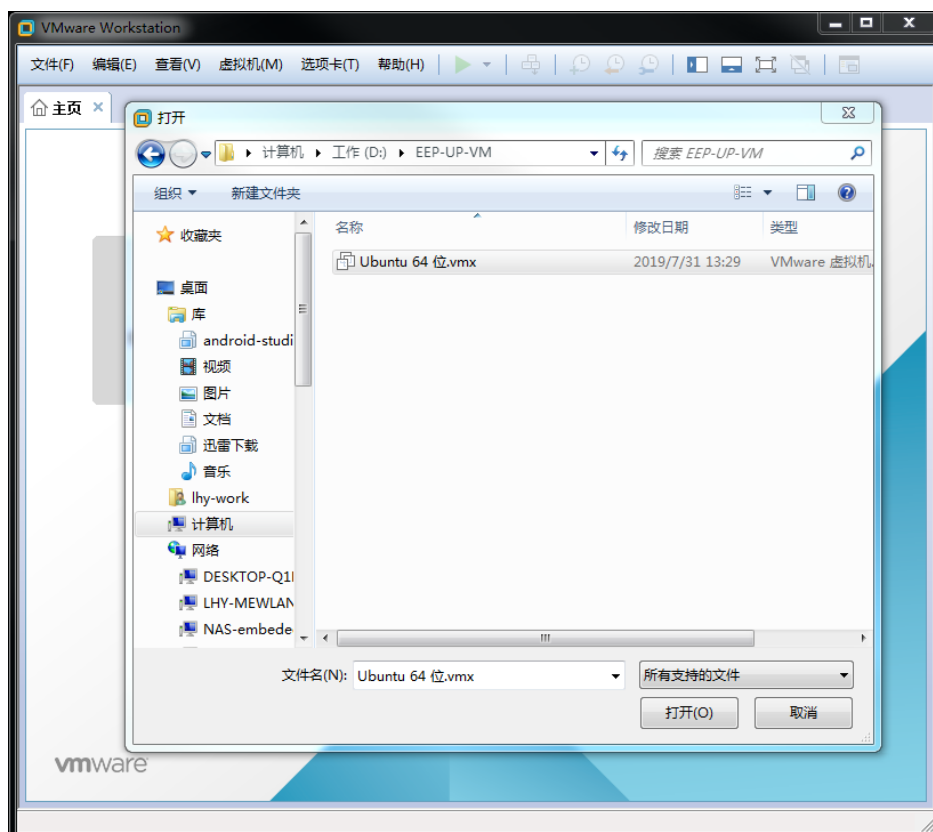
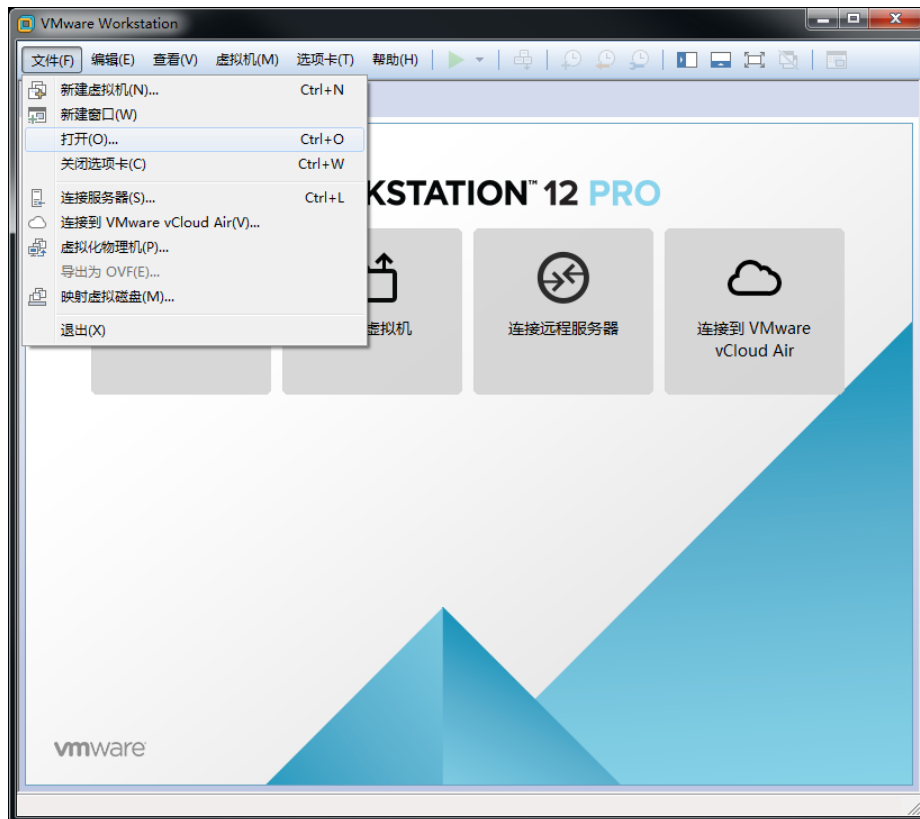
3.2.2 实验内容

采用 Mobilenet-V1 经典网络，基于 IMAGENET 数据集，使用 CAFFE 框架，完成从算法训练、EEP-TPU 编译到可执行文件部署的开发流程实验，并最终在 ZYNQ 开发板上部署用于分类的神经网络算法，实现图像分类任务。该实验的工程目录位于 `/home/eep/embedeep/Tutorials/P3_caffe/`

3.2.3 实验步骤

3.2.3.1 实验环境搭建

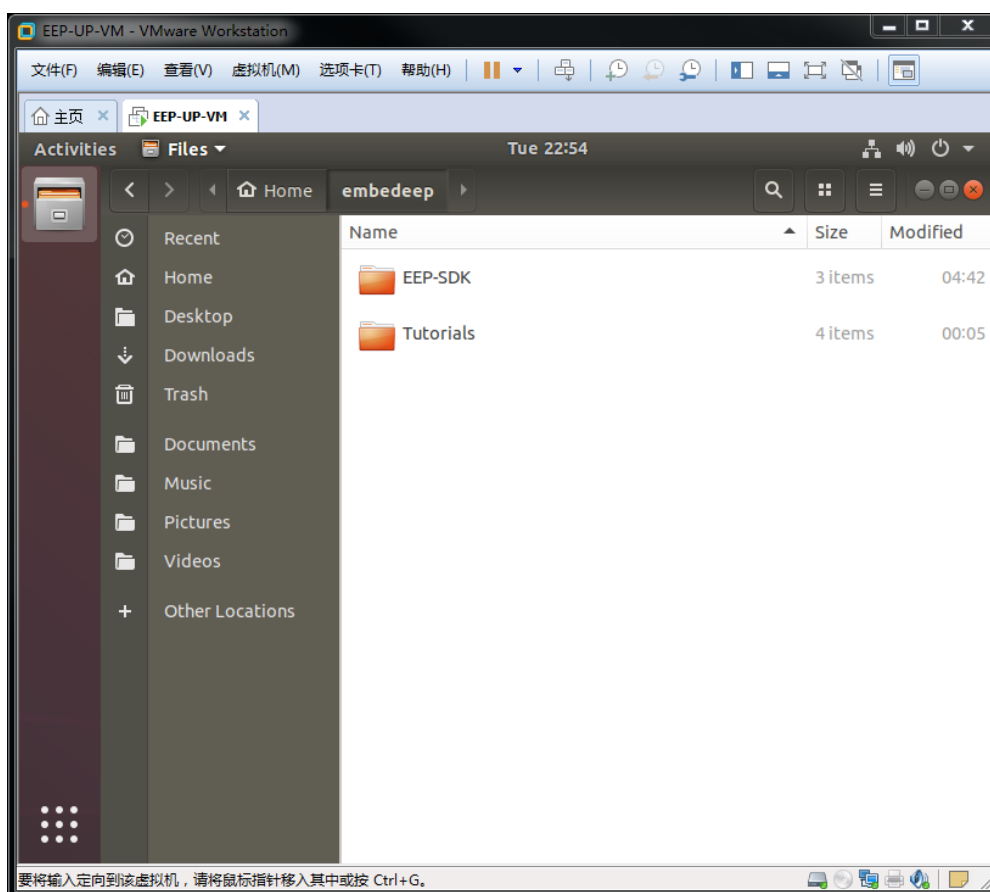
(1) 运行 VMware 虚拟机，启动 EEP-UP-VM 文件夹目录下的虚拟机文件 *Ubuntu 64 位.vmx*。



(2) 该虚拟机默认配置为双核处理器，4GB 内存，40GB 硬盘，运行 Ubuntu 18.04 操作系统，并内置本实验所需的所有代码和开发环境。上述 *Ubuntu 64 位.vmx* 文件正确打开后，可以根据需要点击[编辑虚拟机设置](#)对虚拟机硬件进行配置（例如调整处理器数量、内存和硬盘容量等）。如无需配置硬件，点击[开启此虚拟机](#)（如果该虚拟机上次运行以“挂起客户机”的命令结束，则点击[继续运行此虚拟机](#)）进入 Ubuntu 操作系统。



(3) 系统启动完毕后（根据 PC 性能，启动时间在 30s~300s 之间），点击 EEP-UP 用户，输入密码 *embedeep*。进入桌面系统后，点击左侧快捷栏的文件夹程序，双击进入 Home 目录下的 embedeep 目录，该目录包含 EEP-SDK 和 Tutorials 两个文件夹。



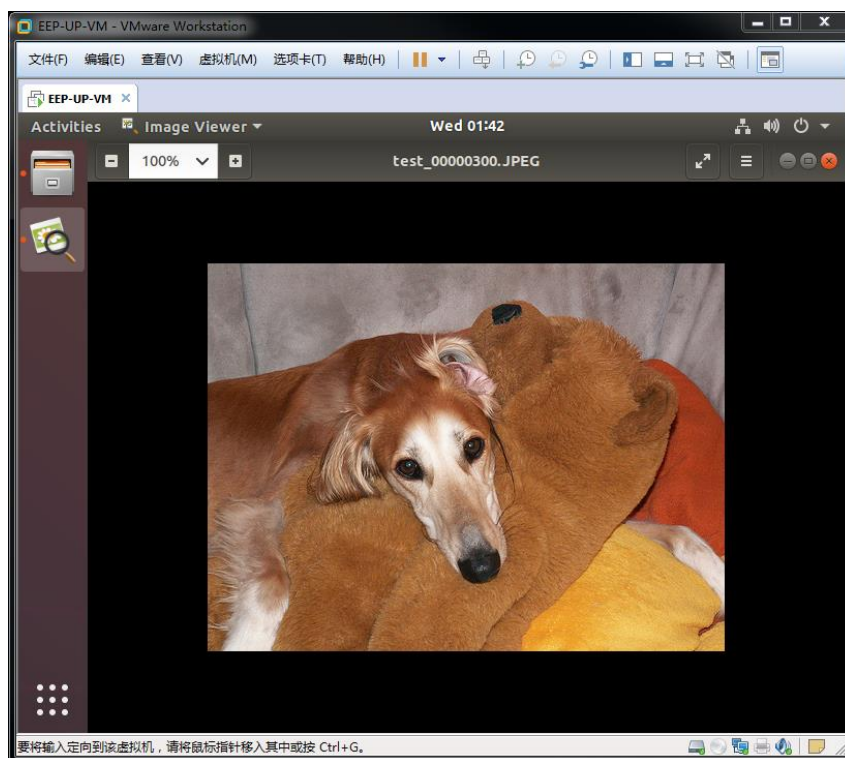
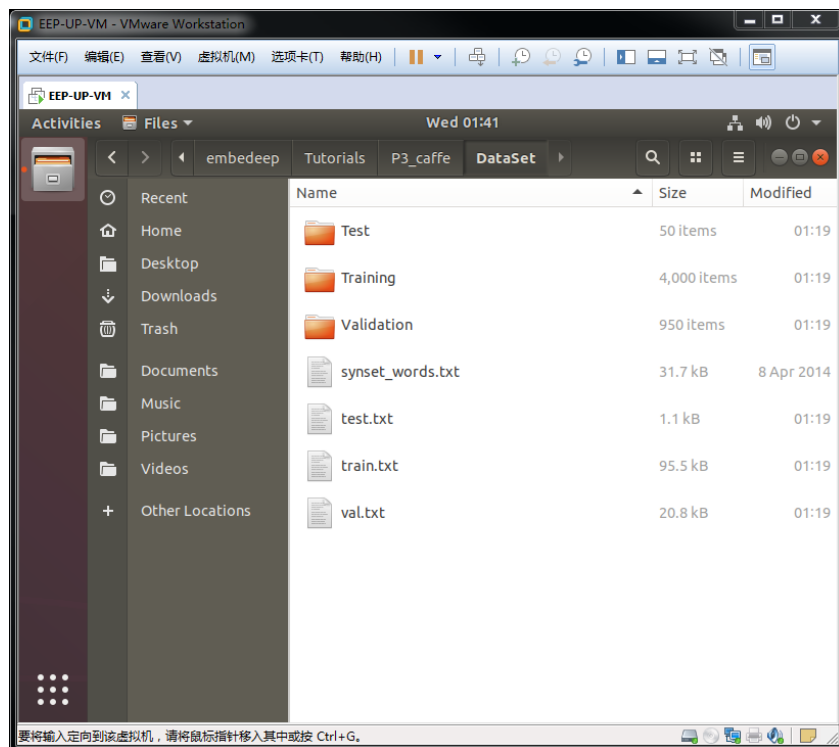
实验环境已预先搭建完毕，进入 `/home/eep/embedeep/Tutorials/P3_caffe/` 目录，按照后续步骤可以完成本实验。

3.2.3.2 制作数据集

本实验使用 IMAGENET 数据集 (<http://www.image-net.org/>)，该数据集包含 1000 类物体共计百万张图片。IMAGENET 数据集需要在上述官网注册后方可下载，需要约 140GB 的空间来存储。由于原始数据集过于庞大，本实验仅使用其中的极小部分来完成。

IMAGENET 提供的数据集被划分为三类：训练数据、验证数据、测试数据。其中，训练数据用于训练模型，验证数据用于在训练过程中检验模型的准确率，测试数据用于评价模型的最终准确率。我们已经完成了数据集的划分，数据图片存放于本实验目录 `/home/eep/embedeep/Tutorials/P3_caffe/DataSet`。

(1) 探索数据集。进入 *DataSet* 目录，可以看到 *Training/Validation/Test* 三个文件夹。该文件夹存放的原始图片分别用于训练、验证和测试。可以打开任一文件夹内的任意图片查看图片内容。



另外，还有 4 个文本文件，其中，*train.txt/val.txt/test.txt* 是图片标注信息，其分别记录了 *Training/Validation/Test* 文件夹内对应图片的类别信息。

例如，打开 *test.txt* 文件，第三行数据为 *test_00000300.JPEG 176*，其表示 Test 文件夹内 *test_00000300.JPEG* 图片的类别是第 176 类。该类别的具体含义可以查找 *synset_words.txt* 文件的第 176 行获得。通过查询可以知道，该图片内含物体为 *n02091635 otterhound, otter hound*，即奥达猎犬。

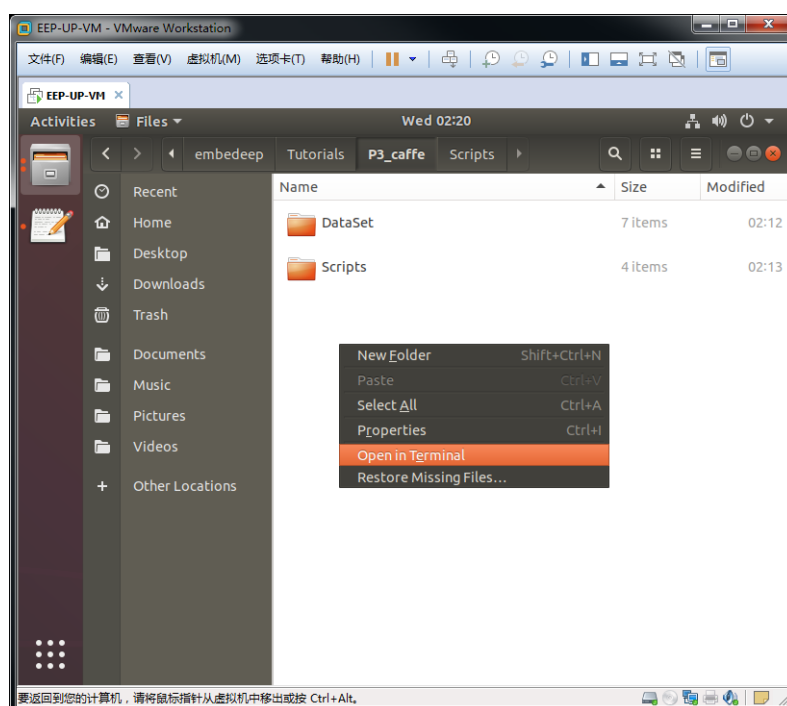
(2) 创建 LMDB 数据库

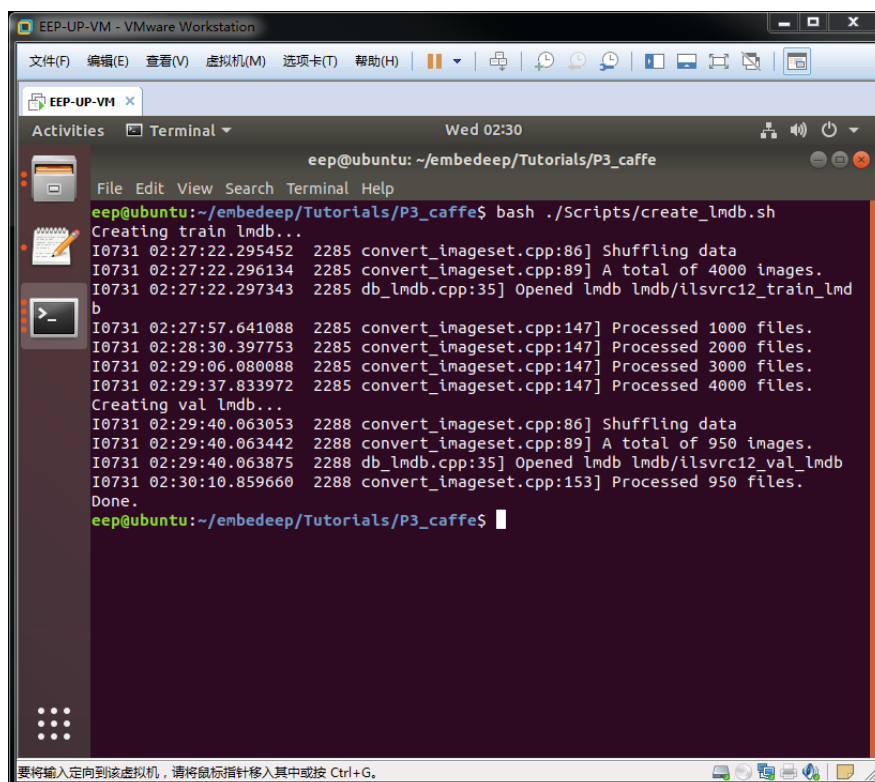
原始数据集以图片的形式存在，但在实际的训练过程中，Caffe 等深度学习框架需要把图片转换成某种数据库来使用。LMDB 数据库的转换工具已经集成在 Caffe 框架中，其运行脚本为 */home/eep/embedeep/Tutorials/P3_caffe/Scripts/create_lmdb.sh*

在 */home/eep/embedeep/Tutorials/P3_caffe* 目录下点击鼠标右键打开终端程序。在终端中输入（注意，如果 *lmdb* 文件夹内已有数据库，需要手动删除）：

```
bash ./Scripts/create_lmdb.sh
```

从打印的日志可以看到，该脚本分别创建了 *train_lmdb* 和 *val_lmdb* 两类数据库分别用于训练和验证，其中，训练数据库包含 4000 个文件（图片），验证数据库包含 950 个文件（图片）。



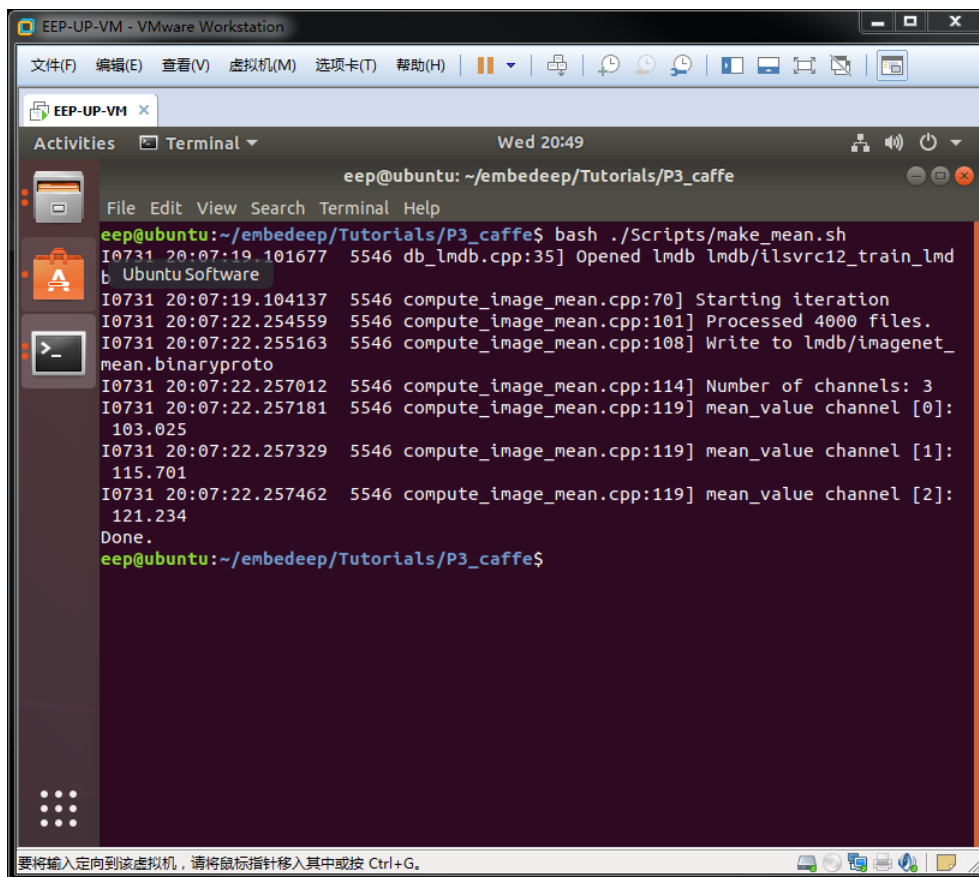


3.2.3.3 算法训练

(1) 均值计算

通常，为了使神经网络更快速的收敛，输入数据需要减去某个均值，从而使输入数据以零为中心分布。上述均值需要从训练数据集中计算得出，相关均值计算工具已经集成在 Caffe 框架中。在 `/home/eep/embedeep/Tutorials/P3_caffe` 目录下点击鼠标右键打开终端程序。在终端中输入：

```
bash ./Scripts/make_mean.sh
```



```
eep@ubuntu: ~/embedeep/Tutorials/P3_caffe
File Edit View Search Terminal Help
eep@ubuntu:~/embedeep/Tutorials/P3_caffe$ bash ./Scripts/make_mean.sh
I0731 20:07:19.101677 5546 db_lmdb.cpp:35] Opened lmdb lmdb/ilsrvrc12_train_lmd
b
Ubuntu Software
I0731 20:07:19.104137 5546 compute_image_mean.cpp:70] Starting iteration
I0731 20:07:22.254559 5546 compute_image_mean.cpp:101] Processed 4000 files.
I0731 20:07:22.255163 5546 compute_image_mean.cpp:108] Write to lmdb/imagenet_
mean.binaryproto
I0731 20:07:22.257012 5546 compute_image_mean.cpp:114] Number of channels: 3
I0731 20:07:22.257181 5546 compute_image_mean.cpp:119] mean_value channel [0]:
103.025
I0731 20:07:22.257329 5546 compute_image_mean.cpp:119] mean_value channel [1]:
115.701
I0731 20:07:22.257462 5546 compute_image_mean.cpp:119] mean_value channel [2]:
121.234
Done.
eep@ubuntu:~/embedeep/Tutorials/P3_caffe$
```

从打印的日志可以看到，均值计算读入 *lmdb/ilsrvrc12_train_lmdb* 数据库，共处理了 4000 个文件，最终的均值计算结果写入 *lmdb/imagenet_mean.binaryproto*，同时也得到三个通道的均值：

mean_value channel [0]: 103.025

mean_value channel [1]: 115.701

mean_value channel [2]: 121.234

上述均值计算结果以两种形式存在：文件 *lmdb/imagenet_mean.binaryproto* 和值 *103.025/115.701/121.234*。

在实际使用过程中，如果使用文件，则在 prototxt 中通过如下方式调用：

mean_file: "lmdb/imagenet_mean.binaryproto"

如果使用值，则在 prototxt 中通过如下方式调用：

mean_value: [103.025,115.701,121.234]

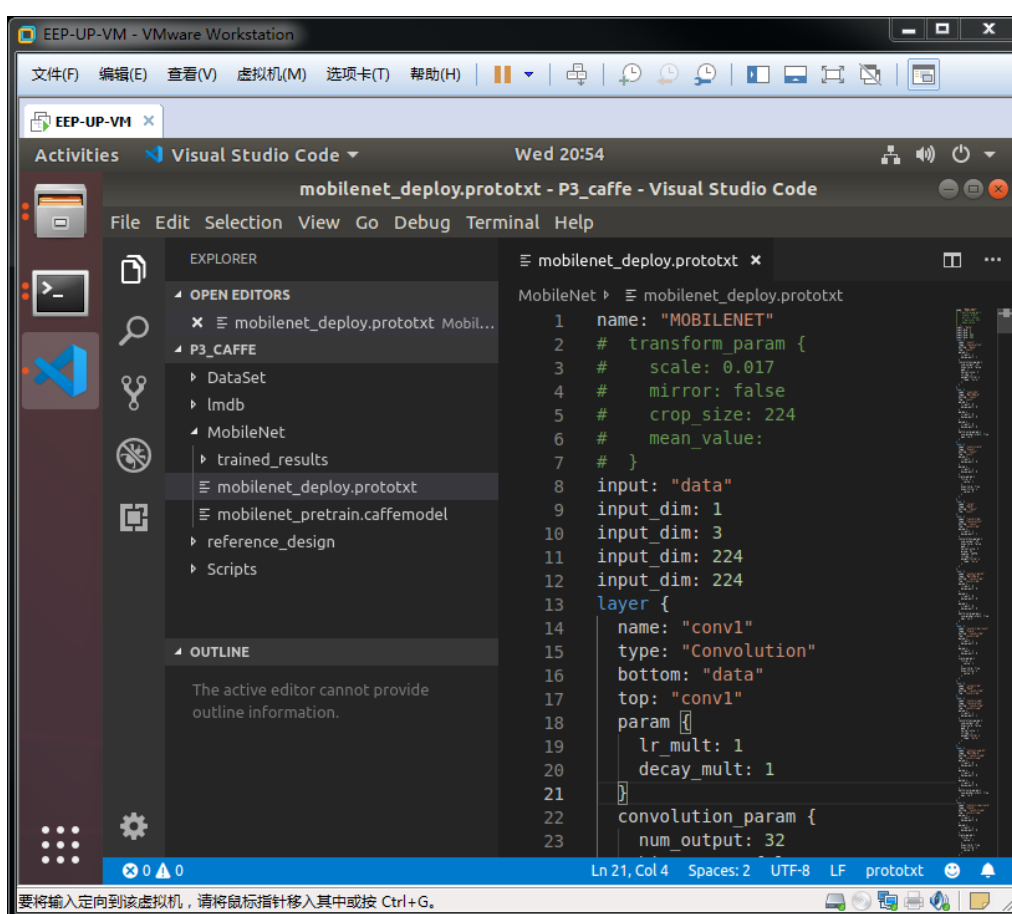
大多数情况下，均值以 `mean_value` 的形式存在，并直接写在神经网络结构中，本教程所有实验全部使用 `mean_value` 的形式。

(2) 构建神经网络结构

在 Caffe 框架中，使用 `prototxt` 格式的文本文件来描述神经网络结构。通过上节我们知道，数据集分为三类：训练集、验证集、测试集。对应的，神经网络结构也分为三类：训练网络（`train_net`）、验证网络（`val_net`）、部署网络（`deploy_net`）。其中，训练网络使用训练数据实现权重参数的学习与更新；验证网络使用验证数据对某个阶段权重参数的准确率进行评估；部署网络是最终使用时的形态（去掉了训练相关的层和参数），其可使用测试集完成最终的算法性能评估工作。

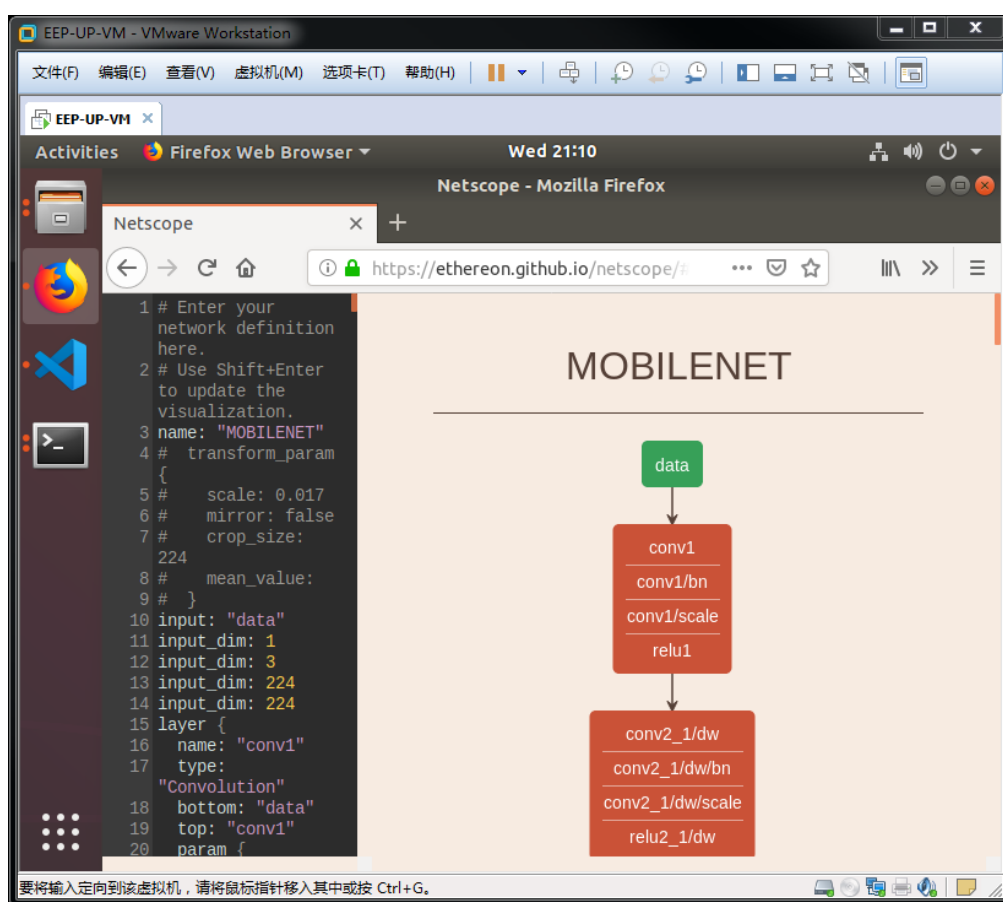
首先，我们打开 Visual Studio Code 文本编辑工具，在 `/home/eep/embedeep/Tutorials/P3_caffe` 目录下点击鼠标右键打开终端程序。在终端中输入：

```
code -n .
```



其中，左栏是该运行目录下的所有文件夹和文件，可以直接浏览并打开。右栏是已经打开的文件。可以看到，MobileNet 目录下有两个文件和一个文件夹，其中，*mobilenet_deploy.prototxt* 文件是 MobileNet 算法的部署网络，*mobilenet_pretrain.caffemodel*文件是该部署网络所对应的权重参数，该权重参数可以直接用于部署环境，也可以用于进一步训练的预训练文件。

我们可以通过可视化工具来探索该神经网络的结构。打开浏览器（建议使用 Chrome 或 Firefox），输入如下网址 <https://ethereon.github.io/netscope/#/editor>。接着在左栏输入 *mobilenet_deploy.prototxt* 文件的所有内容（copy-paste），并按下键盘的 **Shift+Enter** 按键。最终，右栏会出现该神经网络的可视化显示。



接下来，我们以该部署网络（*deploy_net*）为起点来构建训练网络（*train_net*）和验证网络（*val_net*）。

- 复制 *mobilenet_deploy.prototxt* 并重命名为 *mobilenet_train_val.prototxt*
- 删除 *mobilenet_train_val.prototxt* 中的 5 行输入项描述：

```
input: "data"
input_dim: 1
input_dim: 3
input_dim: 224
input_dim: 224
```

c) 在 layer conv1 之前添加新的训练 Data 层，如下：

```
layer {
  name: "data"
  type: "Data"
  top: "data"
  top: "label"
  include {
    phase: TRAIN
  }
  transform_param {
    scale: 0.017
    mirror: true
    crop_size: 224
    mean_value: [103.025, 115.701, 121.234]
  }
  data_param {
    source: "lmdb/ilsrvrc12_train_lmdb"
    batch_size: 4
    backend: LMDB
  }
}
```

这里需要特别说明的是，`include { phase: TRAIN }` 表明该层仅用于训练阶段；`transform_param { ... }` 指定了 data 的预处理步骤；`data_param { source: "lmdb/ilsrvrc12_train_lmdb" }` 指定了训练数据库（LMDB 格式）的位置；

d) 在上述新增加的训练 Data 层后，添加新的验证 Data 层，如下：

```

layer {
  name: "data"
  type: "Data"
  top: "data"
  top: "label"
  include {
    phase: TEST
  }
  transform_param {
    scale: 0.017
    mirror: false
    crop_size: 224
    mean_value: [103.025, 115.701, 121.234]
  }
  data_param {
    source: "lmdb/ilsrvrc12_val_lmdb"
    batch_size: 1
    backend: LMDB
  }
}

```

验证 Data 层与训练 Data 层拥有相似的结构，其中，*include { phase: TEST }* 表明该层仅用于验证阶段，*data_param { source: "lmdb/ilsrvrc12_val_lmdb" }* 指定了验证数据库（LMDB 格式）的位置。

e) 删除最后的 prob 层

```

layer {
  name: "prob"
  type: "Softmax"
  bottom: "fc7"
  top: "prob"
}

```

f) 在 fc7 层之后添加新的 loss 层，用于计算损失函数。

```

layer {

```



```

    name: "loss"
    type: "SoftmaxWithLoss"
    bottom: "fc7"
    bottom: "label"
    top: "loss"
}

```

g) 在上述 loss 层之后添加新的 Accuracy 层，用于验证网络进行准确率计算

```

layer {
  name: "top1/acc"
  type: "Accuracy"
  bottom: "fc7"
  bottom: "label"
  top: "top1/acc"
  include {
    phase: TEST
  }
}
layer {
  name: "top5/acc"
  type: "Accuracy"
  bottom: "fc7"
  bottom: "label"
  top: "top5/acc"
  include {
    phase: TEST
  }
  accuracy_param {
    top_k: 5
  }
}

```

(3) 设置训练控制文件。

新建一个文本，命名为 *solver.prototxt*。该文件用于控制整个训练过程，其内容如下：

```
net: "MobileNet/mobilenet_train_val.prototxt"
test_iter: 100
test_interval: 100
base_lr: 0.0001
lr_policy: "step"
gamma: 0.1
stepsize: 100000
display: 20
max_iter: 500
momentum: 0.9
weight_decay: 0.0005
snapshot: 100
snapshot_prefix: "MobileNet/trained_results"
solver_mode: CPU
```

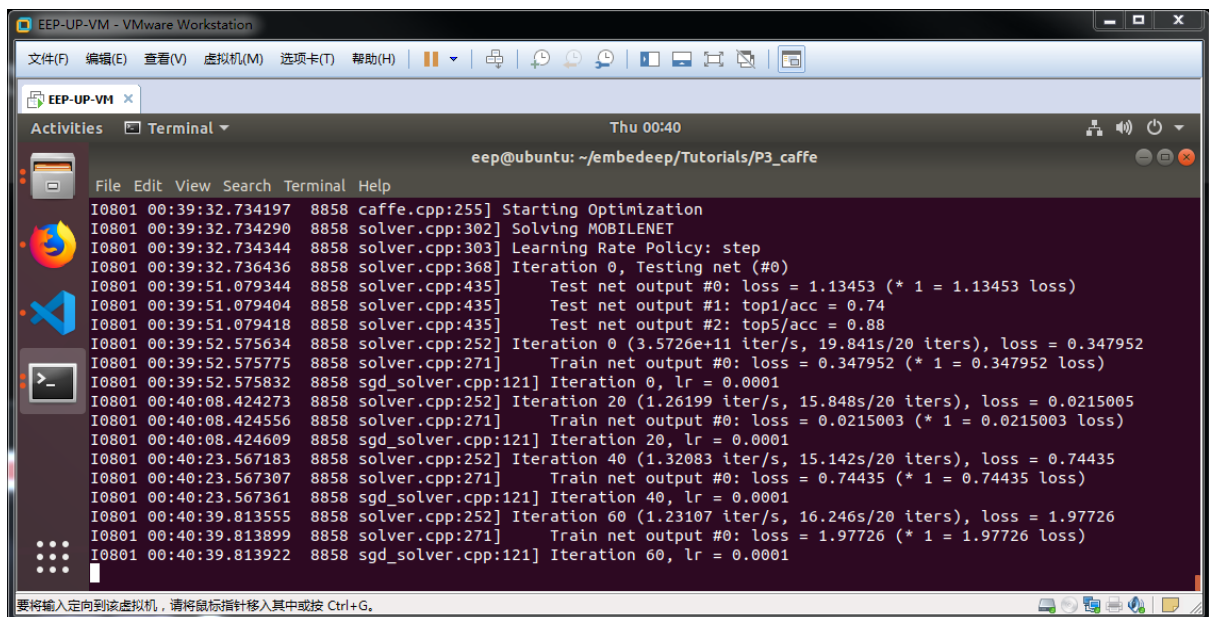
其中，*net* 字段指定了训练用的 prototxt 文件（内部包含了训练和验证网络），*max_iter* 字段指定了训练的最大迭代次数，*test_interval* 字段指定了每隔多少次迭代执行一次验证任务，*solver_mode: CPU* 字段指定使用 CPU 执行训练任务（通常情况下使用 GPU 的效率更高）。

（3）执行训练

上述文件全部准备完毕后，在 */home/eep/embedeep/Tutorials/P3_caffe* 目录下点击鼠标右键打开终端程序。在终端中输入：

```
bash ./Scripts/train.sh
```

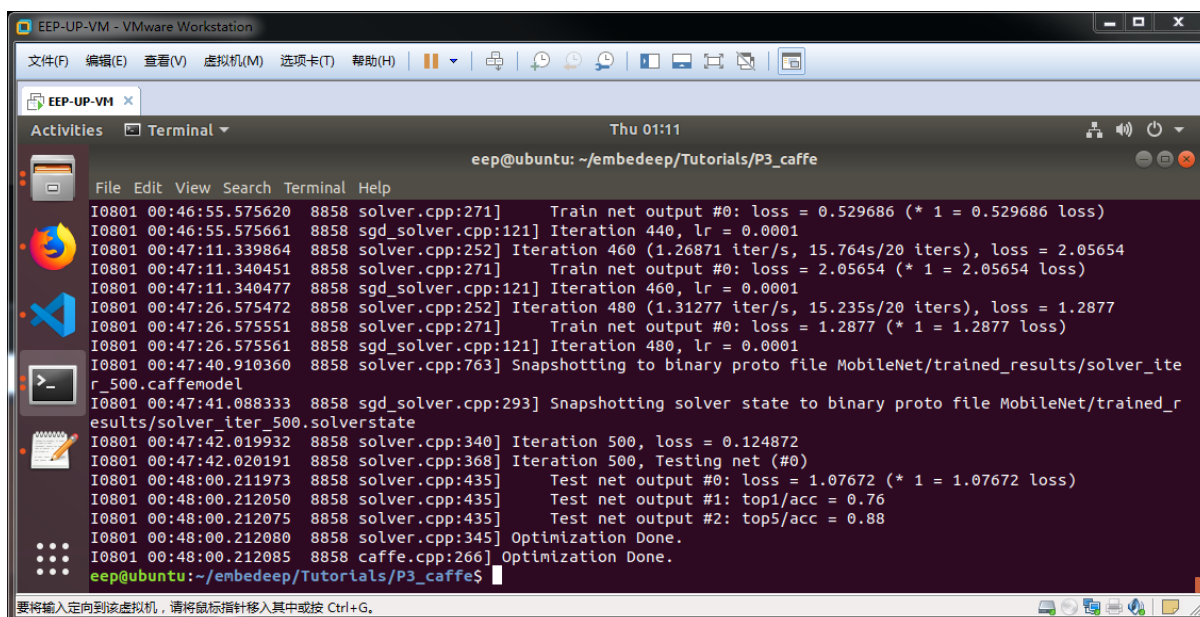
train.sh 文件只有一条命令：*caffe train --solver MobileNet/solver.prototxt --weights MobileNet/mobilenet_pretrain.caffemodel*。其中，*--solver* 指定了上述训练控制文件，*--weight* 指定了预训练权重参数。运行过程如下：



```
I0801 00:39:32.734197 8858 caffe.cpp:255] Starting Optimization
I0801 00:39:32.734290 8858 solver.cpp:302] Solving MOBILENET
I0801 00:39:32.734344 8858 solver.cpp:303] Learning Rate Policy: step
I0801 00:39:32.736436 8858 solver.cpp:368] Iteration 0, Testing net (#0)
I0801 00:39:51.079344 8858 solver.cpp:435] Test net output #0: loss = 1.13453 (* 1 = 1.13453 loss)
I0801 00:39:51.079404 8858 solver.cpp:435] Test net output #1: top1/acc = 0.74
I0801 00:39:51.079418 8858 solver.cpp:435] Test net output #2: top5/acc = 0.88
I0801 00:39:52.575634 8858 solver.cpp:252] Iteration 0 (3.5726e+11 iter/s, 19.841s/20 iters), loss = 0.347952
I0801 00:39:52.575775 8858 solver.cpp:271] Train net output #0: loss = 0.347952 (* 1 = 0.347952 loss)
I0801 00:39:52.575832 8858 sgd_solver.cpp:121] Iteration 0, lr = 0.0001
I0801 00:40:08.424273 8858 solver.cpp:252] Iteration 20 (1.26199 iter/s, 15.848s/20 iters), loss = 0.0215005
I0801 00:40:08.424556 8858 solver.cpp:271] Train net output #0: loss = 0.0215003 (* 1 = 0.0215003 loss)
I0801 00:40:08.424609 8858 sgd_solver.cpp:121] Iteration 20, lr = 0.0001
I0801 00:40:23.567183 8858 solver.cpp:252] Iteration 40 (1.32083 iter/s, 15.142s/20 iters), loss = 0.74435
I0801 00:40:23.567307 8858 solver.cpp:271] Train net output #0: loss = 0.74435 (* 1 = 0.74435 loss)
I0801 00:40:23.567361 8858 sgd_solver.cpp:121] Iteration 40, lr = 0.0001
I0801 00:40:39.813555 8858 solver.cpp:252] Iteration 60 (1.23107 iter/s, 16.246s/20 iters), loss = 1.97726
I0801 00:40:39.813899 8858 solver.cpp:271] Train net output #0: loss = 1.97726 (* 1 = 1.97726 loss)
I0801 00:40:39.813922 8858 sgd_solver.cpp:121] Iteration 60, lr = 0.0001
```

终端所显示的训练日志包含几点重要信息。

- Test net output #0/#1/#2* 包含验证网络输出的三个重要信息 *loss*、*top1/acc*、*top5/acc* 分别来自于 *mobilenet_train_val.prototxt* 文件中的 *loss* 层、*top1/acc* 层和 *top5/acc* 层，是评价算法性能的重要指标 (*loss* 越低越好，*acc* 越高越好)。
- Train net output #0* 包含训练网络的输出 *loss*。
- 通过判断 *loss* 的变化趋势 (通常可以画出一条损失函数曲线) 可以对训练过程进行十分直观的判断。
- 每隔 100 次迭代，中间训练结果 (snapshot) 会以 *caffemodel* 和 *solverstate* 两种形态进行保存于 *MobileNet/trained_results* 文件夹内。其中，*caffemodel* 是权重参数，可作为下一次训练的预训练权重初始值；*solverstate* 是临时状态，以 *solverstate* 为起点开始训练等于接着上次结束的状态继续执行，也即恢复训练操作。
- 经过 500 次迭代后 (训练控制文件 *max_iter* 所定义)，训练结束，最终验证所得的 Top1 准确率为 75% 左右，Top5 准确率为 88% 左右。



(4) 使用神经网络算法

训练完成后，可以使用该神经网络算法执行图片分类任务。在 `/home/eep/embedeep/Tutorials/P3_caffe` 目录下点击鼠标右键打开终端程序。在终端中输入：

```
python ./Scripts/eval_image.py \  
--proto ./MobileNet/mobilenet_deploy.prototxt \  
--model ./MobileNet/trained_results/solver_iter_500.caffemodel \  
--image ./DataSet/Test/test_00000100.JPEG
```

其中，

`--proto` 指定了部署网络 `mobilenet_deploy.prototxt`,

`--model` 指定了上述训练过程所得到的神经网络参数（注意选择最新的模型）。

`--image` 指定了需要计算的图片。

EEP-UP-VM - VMware Workstation

文件(F) 编辑(E) 查看(V) 虚拟机(M) 选项卡(T) 帮助(H)

EEP-UP-VM x

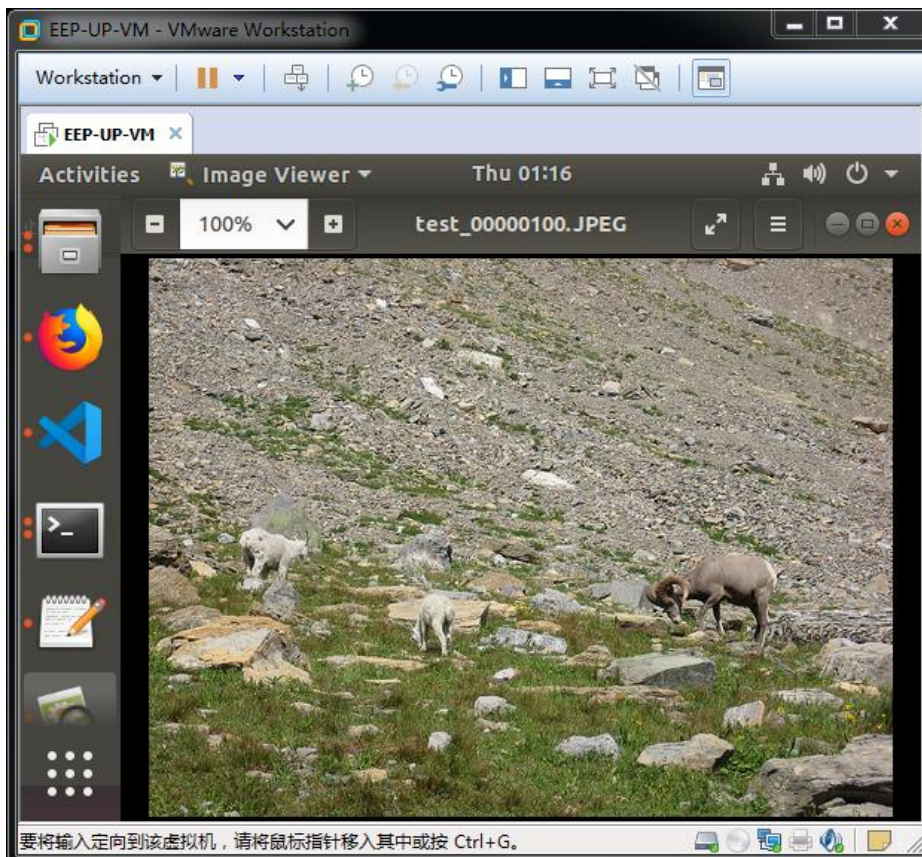
Activities Terminal Fri 03:00

eeep@ubuntu: ~/embedeep/Tutorials/P3_caffe

File Edit View Search Terminal Help

```
I0802 03:00:20.004535 11626 net.cpp:202] input does not need backward computation.
I0802 03:00:20.004542 11626 net.cpp:244] This network produces output prob
I0802 03:00:20.004602 11626 net.cpp:257] Network initialization done.
I0802 03:00:20.020467 11626 upgrade_proto.cpp:79] Attempting to upgrade batch norm layers using deprecated params: ./MobileNet/trained_results/solver_iter_500.caffemodel
I0802 03:00:20.020519 11626 upgrade_proto.cpp:82] Successfully upgraded batch norm layers using deprecated params.
I0802 03:00:20.020529 11626 net.cpp:746] Ignoring source layer data
I0802 03:00:20.023958 11626 net.cpp:746] Ignoring source layer loss
/home/eeep/.local/lib/python2.7/site-packages/skimage/io/_io.py:49: UserWarning: 'as_gray' has been deprecated in favor of 'as_grayscale'
  warn("'as_gray' has been deprecated in favor of 'as_grayscale'")
/home/eeep/.local/lib/python2.7/site-packages/skimage/transform/_warps.py:110: UserWarning: Anti-aliasing will be enabled by default in skimage 0.15 to avoid aliasing artifacts when down-sampling images.
  warn("Anti-aliasing will be enabled by default in skimage 0.15 to ")
#####
### result ###
cost time: 244.419189 ms
Label 349, Probability 0.48 -- n02415577 bighorn, bighorn sheep, cimarron, Rocky Mountain bighorn, Rocky Mountain sheep, Ovis canadensis
Label 350, Probability 0.13 -- n02417914 ibex, Capra ibex
Label 363, Probability 0.06 -- n02454379 armadillo
Label 348, Probability 0.06 -- n02412080 ram, tup
Label 343, Probability 0.04 -- n02397096 warthog
eeep@ubuntu: ~/embedeep/Tutorials/P3_caffe$
```

要将输入定向到该虚拟机，请将鼠标指针移入其中或按 Ctrl+G。



从打印得到的日志可以看到，该图片的 Top5 分类结果如下：

```
cost time: 244.419189 ms
Label 349, Probability 0.48 -- n02415577 bighorn, bighorn sheep, cimarron, Rocky
Mountain bighorn, Rocky Mountain sheep, Ovis canadensis
Label 350, Probability 0.13 -- n02417914 ibex, Capra ibex
Label 363, Probability 0.06 -- n02454379 armadillo
Label 348, Probability 0.06 -- n02412080 ram, tup
Label 343, Probability 0.04 -- n02397096 warthog
```

该 Top1 结果与 DataSet/test.txt 中所记录的标注信息一致，判断结果正确。更换 `--image ./DataSet/Test/test_00000100.JPEG` 命令中的图片，可以获取其他图片的计算结果。

如果准确率符合应用需求，至此，算法训练阶段结束。

3.2.3.4 EEP-TPU 算法编译

人工智能算法训练完毕后，会得到神经网络描述和权重参数两个文件，在上节所述的算法训练步骤完成后，可以得到位于 `/home/eep/embedeep/Tutorials/P3_caffe/MobileNet` 目录下的神经网络（部署）描述文件 `mobilenet_deploy.prototxt`，和位于 `/home/eep/embedeep/Tutorials/P3_caffe/MobileNet/trained_results` 目录下的训练权重文件 `solver_iter_500.caffemodel`。

然而，`mobilenet_deploy.prototxt` 和 `solver_iter_500.caffemodel` 是运行在 X86 体系计算机上的 Caffe 框架所使用的格式。要想把该人工智能算法部署到嵌入式端，还需要进行编译和转换，在基于 EEP-TPU 的嵌入式人工智能计算平台上，可以使用相应的编译器 `eeptpu_compiler` 来完成该转换任务。

一个典型的 `eeptpu_compiler` 编译命令如下所示：

```
eeptpu_compiler \
--mean '103.025,115.701,121.234' \
--norm '0.017,0.017,0.017' \
--prototxt ./MobileNet/mobilenet_deploy.prototxt \
--caffemodel ./MobileNet/trained_results/solver_iter_500.caffemodel \
```

```
--image ./DataSet/Test/test_00000100.JPEG \  
--output ./MobileNet
```

其中,

-- *mean* 指定均值, 该值为浮点, 分别对应输入图像的 3 个通道, 该值与 Caffe 中 Data 层所指定的 *mean_value* 值一致 (顺序也一致)。

-- *norm* 指定归一化值, 该值为浮点, 分别对应输入图像的 3 个通道, 该值与 Caffe 中 Data 层所指定的 *scale* 值一致 (3 个通道内容相同, 都为 *scale* 值)。

-- *prototxt* 指定神经网络描述文件的路径

-- *caffemodel* 指定神经网络权重参数文件的路径

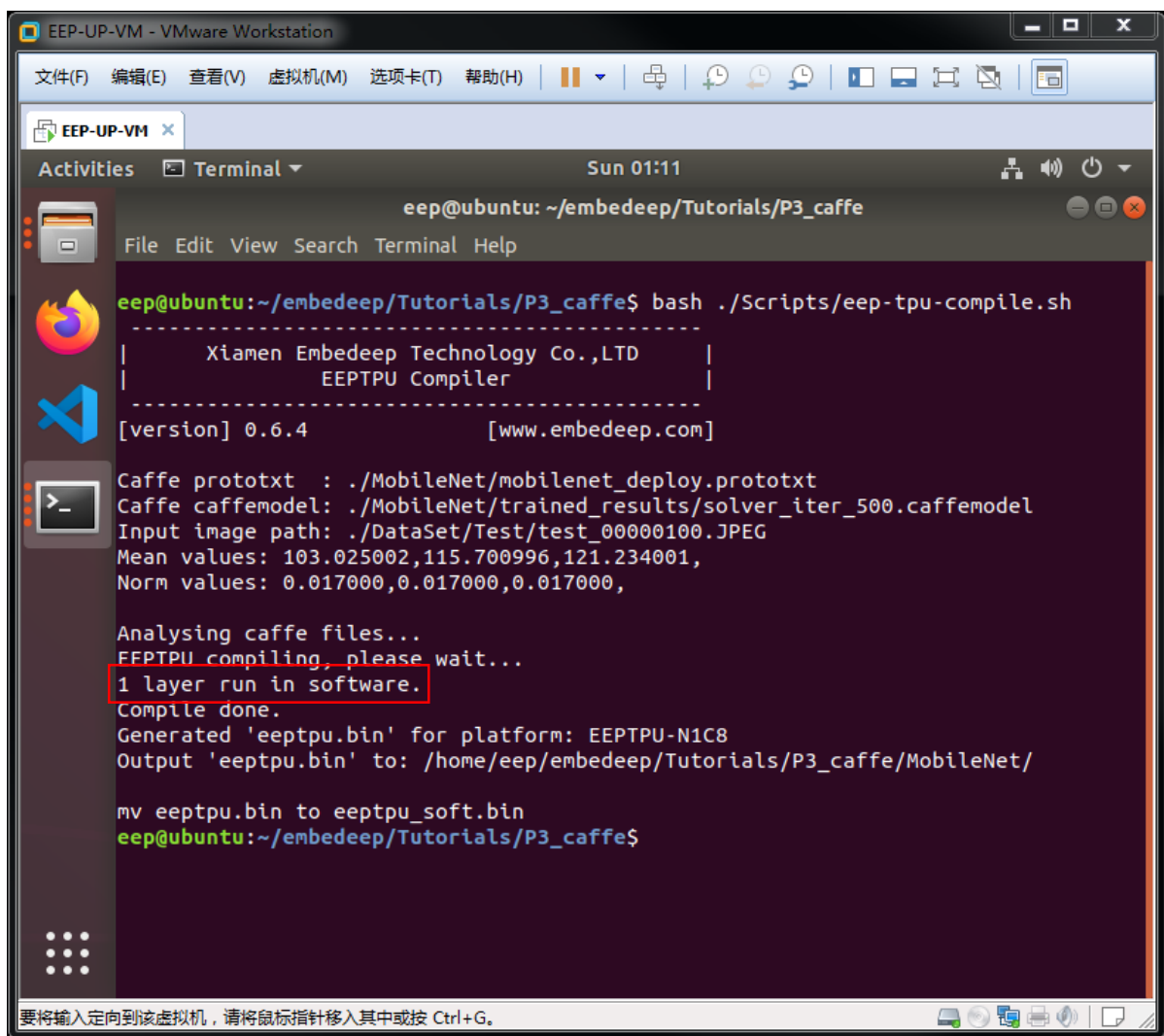
-- *image* 指定来自 Test 数据集的任意一张图片

-- *output* 指定编译结果的输出目录

在 [/home/eep/embedeep/Tutorials/P3_caffe](#) 目录下点击鼠标右键打开终端程序。
在终端中输入:

```
bash ./Scripts/eep-tpu-compile.sh
```

运行结果如下, 终端日志显示了所有信息, 包括运行是否成功。



```
EEP-UP-VM - VMware Workstation
文件(F) 编辑(E) 查看(V) 虚拟机(M) 选项卡(T) 帮助(H)
EEP-UP-VM x
Activities Terminal Sun 01:11
eep@ubuntu: ~/embedeep/Tutorials/P3_caffe
File Edit View Search Terminal Help
eep@ubuntu:~/embedeep/Tutorials/P3_caffe$ bash ./Scripts/eeptpu-compile.sh
-----
| Xiamen Embedeep Technology Co.,LTD |
| EEPTPU Compiler |
-----
[version] 0.6.4 [www.embedeep.com]

Caffe prototxt : ./MobileNet/mobilenet_deploy.prototxt
Caffe caffemodel: ./MobileNet/trained_results/solver_iter_500.caffemodel
Input image path: ./DataSet/Test/test_00000100.JPEG
Mean values: 103.025002,115.700996,121.234001,
Norm values: 0.017000,0.017000,0.017000,

Analysing caffe files...
EEPTPU compiling, please wait...
1 layer run in software.
Compile done.
Generated 'eeptpu.bin' for platform: EEPTPU-N1C8
Output 'eeptpu.bin' to: /home/eep/embedeep/Tutorials/P3_caffe/MobileNet/

mv eeptpu.bin to eeptpu_soft.bin
eep@ubuntu:~/embedeep/Tutorials/P3_caffe$
```

正确执行上述命令，出现 “*Compile Done*” 意味着编译成功，编译所生成的文件名为 `eeptpu.bin`，该文件位于 `--output` 所指定的文件夹内。（注意，这里为了方便管理，脚本中还使用 “`mv ./MobileNet/eeptpu.bin ./MobileNet/eeptpu_soft.bin`” 命令把 `eeptpu.bin` 重新命名为 `eeptpu_soft.bin`）。

需要特别注意的是，从上图的红色部分的日志可以看到，在本次编译过程中，编译器自动把 1 个 layer 用软件来执行。从附录 B. EEPTPU Compiler 我们可以知道，在本实验所使用的 EEPTPU 处理器中，Softmax 层还无法支持，因此，编译器选择使用 CPU 来实现该层的计算。有关异构计算的内容，请参考 2.4 节和附录 B.的相关内容。

为了加速，对于分类网络，通常可以把最后的 Softmax 层去掉，直接输出全连接层的结果。分类网络中，Softmax 层之前的全连接层数据代表每一类的重要程度，而

Softmax 层在此基础上计算每一类占总体的概率。由于最终仅关心排序，原始数据输出不影响排序结果，因此 Softmax 实际上可以去掉。

作为实验，我们需要对测试网络（deploy.prototxt）文件进行修改，去掉 Softmax 层。

在 `/home/eep/embedeep/Tutorials/P3_caffe/MobileNet` 目录下，复制 `mobilenet_deploy.prototxt` 文件并重命名为 `mobilenet_deploy_nosoft.prototxt`，打开该文件，删除 Softmax 层相关代码，并把 fc7 层的 top blob 的名字改为 prob。

```
layer{  
  name: "prob"  
  type: "Softmax"  
  bottom: "fc7"  
  top: "prob"  
}  
layer {  
  name: "fc7"  
  type: "Convolution"  
  bottom: "pool6"  
  top: "prob"  
  param {  
    lr_mult: 1  
    decay_mult: 1  
  }  
  param {  
    lr_mult: 2  
    decay_mult: 0  
  }  
  convolution_param {  
    num_output: 1000  
    kernel_size: 1  
    weight_filler {  
      type: "msra"  
    }  
    bias_filler {  
      type: "constant"  
    }  
  }  
}
```

```

value: 0
}
}
}

```

我们需要新的编译命令

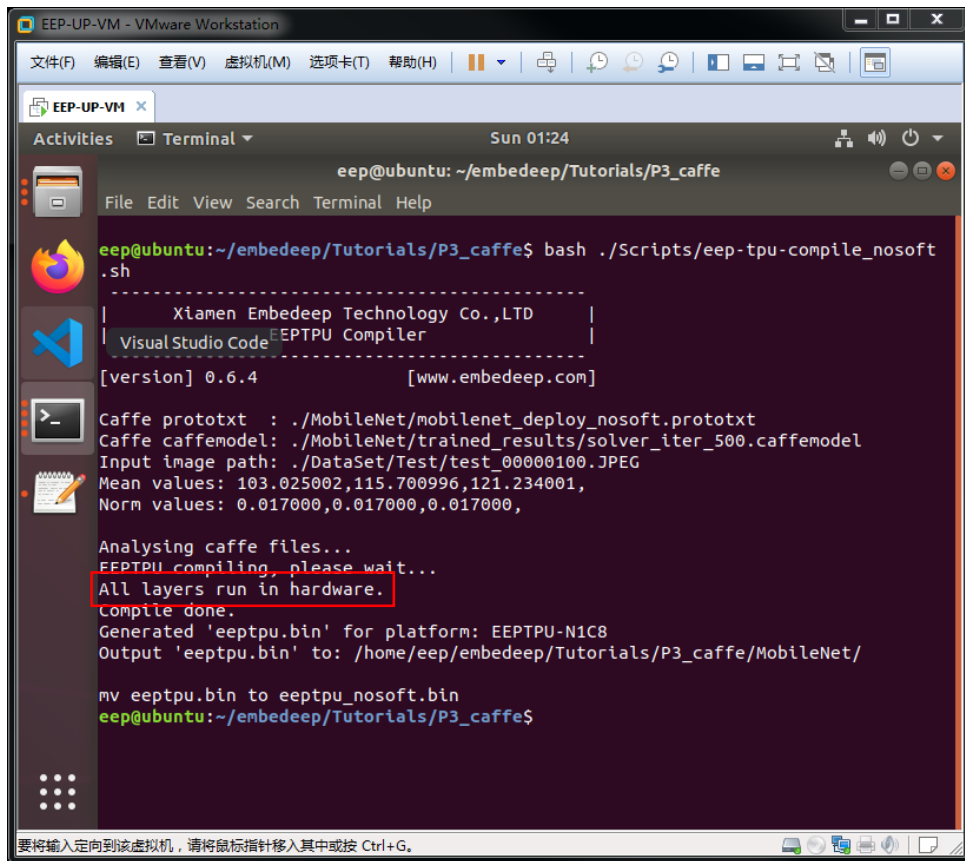
```

eeptpu_compiler \
--mean '103.025,115.701,121.234' \
--norm '0.017,0.017,0.017' \
--prototxt ./MobileNet/mobilenet_deploy_nosoft.prototxt \
--caffemodel ./MobileNet/trained_results/solver_iter_500.caffemodel \
--image ./DataSet/Test/test_00000100.JPEG \
--output ./MobileNet

```

在 `/home/eeep/embedeep/Tutorials/P3_caffe` 目录下点击鼠标右键打开终端程序。
在终端中输入：

```
bash ./Scripts/eeep-tpu-compile_nosoft.sh
```



```

EEPTPU-VM - VMware Workstation
文件(F) 编辑(E) 查看(V) 虚拟机(M) 选项卡(T) 帮助(H)
EEPTPU-VM x
Activities Terminal Sun 01:24
eeep@ubuntu: ~/embedeep/Tutorials/P3_caffe
File Edit View Search Terminal Help
eeep@ubuntu:~/embedeep/Tutorials/P3_caffe$ bash ./Scripts/eeep-tpu-compile_nosoft.sh
-----
| Xiamen Embedeep Technology Co.,LTD |
| Visual Studio Code EEPTPU Compiler |
-----
[version] 0.6.4 [www.embedeep.com]
Caffe prototxt : ./MobileNet/mobilenet_deploy_nosoft.prototxt
Caffe caffemodel: ./MobileNet/trained_results/solver_iter_500.caffemodel
Input image path: ./DataSet/Test/test_00000100.JPEG
Mean values: 103.025002,115.700996,121.234001,
Norm values: 0.017000,0.017000,0.017000,
Analysing caffe files...
EEPTPU compiling, please wait...
All layers run in hardware.
Compile done.
Generated 'eeptpu.bin' for platform: EEPTPU-N1C8
Output 'eeptpu.bin' to: /home/eeep/embedeep/Tutorials/P3_caffe/MobileNet/
mv eeptpu.bin to eeptpu_nosoft.bin
eeep@ubuntu:~/embedeep/Tutorials/P3_caffe$

```

正确执行上述命令，出现 “*Compile Done*” 意味着编译成功，编译所生成的文件名为 eeptpu.bin，该文件位于 *--output* 所指定的文件夹内。（注意，这里为了方便管理，脚本中还使用 “mv ./MobileNet/eeptpu.bin ./MobileNet/eeptpu_nosoft.bin” 把 eeptpu.bin 重新命名为 eeptpu_nosoft.bin）。

需要特别注意的是，从上图的红色部分的日志可以看到，在本次编译过程中，去掉 Softmax，编译器把所有层都用 EEP-TPU 来执行

3.2.3.5 EEP-TPU 嵌入式应用开发

上述 eeptpu.bin 只能在含有 EEP-TPU 张量处理器的嵌入式系统中使用，本节，我们将构建一个简单的程序，使用 eeptpu.bin 在嵌入式端完成上述 PC 端的图片分类任务。

- a) 新建一个文本文件，命名为 main.cpp;
- b) 构建 EEP-TPU 初始化函数，仅有两个步骤。首先，通过 **tpu->eeptpu_set_interface(interface_type)** NNAPI 函数设置 EEP-TPU 的接口类型，其中 interface_type 指 EEP-TPU 与 CPU 的接口，可以有两种：SOC（一般与 ARM 或 RISC-V CPU 通过总线接口）或 PCIE（一般与 X86 CPU 通过 PCIE 接口）；其次，通过 **tpu->eeptpu_load_bin(path_bin)** NNAPI 函数把 eeptpu.bin 文件的内容写入内存。NNAPI 函数可以进一步参考附录 A. 。

```
1. static int eeptpu_init(int interface_type, char* path_bin)
2. {
3.     int ret;
4.     if (tpu == NULL) tpu = tpu->init();
5.     ret = tpu->eeptpu_set_interface(interface_type);#(1) set SOC or PCIE
6.     if (ret < 0) return ret;
7.     printf("EEPTPU library version: %s\n", tpu->eeptpu_get_lib_version());
8.     printf("EEPTPU hardware version: %s\n", tpu->eeptpu_get_tpu_version());
9.     printf("EEPTPU hardware info : %s\n", tpu->eeptpu_get_tpu_info());
10.    ret = tpu->eeptpu_load_bin (path_bin); #(2) load eeptpu.bin
11.    if (ret < 0) {
12.        printf("Load bin fail, ret=%d\n", ret);
```

```

13. return ret;
14. }
15. printf("EEPTPU init ok\n");
16. return 0;
17. }

```

- c) 构建 EEP-TPU 数据写入 (EEP-TPU 专属内存) 函数, 包括两个步骤。首先, 读取输入数据 (图片), 在这里我们使用 opencv 的 `cv::imread` 函数读入输入图片, 并根据神经网络的尺寸使用 `cv::resize` 函数进行图片缩放操作。相应的 `imread` 和 `resize` 操作被封装到 `image_read_to_cv_mat` 函数中, 该函数具体可以参考 `reference_design` 目录下的 `main.cpp` 文件; 其次, 通过 **`tpu->eeptpu_set_input ( )`** NNAPI 函数来完成 EEP-TPU 的内存写入操作。

```

1. static int eeptpu_write_input(char* path_image, int in_c, int in_h, int in_w)
2. {
3.     int ret;
4.     cv::Mat cvimg_orig;
5.     cv::Mat cvimg_resized;
6.     ret = image_read_to_cv_mat(path_image, in_c, in_h, in_w, cvimg_orig,
7.                                cvimg_resized);  #(1) read img
8.     if (ret < 0) {
9.         printf("Read image fail\n");
10.        return ret;
11.    }
12.    ret = tpu->eeptpu_set_input(cvimg_resized.data, in_c, in_h, in_w);  #(2) set
13.    memory of tpu
14.    if (ret < 0) {
15.        printf("Set EEPTPU input fail\n");
16.        return ret;
17.    }
18.    printf("Set EEPTPU input ok\n");
19.    cvimg_orig.release();
20.    cvimg_resized.release();
21.    return 0;
22. }

```

- d) 构建一个 main 函数，该函数有两个输入参数：path_bin、path_image，分别用于指定 eeptpu.bin 和待计算的图片（意味着基于 path_bin 中的算法，使用 EEP-TPU 张量处理器，计算 path_image 中的图片）；

```
1. int main(int argc, char* argv[])
2. {
3.     if (argc != 3)
4.     {
5.         printf("Usage: %s eeptpu_bin_path image_path\n", argv[0]);
6.         return -1;
7.     }
8.     char *path_bin = argv[1];
9.     char *path_image = argv[2];
10.    int ret;
```

- e) 在 main 函数中，通过上述构建的 eeptpu_init 函数进行初始化操作（包括设置接口为 eeptInterfaceType_SOC 类型，根据 eeptpu.bin 的路径读入相应文件）。至此，初始化工作完成，该步骤在整个程序的生命周期内只需调用一次即可。

```
11.    ret = eeptpu_init(eeptInterfaceType_DDR, path_bin);
12.    if (ret < 0)
13.    {
14.        printf("EEPTPU init fail.\n");
15.        return ret;
16.    }
17.    printf("Network input shape: [c,h,w] = [%d,%d,%d]\n", tpu->in_c, tpu->in_h,
        tpu->in_w);
18.    // ===== eeptpu init ok, only init one time in program.
```

- f) 在 main 函数中执行数据处理操作，在这里，一次数据处理流程对应一张图片的处理过程，如果需要处理多张图片，则需要重复该流程多次。包括四个步骤。首先，使用上述构建的 eeptpu_write_input 函数读入数据；其次，使用 **tpu->eeptpu_forward(std::vector<struct EEPTPU_RESULT>& result, int pixel_type)** NNAPI 函数调用 EEP-TPU 处理器执行加速计算操作，其中，pixel_type 有两种，分别是 CHW 排列的输入数据或者 HWC 排列的输入数

据；然后，对计算结果进行后处理，在本实验中，我们使用 `get_topk` 函数对计算结果进行排序，并返回置信度最高的 5 个类别；最后，当全部计算完成后，使用 `tpu->eeptpu_close()` NNAPI 函数关闭 EEP-TPU，并释放相关资源。

```
19. // ===== loop write input and forward if you have many images.
20.   ret = eeptpu_write_input(path_image, tpu->in_c, tpu->in_h, tpu->in_w);
21.   if (ret < 0)
22.   {
23.       printf("EEPTPU write input fail\n");
24.       return ret;
25.   }
26.   double st, et;
27.   st = get_current_time();
28.   std::vector< std::pair<float, int> > top_list;
29.   std::vector<struct EEPTPU_RESULT> result;
30.   ret = tpu->eeptpu_forward(result, eepPixelType_CHW);
31.   if (ret < 0) return ret;
32.   ret = get_topk(result[0], 5, top_list);
33.   if (ret < 0) return ret;
34.   et = get_current_time();
35.   printf("EEPTPU forward ok, cost time(hw+sw): %.3f ms\n", et-st);
36.   unsigned int hwus = tpu->eeptpu_get_tpu_forward_time();
37.   printf("EEPTPU hw cost: %.3f ms\n", (float)hwus/1000);
38.   printf("Result (top 5): \n");
39.   for (int i = 0; i < 5; i++)
40.   {
41.       printf(" [%3d] %.3f\n", top_list[i].second, top_list[i].first);
42.   }
43.   top_list.clear();
44.   // all images done, close eeptpu.
45.   tpu->eeptpu_close();
46.   printf("\nAll done\n");
47.   return 0; }
```

g) 最后在文件的开始指定引用的库文件；

```
1. #include <stdio.h>
```



```

2. #include <vector>
3. #include <sys/time.h>
4. #include "eeptpu.h"
5.
6. #include "opencv2/core/core.hpp"
7. #include "opencv2/highgui/highgui.hpp"
8. #include "opencv2/imgproc/imgproc.hpp"

```

h) 完整的代码位于 reference_design 目录下的 main.cpp 文件，请参考该文件完成代码设计与开发；

3.2.3.6 嵌入式应用程序的编译

在 EEP-TPU 的开发流程中包含两种编译过程，EEP-TPU 编译和 CPU 编译。其中，EEP-TPU 编译使用 EEP-TPU 编译器，输入的是深度学习框架的神经网络算法训练结果（Caffe 框架的 prototxt 和 caffemodel 格式）。CPU 编译使用 GNU C/C++ 编译器，输入的通常是 C/C++ 代码。

在 [/home/eep/embedeep/Tutorials/P3_caffe](#) 目录下点击鼠标右键打开终端程序。在终端中输入：

```
bash ./Scripts/arm-cpu-compile.sh
```

打开 arm-cpu-compile.sh 文件查看具体的编译命令如下：

```

arm-linux-gnueabihf-g++ \
-o ./MobileNet/eeptpu_mobilenet_inference ./MobileNet/main.cpp \
-l./ -l/home/eep/embedeep/EEP-SDK/EEPTPU_Library/arm32/eep/include \
-l/home/eep/embedeep/EEP-SDK/EEPTPU_Library/arm32/opencv3/include/ \
-L/home/eep/embedeep/EEP-SDK/EEPTPU_Library/arm32/eep/lib/ \
-L/home/eep/embedeep/EEP-SDK/EEPTPU_Library/arm32/opencv3/lib/ \
-Wl,-rpath,./../libs/arm32/lib/./../libs/arm32/lib/ \
-leeptpu \
-lopencv_core \
-lopencv_highgui \
-lopencv_imgproc \
-lopencv_imgcodecs \

```

```
-lopencv_videoio
```

从中我们可以获得几个关键信息：命令使用 ARM 交叉编译器 `arm-linux-gnueabihf-g++` 完成软件程序的编译工作；编译结果输出到 `./MobileNet` 目录，文件名为 `eeptpu_mobilenet_inference`；该程序调用了多个库文件，其实际运行时，需要从 `./`、`./libs/arm/eeplib`，及 `./libs/arm/opencv2/lib/` 目录分别调用库：[*eeptpu*](#)、[*opencv_core*](#)、[*opencv_highgui*](#)、[*opencv_imgproc*](#)

3.2.3.7 可执行文件的下载与执行

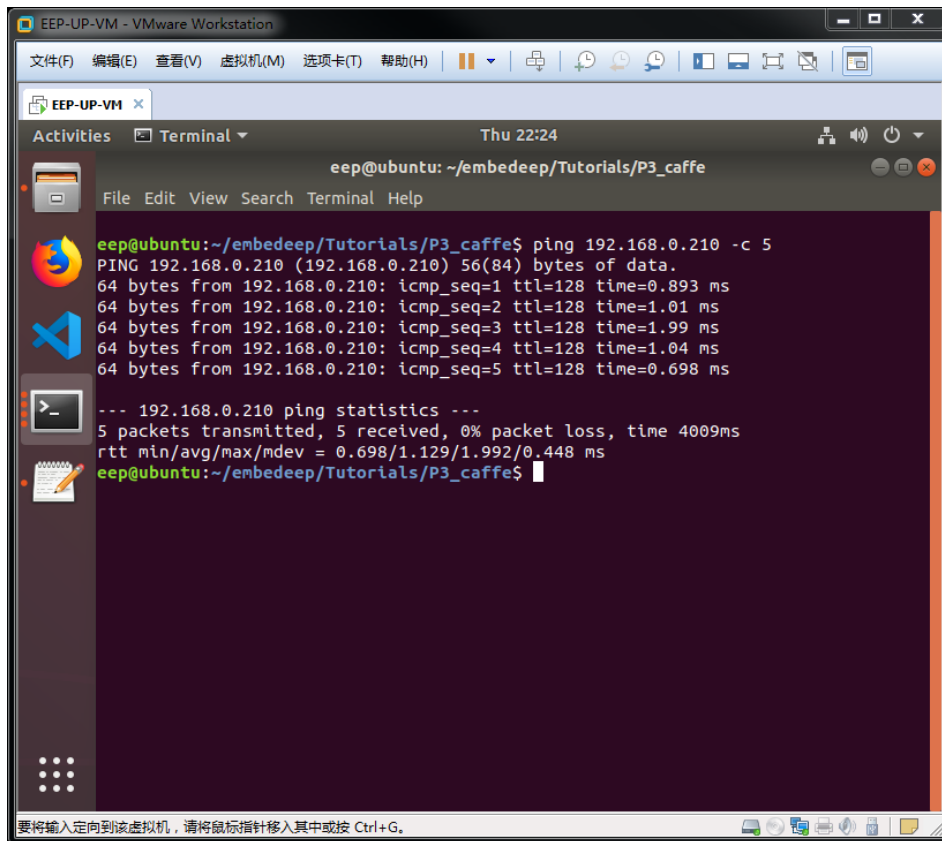
通过前述实验，我们得到了可在 ZYNQ 板卡嵌入式端执行的两个文件：[*eeptpu.bin*](#)和 [*eeptpu_mobilenet_inference*](#)。可以通过网络的方式把这两个文件发送给嵌入式系统。

正确安装 ZYNQ 开发板（预安装 TPU OS 镜像以及 TPU License），把上述运行 EEP-UP-VM 虚拟机的电脑与 ZYNQ 开发板接入同一个网络，打开电源，等待系统正确启动。

EEP-UP-VM 虚拟机环境下，在 `/home/eeplib/embeddeep/Tutorials/P3_caffe` 目录下点击鼠标右键打开终端程序。在终端中输入（此处 IP 地址需根据板卡网络地址进行设定）：

```
ping 192.168.0.200 -c 5
```

若一切正常，会出现如下日志（若 ping 程序无法结束，可以在键盘上按下 CTRL+C 按键强制结束）。如果数据包无法正确传输，检查是否 IP 地址有冲突或被改变，并且自行更改 IP 地址。

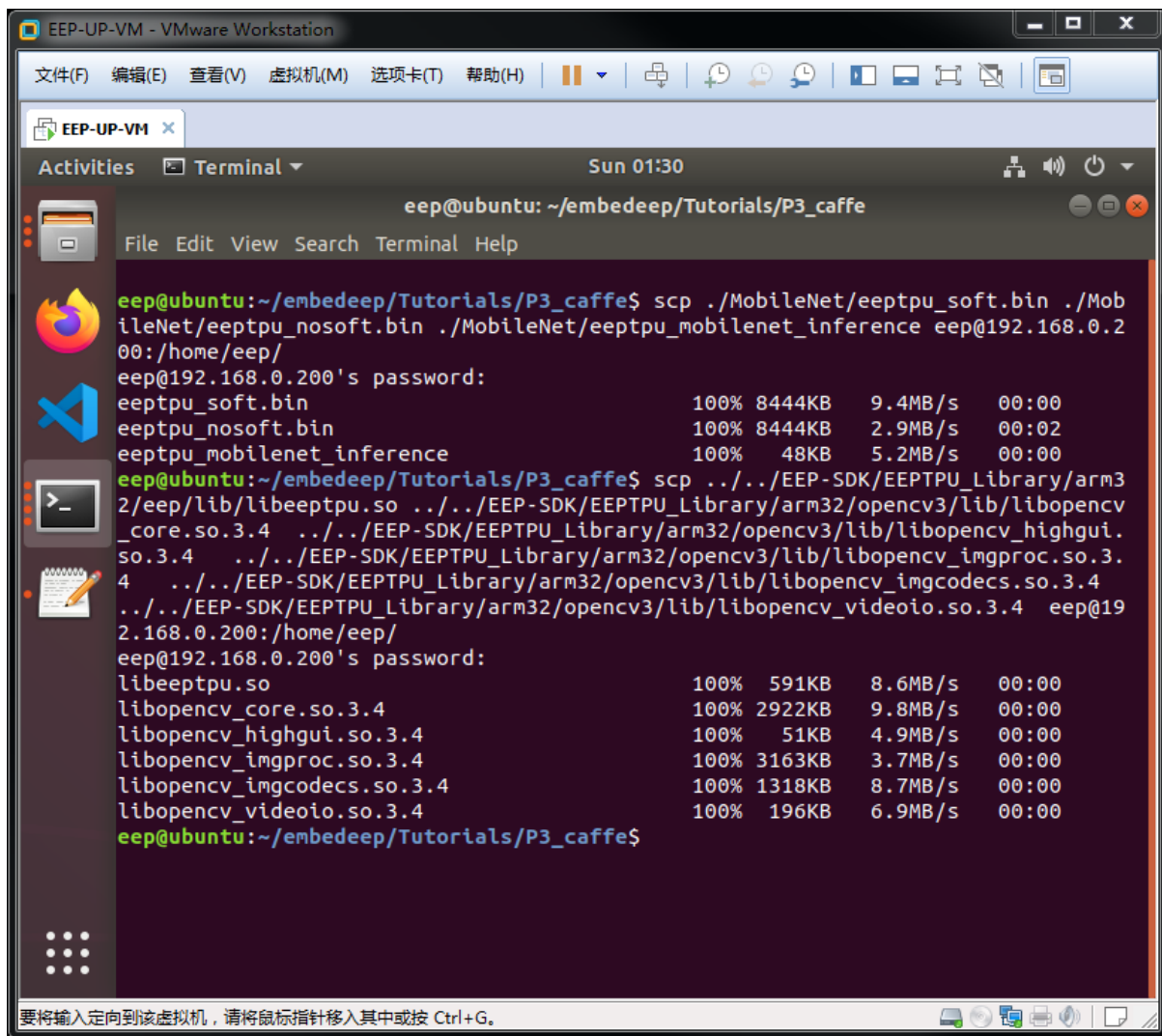


在终端中输入（输入 ZYNQ 开发板 Linux 系统的密码 *embedeep*）：

```
scp ./MobileNet/eeptpu_soft.bin ./MobileNet/eeptpu_nosoft.bin ./MobileNet/eeptpu_mobilenet_inference eep@192.168.0.200:/home/eep/
```

另外，从上节我们知道，*eeptpu_mobilenet_inference* 还调用了多个库文件，我们同时也需要把这些文件下载到目标系统中，输入如下命令：

```
scp      ../EEP-SDK/EEPTPU_Library/arm32/eep/lib/libeeptpu.so      ../EEP-
SDK/EEPTPU_Library/arm32/opencv3/lib/libopencv_core.so.3.4      ../EEP-
SDK/EEPTPU_Library/arm32/opencv3/lib/libopencv_highgui.so.3.4    ../EEP-
SDK/EEPTPU_Library/arm32/opencv3/lib/libopencv_imgproc.so.3.4    ../EEP-
SDK/EEPTPU_Library/arm32/opencv3/lib/libopencv_imgcodecs.so.3.4  ../EEP-
SDK/EEPTPU_Library/arm32/opencv3/lib/libopencv_videoio.so.3.4    eep@192.168.0.200:/home/eep/
```

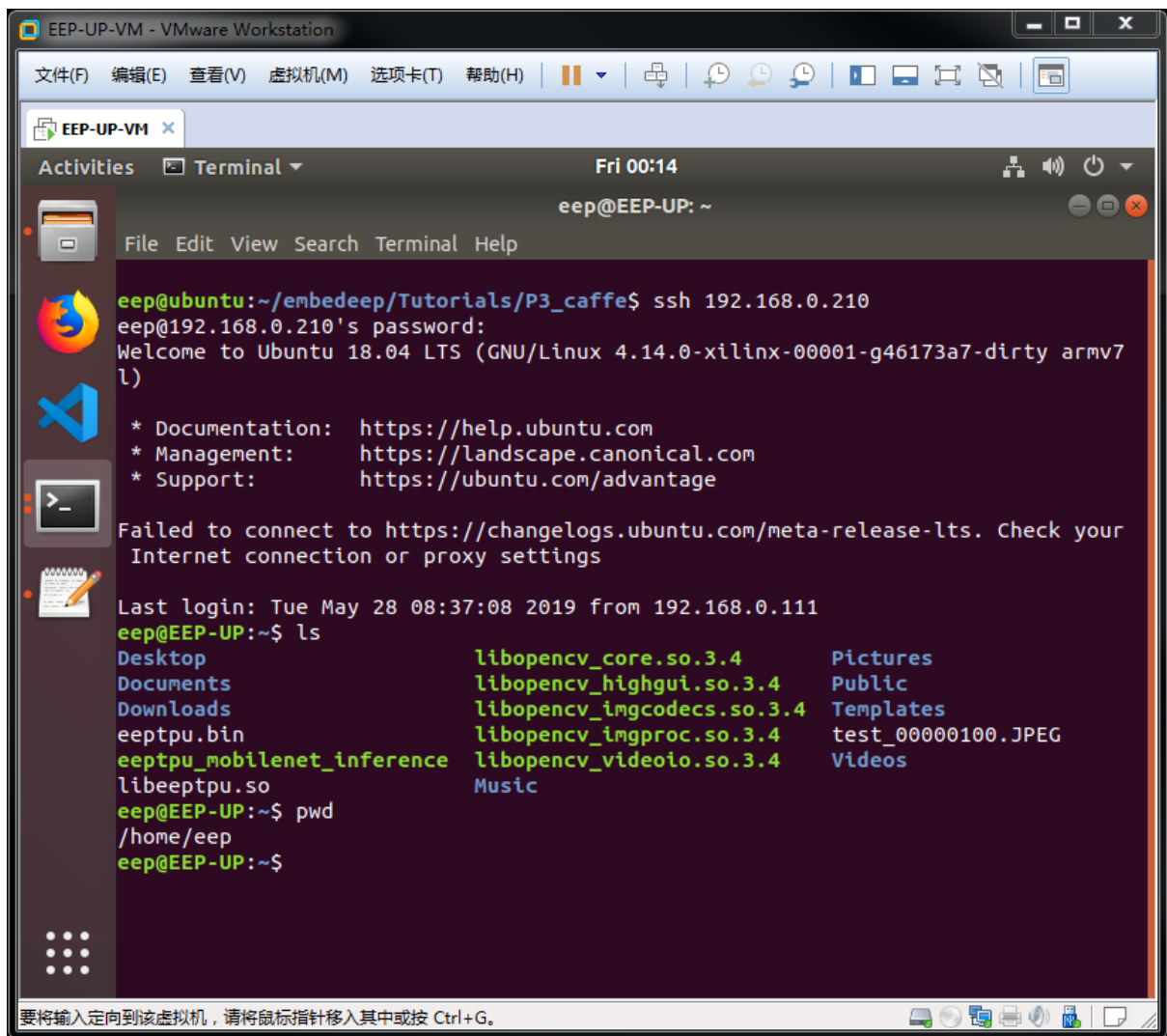


为了测试算法是否正确，我们还需一张来自测试数据集的图片：

```
scp ./DataSet/Test/test_00000100.JPEG eep@192.168.0.200:/home/eep/
```

最后，我们通过 ssh 登陆到 ZYNQ 开发板，在任意终端中输入如下命令（输入 ZYNQ 开发板 Linux 系统的密码 *embedeep*）：

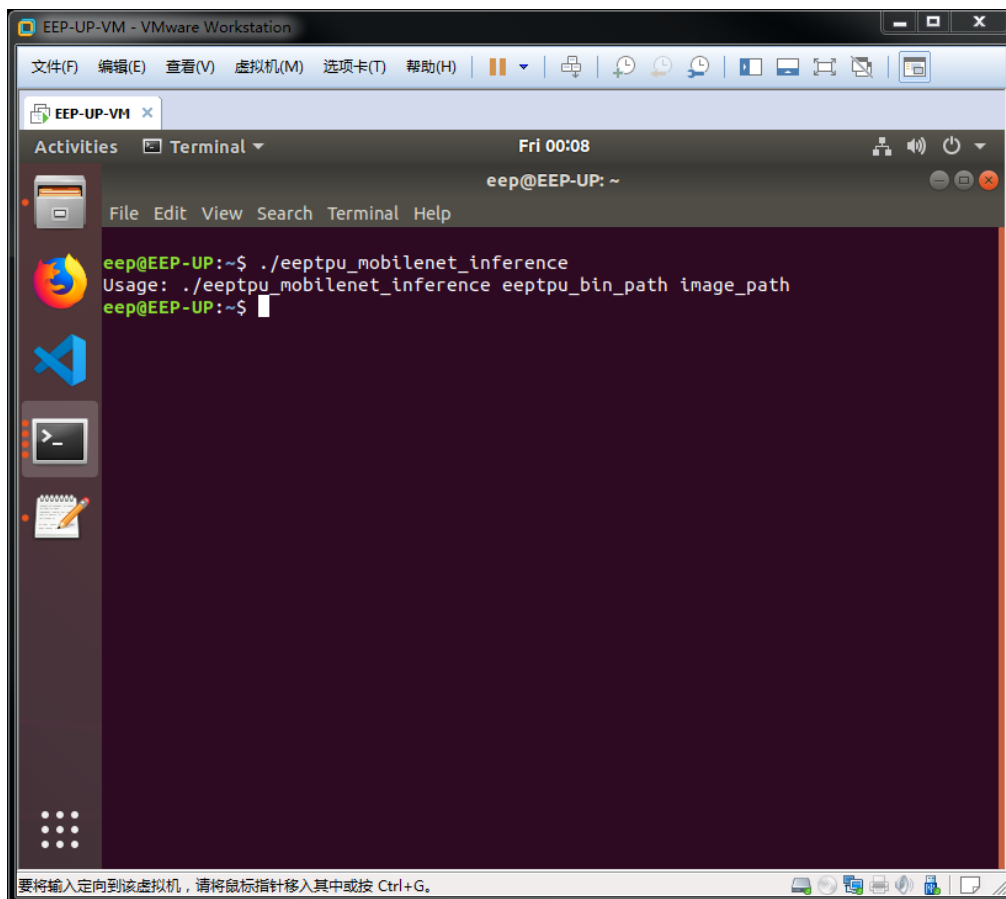
```
ssh 192.168.0.200
```



SSH 连接到 ZYNQ 开发板后，我们可以看到终端的计算名由 ubuntu 变成了 EEP-UP。从欢迎信息 “*Welcome to Ubuntu 18.04 LTS (GNU/Linux 4.14.0-xilinx-00001-g46173a7-dirty armv7l)*” 可以看到，CPU 也变成了 ARMV7 架构。终端中输入 “ls” 命令看到文件夹内的内容也发生了变化，刚才传输到 ZYNQ 开发板的所有文件都出现在了当前目录下，终端中输入 “ls” 命令看到文件夹路径变成了 “/home/eep”。此时，该终端已经运行于 ZYNQ 开发板。

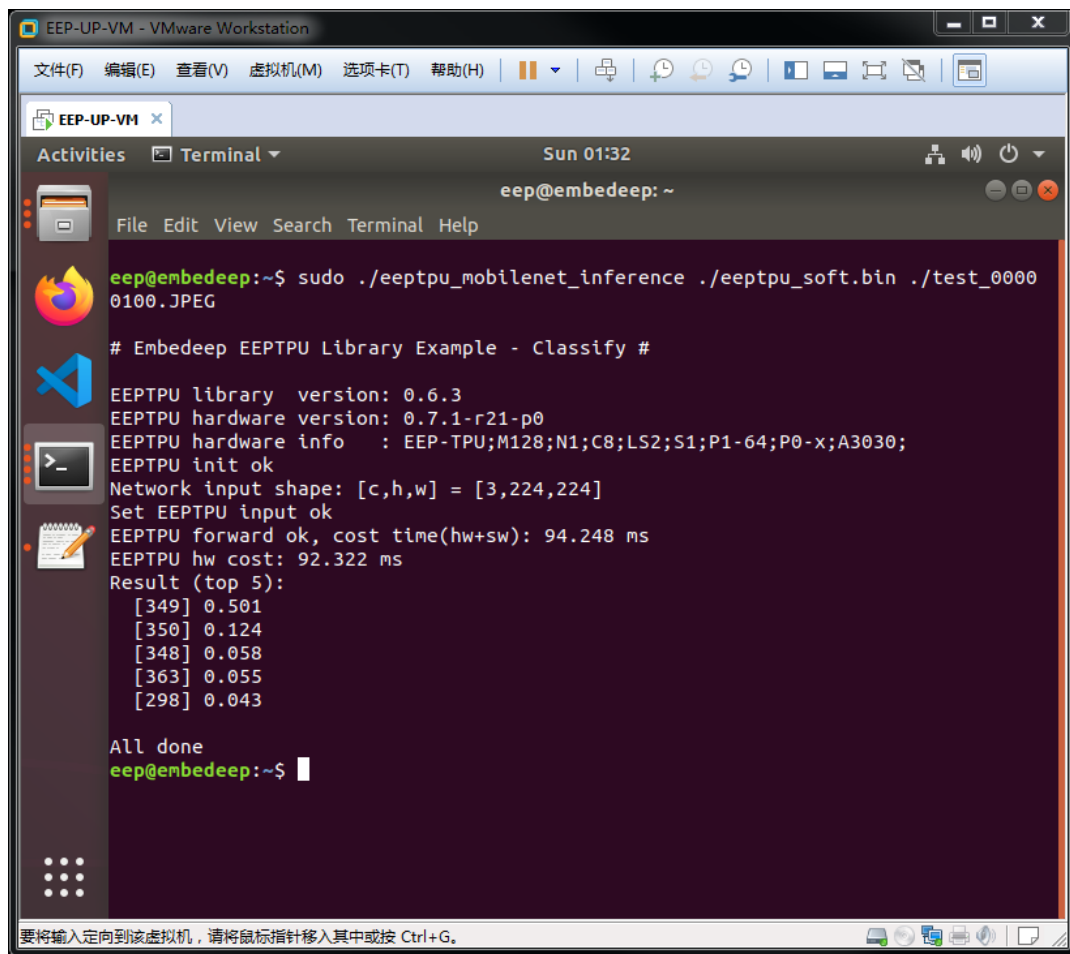
在终端中输入如下名：

```
./eeptpu_mobilenet_inference
```



正如我们之前所设计的那样，当参数不等于 3 时，打印该命令使用方法。我们在该命令后面加上 eeptpu.bin 和待计算图片的路径，如下：

```
sudo ./eeptpu_mobilenet_inference ./eeptpu_soft.bin ./test_00000100.JPEG
```



```
EEP-UP-VM - VMware Workstation
文件(F) 编辑(E) 查看(V) 虚拟机(M) 选项卡(T) 帮助(H)
EEP-UP-VM
Activities Terminal Sun 01:32
eep@embeddeep: ~
File Edit View Search Terminal Help
eep@embeddeep:~$ sudo ./eeptpu_mobilenet_inference ./eeptpu_soft.bin ./test_00000100.JPEG
# Embeddeep EEPTPU Library Example - Classify #
EEPTPU library version: 0.6.3
EEPTPU hardware version: 0.7.1-r21-p0
EEPTPU hardware info : EEP-TPU;M128;N1;C8;LS2;S1;P1-64;P0-x;A3030;
EEPTPU init ok
Network input shape: [c,h,w] = [3,224,224]
Set EEPTPU input ok
EEPTPU forward ok, cost time(hw+sw): 94.248 ms
EEPTPU hw cost: 92.322 ms
Result (top 5):
[349] 0.501
[350] 0.124
[348] 0.058
[363] 0.055
[298] 0.043
All done
eep@embeddeep:~$
```

计算结果为如下，其中中括号内的值为类别，中括号后跟的是神经网络 Softmax 层输出的对应于该类别的置信数据。

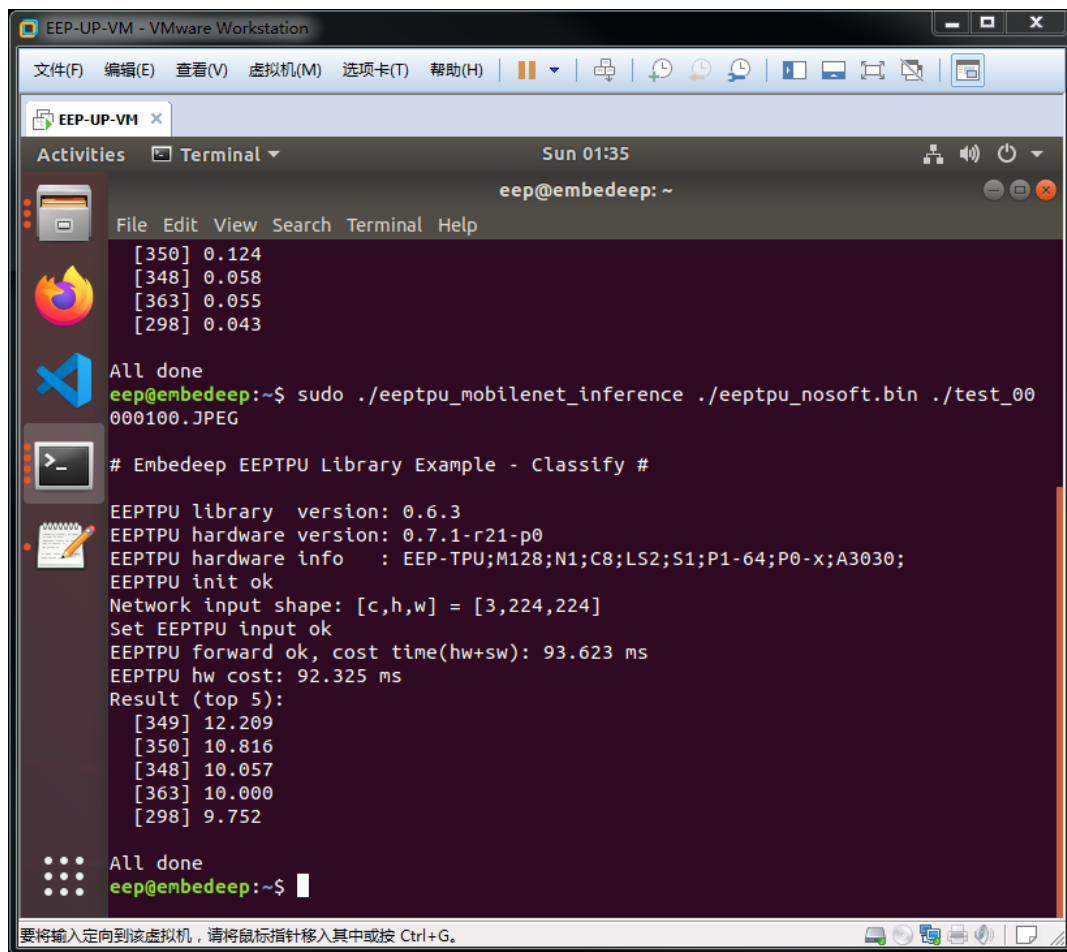
```
[349] 0.501
[350] 0.124
[348] 0.058
[363] 0.055
[298] 0.043
```

该结果与之前的 PC 结果一致。

更进一步，我们再执行 No Softmax 的网络 (*eeptpu_nosoft.bin*) 。

继续在终端中输入：

```
sudo ./eeptpu_mobilenet_inference ./eeptpu_nosoft.bin ./test_00000100.JPEG
```

```
EEP-UP-VM - VMware Workstation
文件(F) 编辑(E) 查看(V) 虚拟机(M) 选项卡(T) 帮助(H)
EEP-UP-VM
Activities Terminal Sun 01:35
eep@embeddeep: ~
File Edit View Search Terminal Help
[350] 0.124
[348] 0.058
[363] 0.055
[298] 0.043
All done
eep@embeddeep:~$ sudo ./eeptpu_mobilenet_inference ./eeptpu_nosoft.bin ./test_00
000100.JPEG
# Embeddeep EEPTPU Library Example - Classify #
EEPTPU library version: 0.6.3
EEPTPU hardware version: 0.7.1-r21-p0
EEPTPU hardware info : EEP-TPU;M128;N1;C8;LS2;S1;P1-64;P0-x;A3030;
EEPTPU init ok
Network input shape: [c,h,w] = [3,224,224]
Set EEPTPU input ok
EEPTPU forward ok, cost time(hw+sw): 93.623 ms
EEPTPU hw cost: 92.325 ms
Result (top 5):
[349] 12.209
[350] 10.816
[348] 10.057
[363] 10.000
[298] 9.752
All done
eep@embeddeep:~$
```

计算结果为如下，其中中括号内的值为类别，中括号后跟的是神经网络 fc7 层输出的对应于该类别的置信数据。

```
[349] 12.209
[350] 10.816
[348] 10.057
[363] 10.000
[298] 9.752
```

在 `/home/eep/embeddeep/Tutorials/P3_caffe` 目录下点击鼠标右键打开终端程序。在终端中输入：

```
python ./Scripts/eval_image.py \
--proto ./MobileNet/mobilenet_deploy_nosoft.prototxt \
--model ./MobileNet/trained_results/solver_iter_500.caffemodel \
--image ./DataSet/Test/test_00000100.JPEG
```

得到结果如下，其中的 *Probability* 已不再是 Softmax 层输出的概率，而是 fc7 层的输出数据，对比上述计算结果，基本一致。需要注意的是，PC 端的 Caffe 框架采用 32 位浮点数计算，而 EEP-TPU 采用 16 位浮点数计算。由于不同的精度、不同的软件实现以及不同的体系结构，嵌入式人工智能计算一般很难得到与 PC 计算数值完全相同结果。但是，从整体的 Top1/Top5 排序及对应的准确率来看，其结果是一致的。

```
cost time: 260.421143 ms
Label 349, Probability 12.25 -- n02415577 bighorn, bighorn sheep, cimarron,
Rocky Mountain bighorn, Rocky Mountain sheep, Ovis canadensis
Label 350, Probability 10.95 -- n02417914 ibex, Capra ibex
Label 363, Probability 10.25 -- n02454379 armadillo
Label 348, Probability 10.18 -- n02412080 ram, tup
Label 343, Probability 9.81 -- n02397096 warthog
```

可以结合 DataSet 目录下的 test.txt 标注文件，尝试 DataSet/Test 目录下的各种图片在 PC 与 ZYNQ 两种不同平台的计算。

3.3 基于 Caffe 框架的深度学习目标检测算法实验

3.3.1 实验目的

1. 掌握 Ubuntu 系统下基于 Caffe 框架的深度学习算法开发方法和流程；
2. 掌握深度学习目标检测算法开发方法和流程
2. 掌握嵌入式人工智能推理计算的开发方法和流程；
3. 掌握自定义数据集的制作步骤；
4. 掌握 EEP-TPU 算法编译与部署流程；
5. 掌握 EEP-TPU 嵌入式应用程序的基本开发方法和流程；

3.3.2 实验内容

采用 YOLO-V3 经典网络，基于自定义数据集，使用 CAFFE 框架，完成从算法训练、EEP-TPU 编译到可执行文件部署的开发流程实验，并最终在 ZYNQ 开发板上部署用于目标检测的神经网络算法，实现图像目标检测任务。该实验的工程目录位于 `/home/eep/embedeep/Tutorials/P3_caffe_YOLOV3/`

3.3.3 实验步骤

3.3.3.1 实验环境搭建

与 3.2.3.1 所描述的步骤相同，请参考该章节。

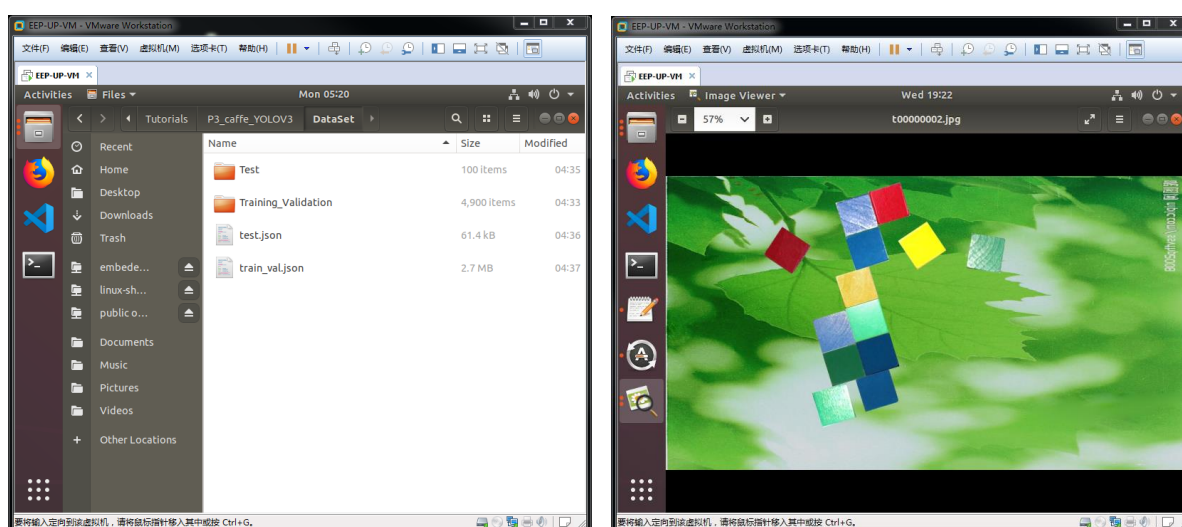
实验环境已预先搭建完毕，打开虚拟机后，进入 `/home/eep/embedeep/Tutorials/P3_caffe_YOLOV3/` 目录，按照后续步骤可以完成本实验。

3.3.3.2 制作数据集

本实验使用自制数据集，该数据集包含“红、黄、蓝、绿”4种颜色的色块，共计5000张图片，每种颜色约1.5万~2万个样本。

该数据被划分为三类：训练数据、验证数据、测试数据。其中，训练数据用于训练模型，验证数据用于在训练过程中检验模型的准确率，测试数据用于评价模型的最终准确率。我们已经完成了数据集的划分，原始数据图片存放于本实验目录 [*/home/eed/embedeep/Tutorials/P3_caffe_YOLOV3/DataSet/raw*](#)。

(1) 探索数据集。进入 [*DataSet/raw*](#) 目录，可以看到 [*Training Validation/Test*](#) 两个文件夹。该文件夹存放的原始图片分别用于训练/验证和测试。可以打开任一文件夹内的任意图片查看图片内容。



另外，还有2个文本文件，其中，*test.json* 是测试数据集的图片标注信息，*train_val.json* 是训练和验证数据集的图片标注信息。

打开*test.json* 文件，文件以json格式记录了每个色块的坐标和类别，例如，*Test* 文件夹内的 *t00000002.jpg*（如上图），其对应的json标注信息如下：

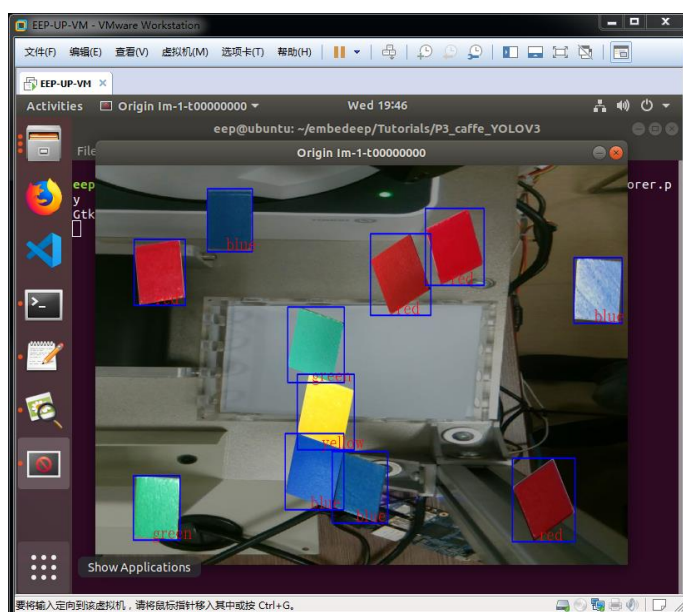
```
{"ID": "t00000002", "gtboxes": [  
  {"tag": "green", "vbox": [384, 410, 107, 106]},  
  {"tag": "blue", "vbox": [359, 328, 107, 106]},  
  {"tag": "green", "vbox": [440, 300, 107, 107]},  
  {"tag": "blue", "vbox": [465, 383, 108, 108]},  
  {"tag": "yellow", "vbox": [416, 221, 106, 103]},  
  {"tag": "blue", "vbox": [440, 124, 108, 108]},  
  {"tag": "blue", "vbox": [416, 41, 106, 107]},  
]}
```

```
{
  "tag": "green", "vbox": [359, 507, 108, 109]},
  "tag": "blue", "vbox": [465, 473, 107, 105]},
  "tag": "red", "vbox": [496, 15, 104, 106]},
  "tag": "red", "vbox": [239, 124, 118, 117]},
  "tag": "yellow", "vbox": [569, 111, 119, 119]},
  "tag": "green", "vbox": [737, 140, 96, 98]}}
```

其中，*ID* 字段表示该标注信息对应的文件名。*gtboxes* 字段表示 *N* 个目标物体的标注信息，每条标注由两个字段 *tag* 和 *vbox* 组成，tag 表示类别名，vbox 表示坐标，该坐标以[x,y,w,h]的形式呈现。

我们可以浏览标注图片。在 [/home/eed/embedeep/Tutorials/P3_caffe_YOLOV3](#) 目录下点击鼠标右键打开终端程序。在终端中输入：

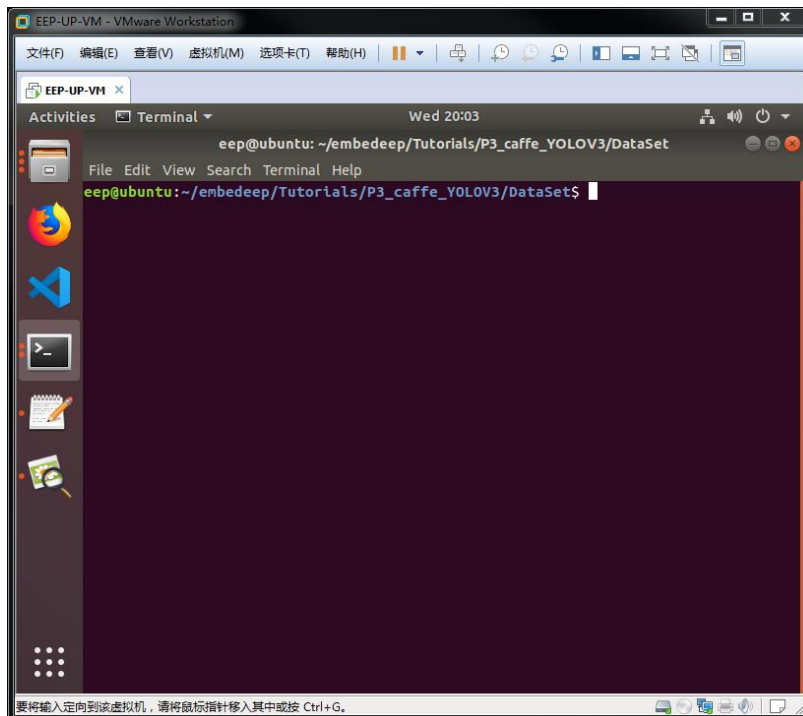
```
python ./Scripts/img-explorer.py
```



执行上述脚本将显示 DataSet/Test 目录下所有带标注的图片，而该标注信息从 DataSet/test.json 中获取。点击该图片，按下“回车键”将会显示下一张图，按下“q”键将退出标注图片浏览。

(2) 创建 LMDB 数据库

我们需要按照一定的格式来构建数据库目录。在 `/home/eep/embedeep/Tutorials/P3_caffe_YOLOV3/DataSet` 目录下点击鼠标右键打开终端程序。



首先，新建 `JPEGImages` 目录用于保存训练和验证图片，新建 `Annotations` 目录用于保存 xml 格式的标注信息，新建 `ImageSets/Main` 目录用于保存数据集图片的文本列表，在终端中输入：

```
mkdir JPEGImages
mkdir Annotations
mkdir ImageSets
mkdir ImageSets/Main
cp ./raw/Training_Validation/* ./JPEGImages
cp ./raw/train_val.json ./
python ../Scripts/gen-annotation.py
```

数据准备完毕后，我们使用 Caffe 框架中已集成的 LMDB 转换工具来生成训练所需的数据库。

在 `/home/eep/embedeep/Tutorials/P3_caffe_YOLOV3` 目录下点击鼠标右键打开终端程序。在终端中输入：

```
bash ./Scripts/create_list.sh  
bash ./Scripts/create_data.sh
```

从打印的日志可以看到，该脚本分别创建了 `DataSet_trainval_lmdb` 和 `DataSet_test_lmdb` 两类数据库分别用于训练和测试，其中，训练数据库包含 3920 个文件（图片），测试数据库包含 980 个文件（图片）。

3.3.3.3 算法训练

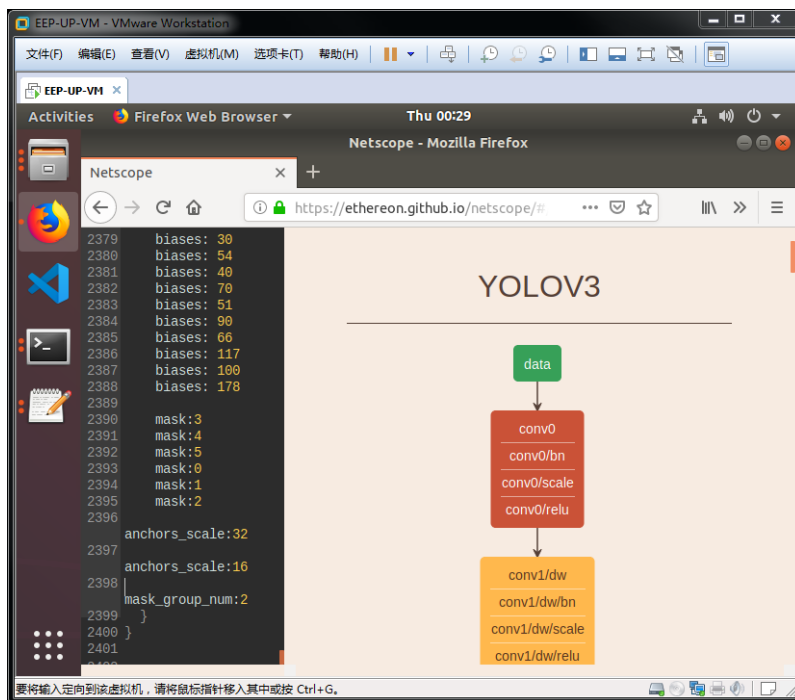
(1) 构建神经网络结构

本实验采用 YOLO-V3 算法完成目标检测任务。该算法的神经网络结构在 `yolov3_train.prototxt`、`yolov3_val.prototxt`、`yolov3_deploy.prototxt` 三个文件中定义。其中，`yolov3_train.prototxt` 是训练网络，其使用训练数据实现权重参数的学习与更新；`yolov3_val.prototxt` 是验证网络，其使用验证数据对某个阶段权重参数的准确率进行评估；`yolov3_deploy.prototxt` 是部署网络，是最终使用时的形态（去掉了训练相关的层和参数）。

在 `/home/eep/embedeep/Tutorials/P3_caffe_YOLOV3` 目录下点击鼠标右键打开终端程序。在终端中输入如下命令可以打开文本编辑工具查看上述网络的文本描述：

```
code -n .
```

我们还可以通过可视化工具来探索该神经网络的结构。打开浏览器（建议使用 Chrome 或 Firefox），输入如下网址 <https://ethereon.github.io/netscope/#/editor>。接着在左栏输入 `yolov3_deploy.prototxt` 文件的所有内容（copy-paste），并按下键盘的 **Shift+Enter** 按键。最终，右栏会出现该神经网络的可视化显示。



(2) 设置训练控制文件

训练控制文件 `solver.prototxt` 位于 YOLOV3 文件夹内，其内容与分类网络的训练控制文件类似（具体可参考 3.2.3.3 算法训练章节中有关训练控制文件的描述），具体如下：

```
train_net: "./YOLOV3/yolov3_train.prototxt"
test_net: "./YOLOV3/yolov3_val.prototxt"
test_iter: 100
test_interval: 100
base_lr: 0.0005
display: 10
max_iter: 200
lr_policy: "multistep"
gamma: 0.5
weight_decay: 0.00005
snapshot: 100
snapshot_prefix: "./YOLOV3/trained_results"
solver_mode: CPU
debug_info: false
snapshot_after_train: true
```

```
test_initialization: false
average_loss: 10
stepvalue: 10000
stepvalue: 20000
stepvalue: 30000
stepvalue: 40000
iter_size: 9
type: "RMSPProp"
eval_type: "detection"
ap_version: "MaxIntegral"
show_per_class_result: true
```

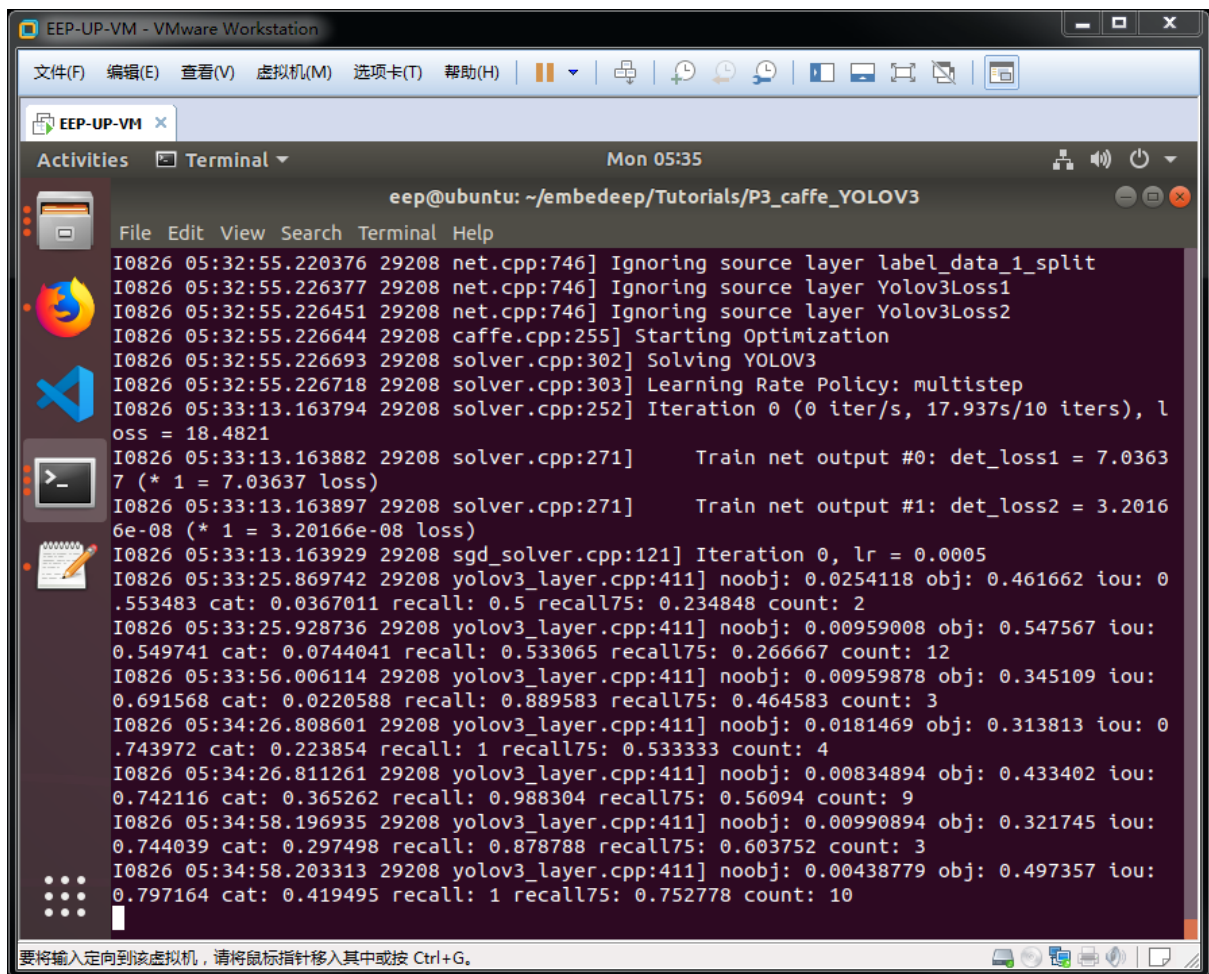
其中，*train_net* 字段指定了训练网络结构的描述文件，*test_net* 字段指定了验证网络结构的描述文件，*base_lr* 字段指定了基础的 learning rate，而该 learning rate 的变化策略在 *lr_policy* 字段中指定（在本例子中，learning rate 按照多级递减）。

(3) 执行训练

上述文件全部准备完毕后，在 */home/eep/embedeep/Tutorials/P3_caffe_YOLOV3* 目录下点击鼠标右键打开终端程序。在终端中输入：

```
bash ./Scripts/train.sh
```

train.sh 文件只有一条命令：*caffe train --solver ./YOLOV3/solver.prototxt --weights ./YOLOV3/mobilenet_pretrain.caffemodel*。其中，*--solver* 指定了上述训练控制文件，*--weight* 指定了预训练权重参数。运行过程如下：



```
eep@ubuntu: ~/embedeep/Tutorials/P3_caffe_YOLOV3
I0826 05:32:55.220376 29208 net.cpp:746] Ignoring source layer label_data_1_split
I0826 05:32:55.226377 29208 net.cpp:746] Ignoring source layer Yolov3Loss1
I0826 05:32:55.226451 29208 net.cpp:746] Ignoring source layer Yolov3Loss2
I0826 05:32:55.226644 29208 caffe.cpp:255] Starting Optimization
I0826 05:32:55.226693 29208 solver.cpp:302] Solving YOLOV3
I0826 05:32:55.226718 29208 solver.cpp:303] Learning Rate Policy: multistep
I0826 05:33:13.163794 29208 solver.cpp:252] Iteration 0 (0 iter/s, 17.937s/10 iters), l
oss = 18.4821
I0826 05:33:13.163882 29208 solver.cpp:271]      Train net output #0: det_loss1 = 7.0363
7 (* 1 = 7.03637 loss)
I0826 05:33:13.163897 29208 solver.cpp:271]      Train net output #1: det_loss2 = 3.2016
6e-08 (* 1 = 3.20166e-08 loss)
I0826 05:33:13.163929 29208 sgdsolver.cpp:121] Iteration 0, lr = 0.0005
I0826 05:33:25.869742 29208 yolov3_layer.cpp:411] noobj: 0.0254118 obj: 0.461662 iou: 0
.553483 cat: 0.0367011 recall: 0.5 recall75: 0.234848 count: 2
I0826 05:33:25.928736 29208 yolov3_layer.cpp:411] noobj: 0.00959008 obj: 0.547567 iou:
0.549741 cat: 0.0744041 recall: 0.533065 recall75: 0.266667 count: 12
I0826 05:33:56.006114 29208 yolov3_layer.cpp:411] noobj: 0.00959878 obj: 0.345109 iou:
0.691568 cat: 0.0220588 recall: 0.889583 recall75: 0.464583 count: 3
I0826 05:34:26.808601 29208 yolov3_layer.cpp:411] noobj: 0.0181469 obj: 0.313813 iou: 0
.743972 cat: 0.223854 recall: 1 recall75: 0.533333 count: 4
I0826 05:34:26.811261 29208 yolov3_layer.cpp:411] noobj: 0.00834894 obj: 0.433402 iou:
0.742116 cat: 0.365262 recall: 0.988304 recall75: 0.56094 count: 9
I0826 05:34:58.196935 29208 yolov3_layer.cpp:411] noobj: 0.00990894 obj: 0.321745 iou:
0.744039 cat: 0.297498 recall: 0.878788 recall75: 0.603752 count: 3
I0826 05:34:58.203313 29208 yolov3_layer.cpp:411] noobj: 0.00438779 obj: 0.497357 iou:
0.797164 cat: 0.419495 recall: 1 recall75: 0.752778 count: 10
```

终端所显示的训练日志包含几点重要信息。

- Train net output #0/#1* 包含训练网络输出的两个重要信息 *det_loss1* 和 *det_loss2*，其分别来自于 *yolov3_train.prototxt* 文件中的 *Yolov3Loss1* 层和 *Yolov3Loss2* 层，是评价算法训练过程的重要指标（loss 越低越好）。正常情况下，随着训练迭代次数的增加，loss 会不断降低。
- 网络训练过程的具体细节由 *yolov3_layer.cpp* 文件中的相应代码打印输出（该细节由训练中的推理计算过程获得，反应的是该算法针对训练集的实时计算性能结果），其中，最关键的三个指标是 cat、recall 及 recall75。cat 指标表示预测结果中类别正确的概率，recall 指标表示 IOU > 0.5 时坐标框正确的概率，recall75 指标表示 IOU > 0.75 时坐标框正确的概率。神经网络目标检测算法训练的目标，就是使得三个指标越高越好，甚至接近 1。

- c) 每隔 100 次迭代（该次数可在 `solver.prototxt` 中指定），便可采用 [`yolov3\_val.prototxt`](#) 文件，使用验证数据集对算法性能进行测试。测试结果可以显示每隔类别的准确率。
- d) 每隔 100 次迭代，中间训练结果（snapshot）会以 `caffemodel` 和 `solverstate` 两种形态进行保存于 `MobileNet/trained_results` 文件夹内。其中，`caffemodel` 是权重参数，可作为下一次训练的预训练权重初始值；`solverstate` 是临时状态，以 `solverstate` 为起点开始训练等于接着上次结束的状态继续执行，也即恢复训练操作。
- e) 经过 200 次迭代后（训练控制文件 `max_iter` 所定义），训练结束，最终验证所得的准确率为 96% 左右。

(4) 使用神经网络算法

训练完成后，可以在 PC 上通过 `caffe` 框架直接使用该神经网络算法实现目标检测任务。在 [`/home/eep/embedeep/Tutorials/P3\_caffe\_YOLO`](#) 目录下点击鼠标右键打开终端程序。在终端中输入：

```
bash Scripts/eval_yolov3.sh
```

`eval_yolov3.sh` 脚本的内容如下：

```
/home/eep/embedeep/EEP-SDK/caffe/build/examples/yolo/yolo_detect.bin  
YOLOV3/yolov3_deploy.prototxt  
YOLOV3/trained_results/solver_iter_500.caffemodel -resize_mode 0 -cpu_mode  
cpu -file_type image -wait_time 30000 -mean_value 1,1,1 -normalize_value  
0.007843 -confidence_threshold 0.3 --  
indir ../../Tutorials/P3_caffe_YOLOV3/Test_imgs/
```

其中，

`yolo_detect.bin` 是 PC 端使用的测试程序，

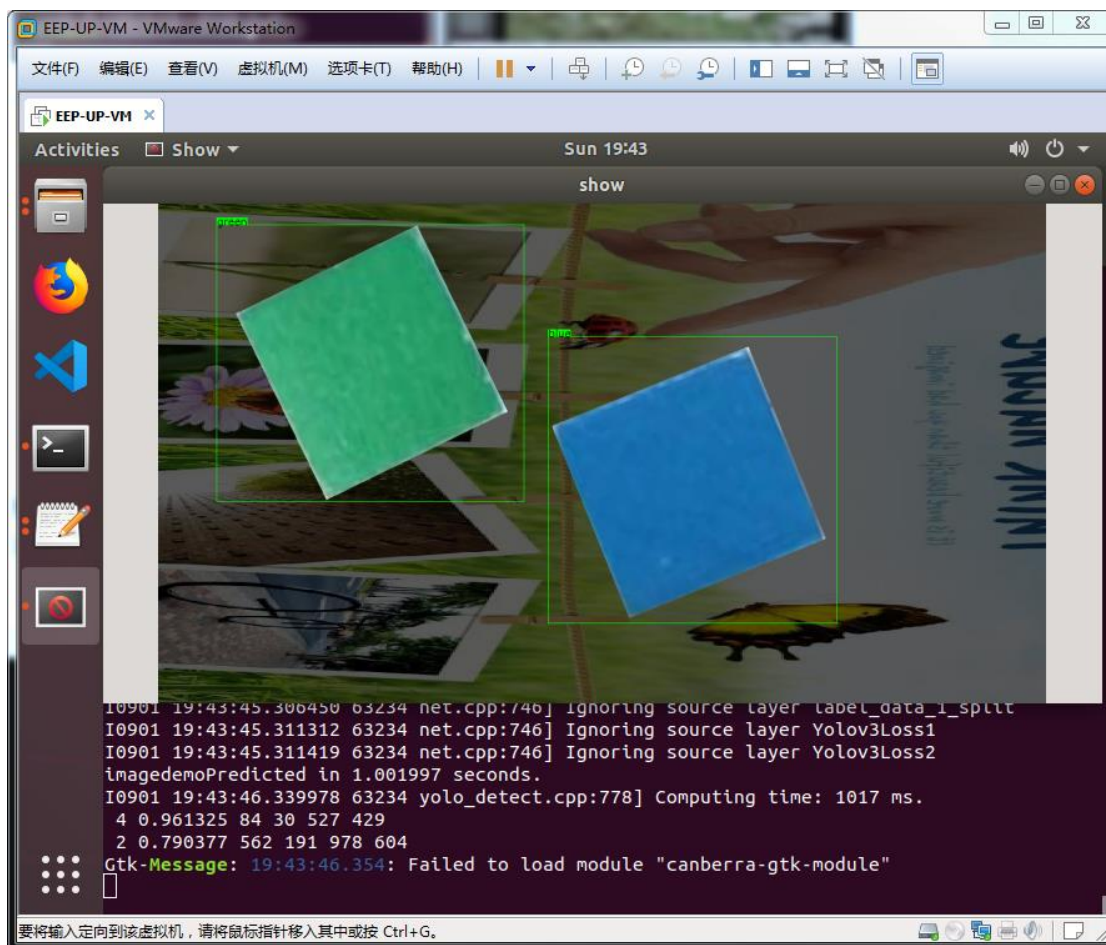
`yolov3_deploy.prototxt` 是推理计算所使用的网络，

`solver_iter_500.caffemodel` 是训练所得的模型参数文件，

`-wait_time` 可以指定两次计算的间隔时间，

`-confidence_threshold` 可以指定置信度过滤规则，

--indir可以指定测试图片所在的文件夹。



从打印得到的日志可以看到具体的检测结果如下：

```
4 0.961325 84 30 527 429
```

```
2 0.790377 562 191 978 604
```

其中，第一个整数值是**类别**，与 Scripts 目录下的 labelmap.prototxt 文件内容一致，“3”代表 yellow，“1”代表 red；类别后小于 1 的浮点数为**置信度**，只有置信度大于上述所设置 *confidence_threshold* 值的条目才会显示出来；置信度后面的四个整数代表对应检测目标矩形框的**坐标值**，该坐标以矩形框左下角和右上角 x/y 坐标的形式定义。

从显示的图像信息可以看到检测结果（包括类别和矩形框位置）是否正确，至此，算法训练阶段结束。

3.3.3.4 EEP-TPU 算法编译

人工智能算法训练完毕后，会得到神经网络描述和权重参数两个文件，在上节所述的算法训练步骤完成后，可以得到位于 `/home/eep/embedeep/Tutorials/P3_caffe_YOLOV3/YOLOV3` 目录下的神经网络（部署）描述文件 `yolov3_deploy.prototxt`，和位于 `/home/eep/embedeep/Tutorials/P3_caffe_YOLOV3/YOLOV3/trained_results` 目录下的训练权重文件 `solver_iter_500.caffemodel`。

然而，`yolov3_deploy.prototxt` 和 `solver_iter_500.caffemodel` 是运行在 X86 体系计算机上的 Caffe 框架所使用的格式。要想把该人工智能算法部署到嵌入式端，还需要进行编译和转换，在基于 EEP-TPU 的嵌入式人工智能计算平台上，可以使用相应的编译器 `eeptpu_compiler` 来完成该转换任务。

一个典型的 `eeptpu_compiler` 编译命令如下所示：

```
eeptpu_compiler_c8 \  
--mean '1.0,1.0,1.0' \  
--norm '0.007843,0.007843,0.007843' \  
--prototxt ./YOLOV3/yolov3_deploy.prototxt \  
--caffemodel ./YOLOV3/trained_results/solver_iter_500.caffemodel \  
--image ./DataSet/JPEGImages/t00000100.jpg \  
--output ./YOLOV3
```

其中，

-- *mean* 指定均值，该值为浮点，分别对应输入图像的 3 个通道，该值与 Caffe 中 Data 层所指定的 `mean_value` 值一致（顺序也一致）。

-- *norm* 指定归一化值，该值为浮点，分别对应输入图像的 3 个通道，该值与 Caffe 中 Data 层所指定的 `scale` 值一致（3 个通道内容相同，都为 `scale` 值）。

-- *prototxt* 指定神经网络描述文件的路径

-- *caffemodel* 指定神经网络权重参数文件的路径

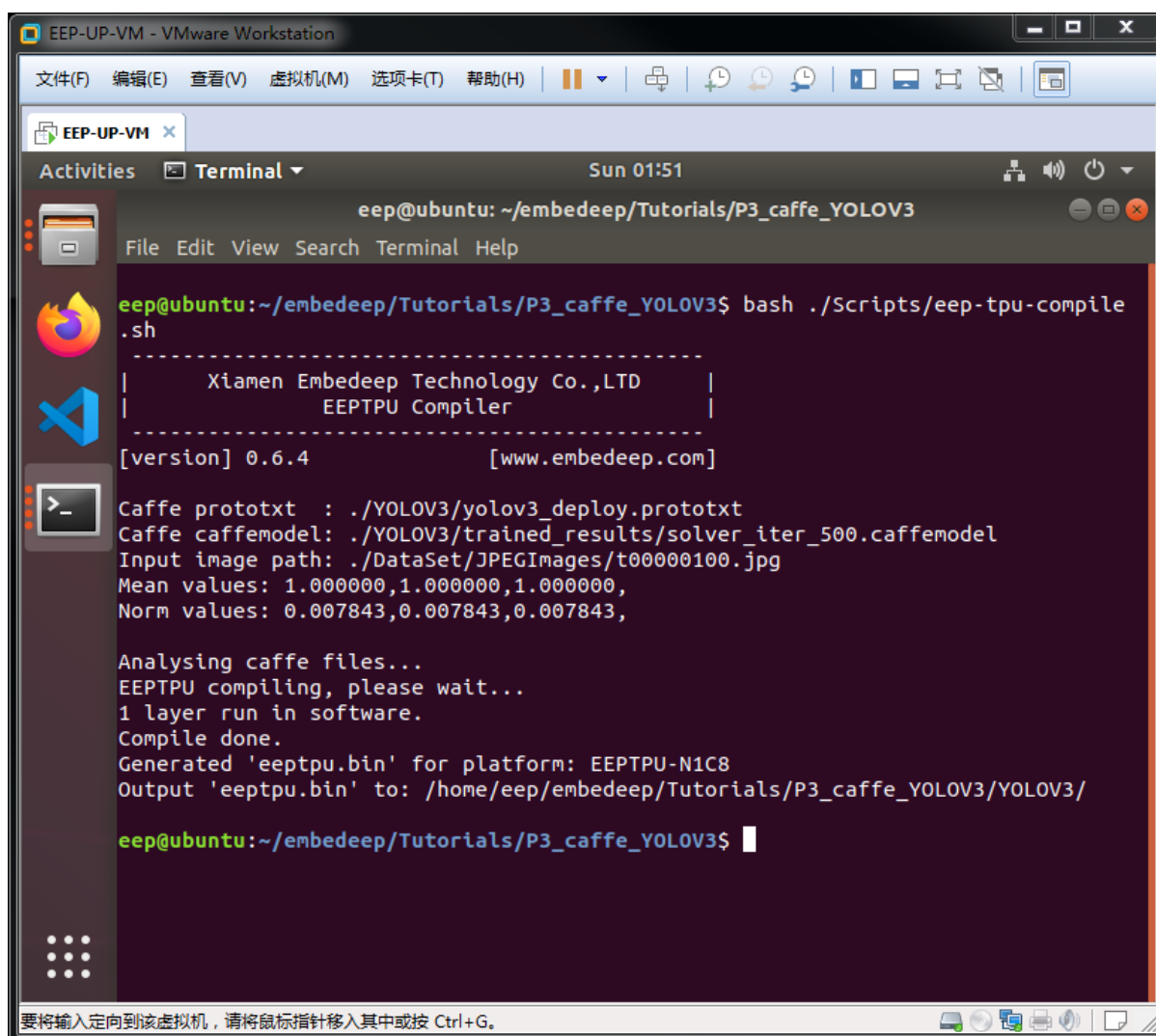
-- *image* 指定来自 *DataSet* 数据集的任意一张图片

-- *output* 指定编译结果的输出目录

在 `/home/eep/embedeep/Tutorials/P3_caffe_YOLOV3` 目录下点击鼠标右键打开终端程序。在终端中输入：

```
bash ./Scripts/eep-tpu-compile.sh
```

运行结果如下，终端日志显示了所有信息，包括运行是否成功。



```
EEP-UP-VM - VMware Workstation
文件(F) 编辑(E) 查看(V) 虚拟机(M) 选项卡(T) 帮助(H)
EEP-UP-VM x
Activities Terminal Sun 01:51
eep@ubuntu: ~/embedeep/Tutorials/P3_caffe_YOLOV3
File Edit View Search Terminal Help
eep@ubuntu:~/embedeep/Tutorials/P3_caffe_YOLOV3$ bash ./Scripts/eep-tpu-compile.sh
-----
| Xiamen Embedeep Technology Co.,LTD |
| EEPTPU Compiler |
-----
[version] 0.6.4 [www.embedeep.com]

Caffe prototxt : ./YOLOV3/yolov3_deploy.prototxt
Caffe caffemodel: ./YOLOV3/trained_results/solver_iter_500.caffemodel
Input image path: ./DataSet/JPEGImages/t00000100.jpg
Mean values: 1.000000,1.000000,1.000000,
Norm values: 0.007843,0.007843,0.007843,

Analysing caffe files...
EEPTPU compiling, please wait...
1 layer run in software.
Compile done.
Generated 'eeptpu.bin' for platform: EEPTPU-N1C8
Output 'eeptpu.bin' to: /home/eep/embedeep/Tutorials/P3_caffe_YOLOV3/YOLOV3/

eep@ubuntu:~/embedeep/Tutorials/P3_caffe_YOLOV3$
```

正确执行上述命令，出现 “*Compile done*” 意味着编译成功，编译所生成的文件名为 **eeptpu.bin**，该文件位于 `--output` 所指定的文件夹内。

3.3.3.5 EEP-TPU 嵌入式应用开发

上述 eeptpu.bin 只能在含有 EEP-TPU 张量处理器的嵌入式系统中使用，本节，我们将构建一个简单的程序，使用 eeptpu.bin 在嵌入式端完成上述 PC 端的目标检测任务。

- a) 新建一个文本文件，命名为 main.cpp;
- b) 构建 EEP-TPU 初始化函数，仅有两个步骤。首先，通过 **tpu->eeptpu_set_interface(interface_type)** NNAPI 函数设置 EEP-TPU 的接口类型，其中 interface_type 指 EEP-TPU 与 CPU 的接口，可以有两种：SOC（一般与 ARM 或 RISC-V CPU 通过总线接口）或 PCIE（一般与 X86 CPU 通过 PCIE 接口）；其次，通过 **tpu->eeptpu_load_bin(path_bin)** NNAPI 函数把 eeptpu.bin 文件的内容写入内存。NNAPI 函数可以进一步参考附录 A. 。

```
1. static int eeptpu_init(int interface_type, char* path_bin)
2. {
3.     int ret;
4.     if (tpu == NULL) tpu = tpu->init();
5.     ret = tpu->eeptpu_set_interface(interface_type);#(1) set SOC or PCIE
6.     if (ret < 0) return ret;
7.     printf("EEPTPU library version: %s\n", tpu->eeptpu_get_lib_version());
8.     printf("EEPTPU hardware version: %s\n", tpu->eeptpu_get_tpu_version());
9.     printf("EEPTPU hardware info : %s\n", tpu->eeptpu_get_tpu_info());
10. ret = tpu->eeptpu_load_bin (path_bin); #(2) load eeptpu.bin
11. if (ret < 0) {
12.     printf("Load bin fail, ret=%d\n", ret);
13.     return ret;
14. }
15. printf("EEPTPU init ok\n");
16. return 0;
17. }
```

c) 构建 EEP-TPU 数据写入 (EEP-TPU 专属内存) 函数, 包括两个步骤。首先, 读取输入数据 (图片), 在这里我们使用 opencv 的 `cv::imread` 函数读入输入图片, 并根据神经网络的尺寸使用 `cv::resize` 函数进行图片缩放操作。相应的 `imread` 和 `resize` 操作被封装到 `image_read_to_cv_mat` 函数中, 该函数具体可以参考 `reference_design` 目录下的 `main.cpp` 文件; 其次, 通过 **`tpu->eeptpu_set_input ( )`** NNAPI 函数来完成 EEP-TPU 的内存写入操作。在这里, 需要特别注意, 和分类算法不同的是, 检测算法的后处理流程中 (如显示), 往往需要使用到原始图片。因此, 我们通过 `eeptpu_write_input` 函数读取图片时, 需要把原始图片的引用 (**`cv::Mat& cvimg_orig`**) 在 `main` 函数中定义, 并通过参数传入本函数, 从而使得该图片也能在后处理流程中被使用。

```
1. static int eeptpu_write_input(char* path_image, int in_c, int in_h, int in_w,
   cv::Mat& cvimg_orig)
2. {
3.     int ret;
4.     cv::Mat cvimg_resized;
5.     ret = image_read_to_cv_mat(path_image, in_c, in_h, in_w, cvimg_orig,
   cvimg_resized);  #(1) read img
6.     if (ret < 0) {
7.         printf("Read image fail\n");
8.         return ret;
9.     }
10.    ret = tpu->eeptpu_set_input(cvimg_resized.data, in_c, in_h, in_w);  #(2) set
   memory of tpu
11.    if (ret < 0) {
12.        printf("Set EEPTPU input fail\n");
13.        return ret;
14.    }
15.    printf("Set EEPTPU input ok\n");
16.    cvimg_resized.release();
17.    return 0;
18. }
```

d) 构建目标检测后处理函数。该函数的主要任务是构建一个包含所有目标的结
构体列表，其中，需要根据具体检测类别的名字修改 `class_names_default[]`。

```
1. static const char *class_names_default[] = {"background",
2.                                             "red", "blue", "yellow", "green"}; #define your onw class
3. struct Object
4. {
5.     cv::Rect_<float> rect;
6.     int label;
7.     float prob;
8.     char obj_class_name[32];
9. };
10. static void clear_objects()
11. {
12.     g_objects.clear();
13. }
14. static std::vector<Object> g_objects;
15.
16. static int post_process_obj_detect(cv::Mat& cvimg, struct EEPTPU_RESULT&
    result)
17. {
18.     clear_objects(); #clear the previous results
19.     if ( (result.c * result.h * result.w == 0) || (result.c != 1) )
20.     {
21.         printf("Nothing detected !\n");
22.         return 0;
23.     }
24.     class_names = (char**)class_names_default;
25.     for (int i = 0; i < result.h; i++)
26.     {
27.         const float *values = result.data + i * result.w;
28.         Object object;
29.         object.label = values[0];
30.         object.prob = values[1];
31.         object.rect.x = values[2] * cvimg.cols;
```

```

32.     object.rect.y = values[3] * cvimg.rows;
33.     object.rect.width = values[4] * cvimg.cols - object.rect.x;
34.     object.rect.height = values[5] * cvimg.rows - object.rect.y;
35.     strcpy(object.obj_class_name, class_names[object.label]);
36.     if (object.rect.x < 0) { object.rect.width += object.rect.x; object.rect.x = 0; }
37.     if (object.rect.y < 0) { object.rect.height += object.rect.y; object.rect.y = 0; }
38.     g_objects.push_back(object);
39. }
40. printf("Detection result: \n");
41. for (unsigned int i = 0; i < g_objects.size(); i++)
42. {
43.     const Object &obj = g_objects[i];
44.     printf("    [%d] %d = %.5f at %.2f %.2f %.2f x %.2f  (%s)\n", i, obj.label,
obj.prob,
45.         obj.rect.x,      obj.rect.y,      obj.rect.width,      obj.rect.height,
class_names[obj.label]);
46. }
47. return 0;
48. }
49.

```

e) 构建 Opencv 画图函数。该函数的主要作用是提供可视化的结果显示，把 d 步骤中所获得的每个目标的类别名、置信度、坐标框信息标注在原图上。

```

1. static cv::Mat draw_objects(const cv::Mat &bgr, const std::vector<Object>
&objects)
2. {
3.     cv::Mat image = bgr.clone();
4.     double font_size = 0.6;
5.     int font_bold = 1;
6.     for (int i = objects.size()-1; i >= 0; i--)
7.     {
8.         const Object &obj = objects[i];
9.         sprintf(text, "%s %.1f%%", class_names[obj.label], obj.prob * 100);
10.        int baseLine = 0;
11.        cv::rectangle(image, obj.rect, cv::Scalar(0, 255, 0), font_bold);

```

```

12.     cv::Size label_size = cv::getTextSize(text, cv::FONT_HERSHEY_SIMPLEX,
        font_size, font_bold, &baseLine);
13.     int x = obj.rect.x;
14.     int y = obj.rect.y - label_size.height - baseLine;
15.     if (y < 0) y = 0;
16.     if (x + label_size.width > image.cols) x = image.cols - label_size.width;
17.     cv::Point point;
18.     point = cv::Point(x, y + label_size.height);
19.     cv::rectangle(image, cv::Rect(cv::Point(x, y), cv::Size(label_size.width,
        label_size.height + baseLine)), cv::Scalar(128, 128, 0), CV_FILLED);
20.     cv::putText(image, text, point, cv::FONT_HERSHEY_SIMPLEX, font_size,
        cv::Scalar(0, 255, 0), font_bold);
21. }
22. return image;
23. }

```

- f) 构建一个 main 函数，该函数有两个输入参数：path_bin、path_image，分别用于指定 eeptpu.bin 和待计算的图片（意味着基于 path_bin 中的算法，使用 EEP-TPU 张量处理器，计算 path_image 中的图片）；

```

1. int main(int argc, char* argv[])
2. {
3.     if (argc != 3)
4.     {
5.         printf("Usage: %s eeptpu_bin_path image_path\n", argv[0]);
6.         return -1;
7.     }
8.     char *path_bin = argv[1];
9.     char *path_image = argv[2];
10.    int ret;

```

- g) 在 main 函数中，通过上述构建的 eeptpu_init 函数进行初始化操作（包括读入 eeptpu.bin）；通过 **eeptpu_get_input_shape ()** NNAPI 函数从 eeptpu_inith 函数已读入的 eeptpu.bin 文件中获取神经网络输入张量的

c/h/w (通道数、长、宽) 参数。至此，初始化工作完成，该步骤在整个程序的生命周期内只需调用一次即可。

```
11.  ret = eeptpu_init(eeplInterfaceType_DDR, path_bin);
12.  if (ret < 0)
13.  {
14.      printf("EEPTPU init fail.\n");
15.      return ret;
16.  }
17.  printf("Network input shape: [c,h,w] = [%d,%d,%d]\n", tpu->in_c, tpu->in_h,
        tpu->in_w);
18.  // ===== eeptpu init ok, only init one time in program.
```

h) 在 main 函数中执行数据处理操作，包括两个步骤。首先，使用上述构建的 eeptpu_write_input 函数读入数据；其次，使用 **tpu->eeptpu_forward ()** NNAPI 函数实现 EEP-TPU 加速计算。

```
19. // ===== loop write input and forward if you have many images.
20.  cv::Mat cvimg_orig;
21.  ret = eeptpu_write_input(path_image, in_c, in_h, in_w, cvimg_orig);
22.  if (ret < 0)
23.  {
24.      printf("EEPTPU write input fail\n");
25.      return ret;
26.  }
27.  std::vector<struct EEPTPU_RESULT> result;
28.  double st = get_current_time();
29.  double et;
30.  ret = tpu->eeptpu_forward(result, eepPixelType_CHW);
31.  if (ret < 0)
32.  {
33.      printf("eeptpu forward fail, ret = %d\n", ret);
34.      return ret;
35.  }
36.  et = get_current_time();
37.  printf("EEPTPU forward ok, cost time(hw+sw): %.3f ms\n", et-st);
```

```
38. unsigned int hwus = tpu->eeptpu_get_tpu_forward_time();
39. printf("EEPTPU hardware cost: %.3f ms\n", (float)hwus/1000);
```

- i) 在 main 函数中执行数据后处理操作。在本教程中，我们仅获取计算结果，将其标注信息写入原图中，并存储该附带标注信息的图片为 yolov3.jpg。可以在此加入更多的后处理步骤，例如通过网络发送。

```
40. if (result.size() == 0)
41. {
42.     printf("Nothing detected. ret=%d",ret);
43. }
44. else
45. {
46.     ret = post_process_obj_detect(cvimg_orig, result[0]);
47.     if (ret < 0) return ret;
48.     // draw object
49.     cv::Mat final_image = draw_objects(cvimg_orig, g_objects);
50.     cv::imwrite("./yolov3.jpg", final_image);
51.     printf("Saved result image to: ./yolov3.jpg\n");
52.     if (final_image.empty() == false) final_image.release();
53. }
54. cvimg_orig.release();
55. tpu->eeptpu_close();
56. clear_objects();
57. printf("\nAll done\n");
58. return 0;
59. }
```

- j) 最后在文件的开始指定引用的库文件；

```
1. #include <stdio.h>
2. #include <vector>
3. #include <sys/time.h>
```



```
4. #include "eeptpu.h"
5.
6. #include "opencv2/core/core.hpp"
7. #include "opencv2/highgui/highgui.hpp"
8. #include "opencv2/imgproc/imgproc.hpp"
```

完整的代码位于 reference_design 目录下的 main.cpp 文件，请参考该文件完成代码设计与开发。

3.3.3.6 嵌入式应用程序的编译

在 EEP-TPU 的开发流程中包含两种编译过程，EEP-TPU 编译和 CPU 编译。其中，EEP-TPU 编译使用 EEP-TPU 编译器，输入深度学习框架的神经网络算法训练结果（Caffe 框架的 prototxt 和 caffemodel 格式）。CPU 编译使用 GNU C/C++ 编译器，输入通常是 C/C++ 软件代码。

在 [/home/eeep/embedeep/Tutorials/P3_caffe_YOLOV3](#) 目录下点击鼠标右键打开终端程序。在终端中输入：

```
bash ./Scripts/arm-cpu-compile.sh
```

打开 arm-cpu-compile.sh 文件查看具体的编译命令如下：

```
arm-linux-gnueabi-g++ \
-o ./YOLOV3/eeptpu_yolov3_inference \
./YOLOV3/main.cpp \
-I./ -I/home/eeep/embedeep/EEP-SDK/EEPTPU_Library/arm32/eeep/include \
-I/home/eeep/embedeep/EEP-SDK/EEPTPU_Library/arm32/opencv3/include/ \
-L/home/eeep/embedeep/EEP-SDK/EEPTPU_Library/arm32/eeep/lib/ \
-L/home/eeep/embedeep/EEP-SDK/EEPTPU_Library/arm32/opencv3/lib/ \
-Wl,-rpath,./:/lib/arm32/lib:/lib/arm32/lib/ \
-leeptpu \
-lopenv_core \
-lopenv_highgui \
-lopenv_imgproc \
-lopenv_imgcodecs \
-lopenv_videoio
```

从中我们可以获得几个关键信息：命令使用 ARM 的交叉编译器 `arm-linux-gnueabihf-g++` 完成软件程序的编译工作；编译结果输出到 `./YOLOV3` 目录，文件名为 `eeptpu_yolov3_inference`；该程序调用了多个库文件，其在实际运行时，需要从 `./`，`./libs/arm/eeplib`，及 `./libs/arm/opencv2/lib/` 目录分别调用库：[*eeptpu*](#)、[*opencv_core*](#)、[*opencv_highgui*](#)、[*opencv_imgproc*](#)。

3.2.3.7 可执行文件的下载与执行

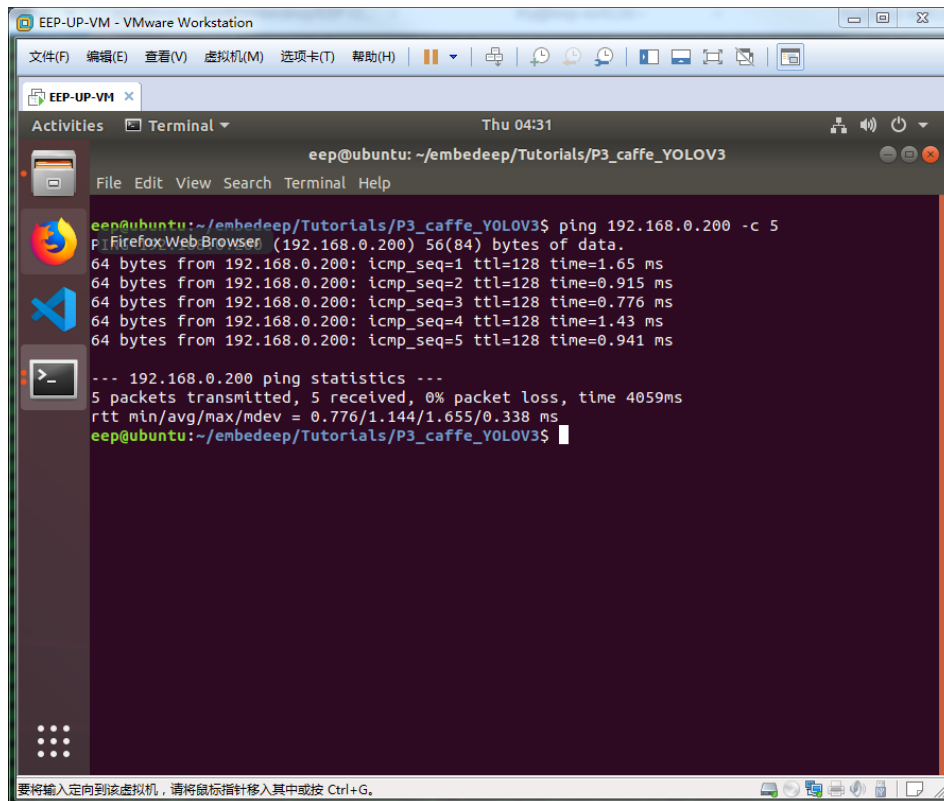
通过前述实验，我们得到了可在 ZYNQ 嵌入式端执行的两个文件：[*eeptpu.bin*](#) 和 [*eeptpu_mobilenet_inference*](#)。可以通过网络的方式把这两个文件发送给嵌入式系统。

正确安装 ZYNQ 开发板（预安装 TPU OS 镜像以及 TPU License），把上述运行 EEP-UP-VM 虚拟机的电脑与 ZYNQ 开发板接入同一个网络，打开电源，等待系统正确启动。

EEP-UP-VM 虚拟机环境下，在 `/home/eeplib/embeddeep/Tutorials/P3_caffe` 目录下点击鼠标右键打开终端程序。在终端中输入(注意，请根据开发板的实际 IP 地址更改下述的红色命令)：

```
ping 192.168.0.200 -c 5
```

若一切正常，会出现如下日志（若 `ping` 程序无法结束，可以在键盘上按下 `CTRL+C` 按键强制结束）。如果数据包无法正确传输，检查是否 IP 地址有冲突或被改变：

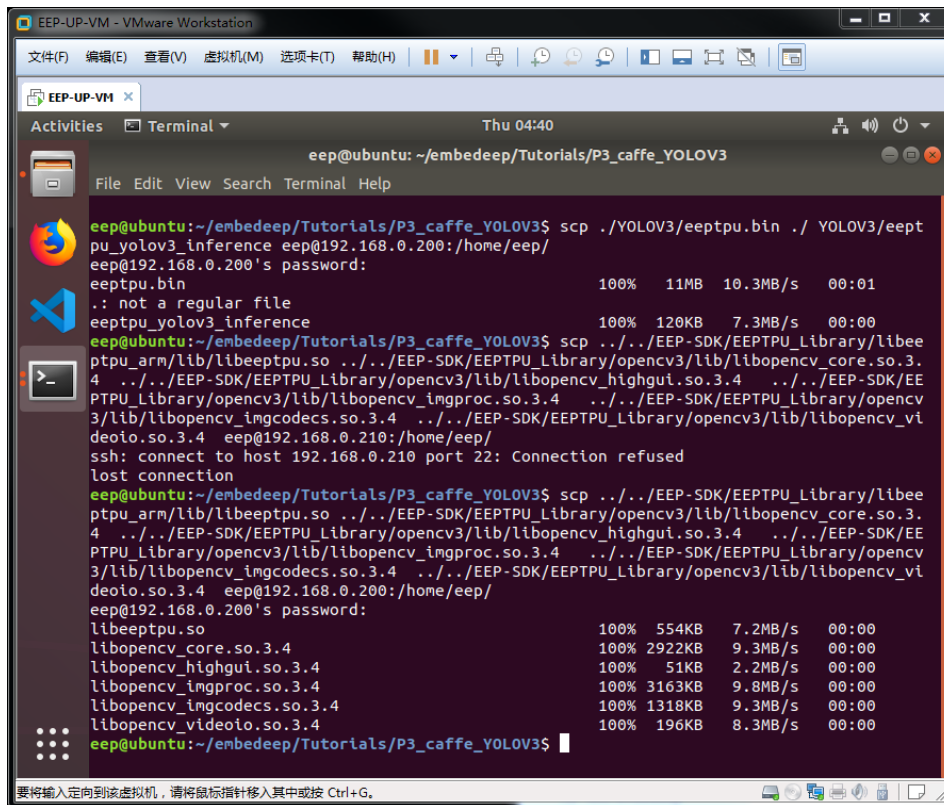


在终端中输入（输入 ZYNQ Linux 系统的密码 *embedeep*），请确保两个文件都有正确传输：

```
scp      ./YOLOV3/eeptpu.bin      ./YOLOV3/eeptpu_yolov3_inference
eep@192.168.0.200:/home/eep/
```

另外，从上节我们知道，*eeptpu_mobilenet_inference* 还调用了多个库文件，我们同时也需要把这些文件下载到目标系统中，输入如下命令：

```
scp      ../../EEP-SDK/EEPTPU_Library/arm32/eep/lib/libeeptpu.so      ../../EEP-
SDK/EEPTPU_Library/arm32/opencv3/lib/libopencv_core.so.3.4      ../../EEP-
SDK/EEPTPU_Library/arm32/opencv3/lib/libopencv_highgui.so.3.4      ../../EEP-
SDK/EEPTPU_Library/arm32/opencv3/lib/libopencv_imgproc.so.3.4      ../../EEP-
SDK/EEPTPU_Library/arm32/opencv3/lib/libopencv_imgcodecs.so.3.4      ../../EEP-
SDK/EEPTPU_Library/arm32/opencv3/lib/libopencv_videoio.so.3.4
eep@192.168.0.200:/home/eep/
```



为了测试算法是否正确，我们还需一张来自测试数据集的图片：

```
scp ./DataSet/JPEGImages/t00000100.jpg eep@192.168.0.200:/home/eep/
```

最后，我们通过 ssh 登陆到 ZYNQ 开发板，在任意终端中输入如下命令（输入 ZYNQ Linux 系统的密码 *embedeep*）：

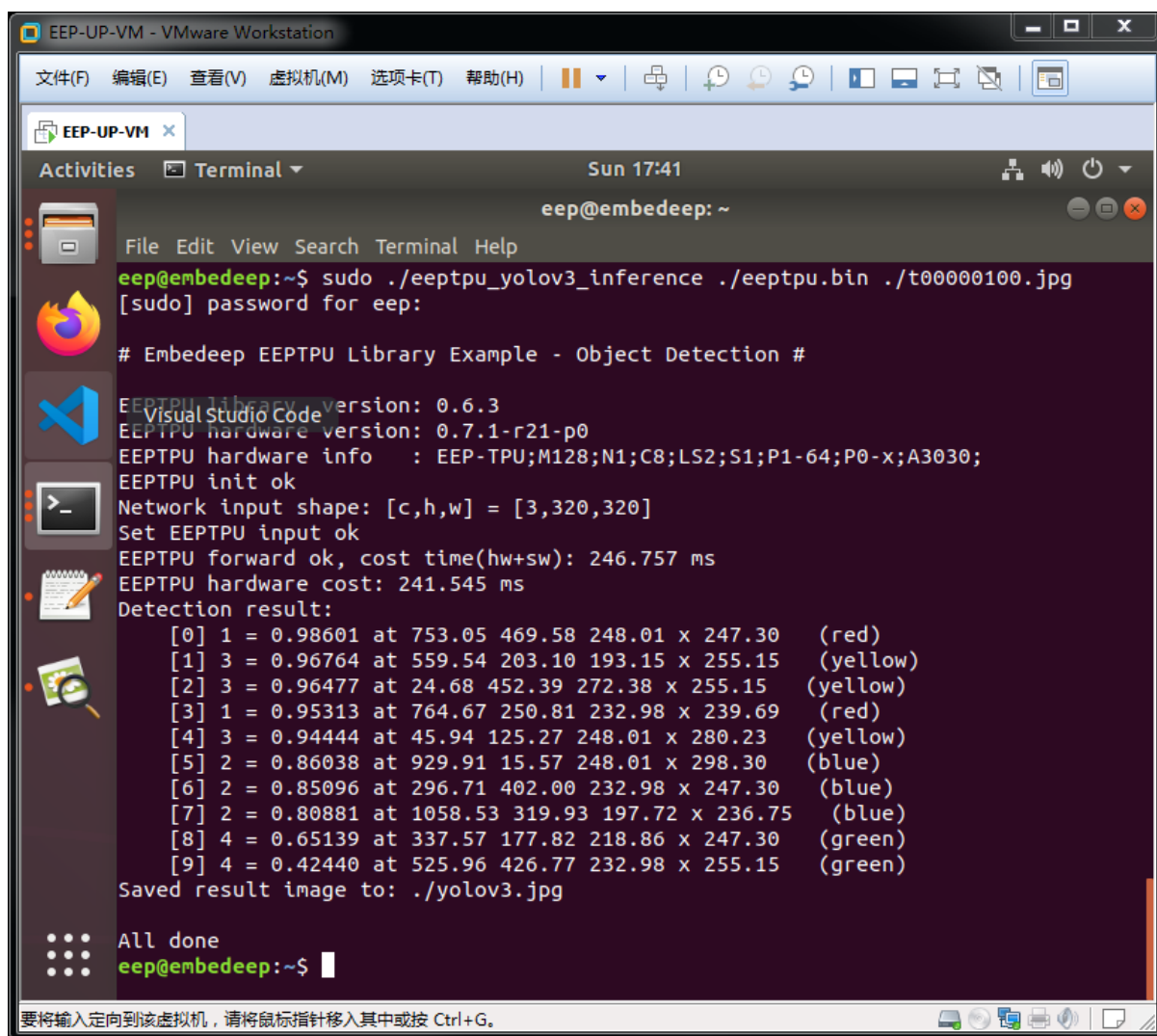
```
ssh 192.168.0.200
```

SSH 连接到 ZYNQ 开发板后，我们可以看到终端的计算名由 ubuntu 变成了 EEP-UP。从欢迎信息 “*Welcome to Ubuntu 18.04 LTS (GNU/Linux 4.14.0-xilinx-00001-g46173a7-dirty armv7l)*” 可以看到，CPU 也变成了 ARMV7 架构。终端中输入 “ls” 命令看到文件夹内的内容也发生了变化，刚才传输到 ZYNQ 开发板的所有文件都出现在了当前目录下，终端中输入 “ls” 命令看到文件夹路径变成了 “/home/eep”。此时，该终端已经运行与 ZYNQ 开发板。

在该终端中继续输入如下命令：

```
sudo ./eeptpu_yolov3_inference ./eeptpu.bin ./t00000100.jpg
```

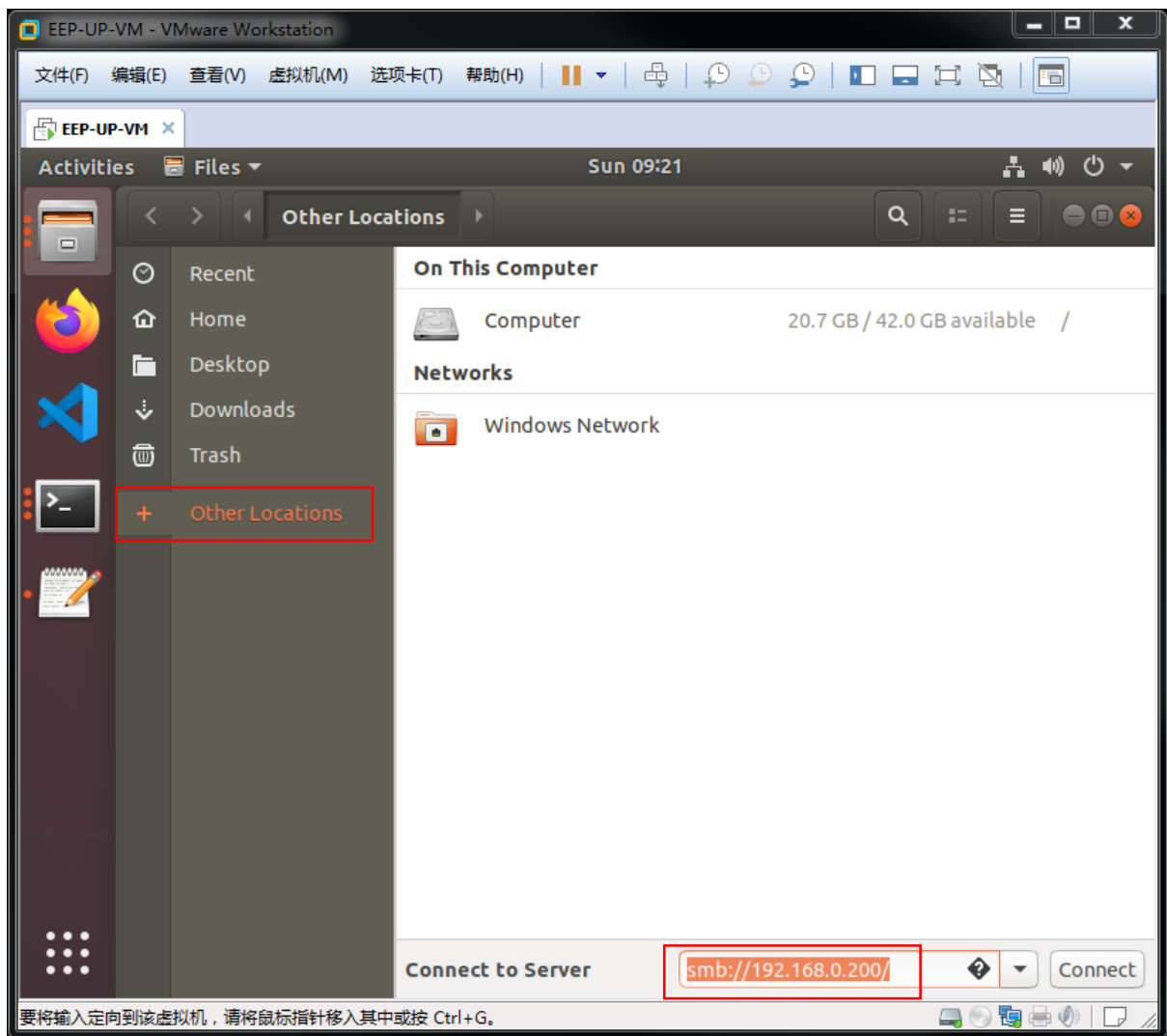
运行结果如下：



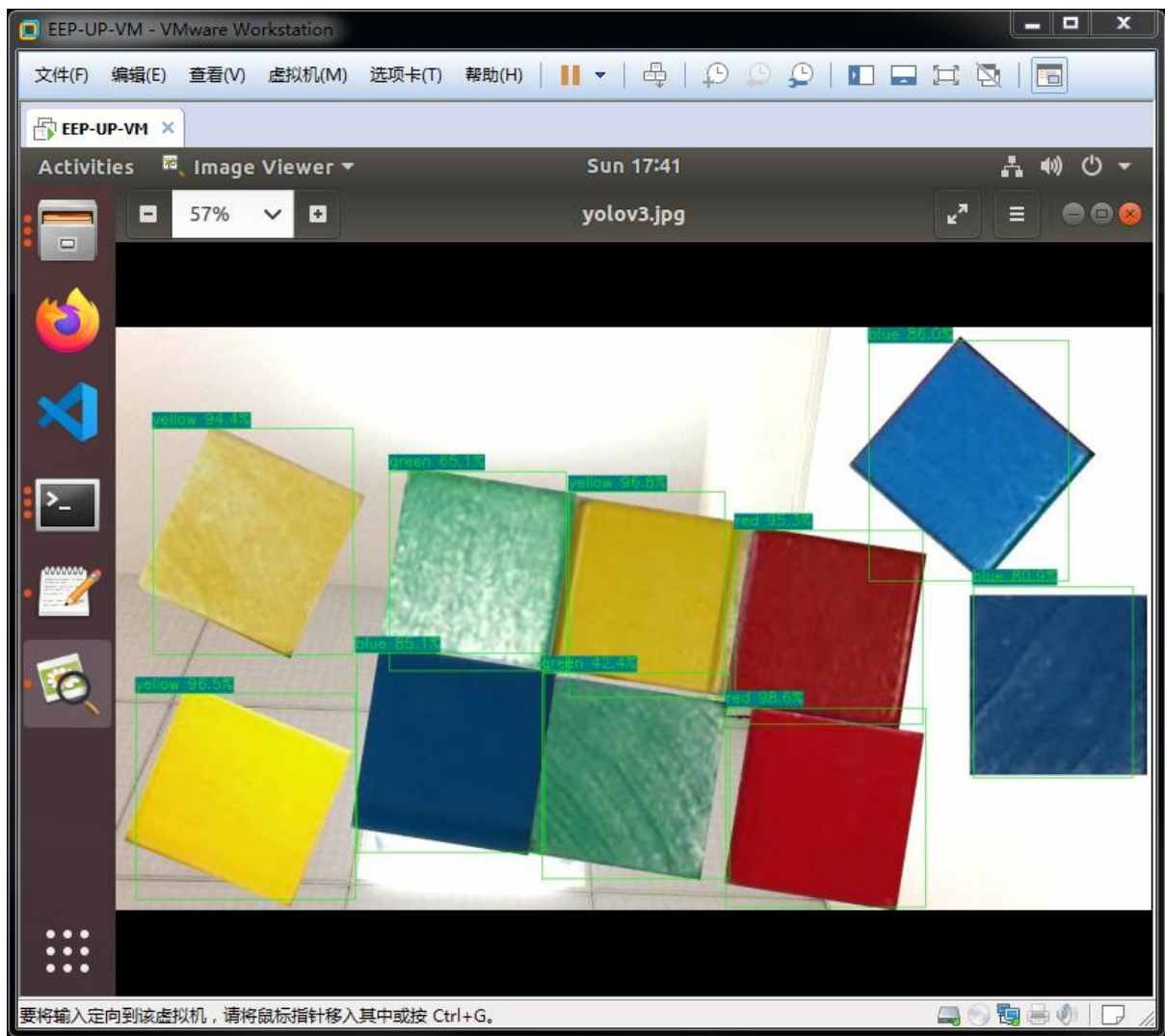
```
EEP-UP-VM - VMware Workstation
文件(F) 编辑(E) 查看(V) 虚拟机(M) 选项卡(T) 帮助(H)
EEP-UP-VM x
Activities Terminal Sun 17:41
eep@embedeep: ~
File Edit View Search Terminal Help
eep@embedeep:~$ sudo ./eeptpu_yolov3_inference ./eeptpu.bin ./t00000100.jpg
[sudo] password for eep:
# Embedeep EEPTPU Library Example - Object Detection #
EEPTPU library version: 0.6.3
EEPTPU hardware version: 0.7.1-r21-p0
EEPTPU hardware info : EEP-TPU;M128;N1;C8;LS2;S1;P1-64;P0-x;A3030;
EEPTPU init ok
Network input shape: [c,h,w] = [3,320,320]
Set EEPTPU input ok
EEPTPU forward ok, cost time(hw+sw): 246.757 ms
EEPTPU hardware cost: 241.545 ms
Detection result:
[0] 1 = 0.98601 at 753.05 469.58 248.01 x 247.30 (red)
[1] 3 = 0.96764 at 559.54 203.10 193.15 x 255.15 (yellow)
[2] 3 = 0.96477 at 24.68 452.39 272.38 x 255.15 (yellow)
[3] 1 = 0.95313 at 764.67 250.81 232.98 x 239.69 (red)
[4] 3 = 0.94444 at 45.94 125.27 248.01 x 280.23 (yellow)
[5] 2 = 0.86038 at 929.91 15.57 248.01 x 298.30 (blue)
[6] 2 = 0.85096 at 296.71 402.00 232.98 x 247.30 (blue)
[7] 2 = 0.80881 at 1058.53 319.93 197.72 x 236.75 (blue)
[8] 4 = 0.65139 at 337.57 177.82 218.86 x 247.30 (green)
[9] 4 = 0.42440 at 525.96 426.77 232.98 x 255.15 (green)
Saved result image to: ./yolov3.jpg
All done
eep@embedeep:~$
```

从日志可以清楚的看到，在本章图片中，总共有 10 个目标被检测出，其对应的置信度和类别都分别显示在每个条目中，对应的带标注信息的图存储在./yolov3.jpg 中。

我们可以通过 SAMBA 协议来方便的获取 yolov3.jpg 这张图片，打开文件夹，点击左侧栏的“**Other Locations**”，并在右下方的文本框内输入 smb://192.168.0.200，并单击回车键，如下图所示。



点击 eep 文件夹便可进入 FPGA 开发板的文件系统中（**第一次登录时需要输入密码，用户名为 eep，密码为 embedeep，登录框中的 Domain 保持默认值**），找到 yolov3.jpg，可以双击打开直接查看。



尽管各色块由于环境因素会出现较大的差别，但本实验所设计的算法都能正确的识别出。

IV. EEP-TPU 创新应用

4.1 应用开发介绍

人工智能算法作为人工智能应用的核心技术，起着关键性的作用。然而，在实际的人工智能应用开发过程中，人工智能算法却往往仅是整个业务流程的一部分，甚至有可能仅仅是很小一部分。

在实际的基于图像视觉的任务流程中，人工智能算法一般起着图像特征提取（分类）、目标坐标框回归（检测）、目标关键点定位（定位）、图像像素级分类（分割）几大作用。如何利用这些功能或者功能的组合来解决应用场景中的实际问题，是人工智能应用开发所关注的核心问题。

更进一步，在某个应用方案中，可能会组合使用多种不同类型的人工智能算法，并在人工智能算法中穿插一些传统算法，从而实现人工智能算法与传统算法融合使用来提供整体解决方案。

因此，对于人工智能应用开发，我们应该清楚的认识到的，人工智能算法提供的优秀性能会改变整体方案的流程和方式，但其肯定无法完全覆盖方案的所有方案，我们应该把人工智能算法当成一种工具，并且把核心回归到问题本身，而不是一味的希望采用人工算法实现所谓“端到端”解决方案。本章，我们通过两个实验，来学习如何利用人工智能算法解决实际问题。

4.2 基于 USB 摄像头的人脸检测实验

4.2.1 实验目的

1. 掌握人工智能应用开发方法；
2. 掌握 USB 摄像头读取方法；
3. 掌握人脸检测的基本原理；
4. 掌握 EEP-TPU 人工智能应用开发流程；
5. 掌握 EEP-TPU 人工智能应用部署流程；

4.2.2 实验内容

采用 YOLO-V3 经典网络，基于已训练好的人脸检测算法，完成人脸检测应用开发，并最终部署在 ZYNQ 开发板上，实现摄像头读取图像的人脸检测与显示。该实验的工程目录位于/home/eep/embedeep/Tutorials/P4_face/

4.2.3 实验步骤

4.2.3.1 实验环境搭建

与 3.2.3.1 所描述的步骤相同，请参考该章节。

实验环境已预先搭建完毕，打开虚拟机后，进入 /home/eep/embedeep/Tutorials/P4_face/目录，按照后续步骤可以完成本实验。

4.2.3.2 应用程序开发

我们直接使用已经开发好的人脸检测算法，具体算法训练流程可以参考上一章节，该算法以 eeptpu.bin 文件的形式呈现，在应用开发中，可以直接使用。

从第三章的学习我们知道，EEP-TPU 最基本的推理流程为：**初始化->图像输入->推理计算->结果输出**。

初始化对于绝大多数的应用程序来讲都是相同的，其主要作用是设置 EEP-TPU 的物理层接口（SoC 或 PCIE）以及读入程序所用到的所有深度学习算法 BIN 文件（有关多个 BIN 文件的读入，请参考附录 A. NNAPI 中第四节的内容）。

图像输入需要根据具体的应用来设计。对于图像视觉类的应用，输入数据以图片的形式存在，输入的形式多样，可以来自某个文件、来自 USB 摄像头、来自 DVP 摄像头、来自 MIPI 摄像头或来自网络等。开发人员需要根据具体系统的要求，开发合适的图像输入程序。

推理计算需要根据具体的业务流程来决定。在实际的应用中，组合使用多个不同的算法来完成业务流程的计算是比较常见的情况，这需要在图像读入后，多次调用推理计算来完成。

结果输出需要根据具体的应用来设计。结果输出方式可以有多种形态，可以是写入文本文件、写入数据库、通过显示屏显示、通过串口发送、通过网络发送、通过 LED 显示等等。

基本的推理计算流程可以参考第三章的内容，本节我们介绍如何使用 USB 摄像头采集图片，完成推理计算，并通过网络发送结果。

(1) EEP-TPU 的初始化

在使用 EEP-TPU 之前，我们必须完成初始化操作，需要注意的是，该初始化只需在程序生命周期内执行一次即可，用到的 NNAPI 函数分别是：

```
tpu->eeptpu_set_interface(interface_type)
```

```
tpu->eeptpu_load_bin(path_bin)
```

具体内容如下：

```
1. static int eeptpu_init(int interface_type, char* path_bin)
2. {
3.     int ret;
4.     if (tpu == NULL) tpu = tpu->init();
5.     ret = tpu->eeptpu_set_interface(interface_type);
6.     if (ret < 0) return ret;
7.     printf("EEPTPU library version: %s\n", tpu->eeptpu_get_lib_version());
8.     printf("EEPTPU hardware version: %s\n", tpu->eeptpu_get_tpu_version());
9.     printf("EEPTPU hardware info : %s\n", tpu->eeptpu_get_tpu_info());
```

```

10.  ret = tpu->eeptpu_load_bin(path_bin);
11.  if (ret < 0) {
12.      printf("Load bin fail, ret=%d\n", ret);
13.      return ret;
14.  }
15.      printf("EEPTPU init ok\n");
16.      return 0;
17. }

```

(2) USB 摄像头输入

当开发板插入 USB 摄像头时，会在 Linux 系统的/dev/文件夹内出现一个 videoX 文件（X 为数字，当系统只有一个摄像头时，X=0），该文件就代表了 USB 摄像头这个设备。

我们使用 OPENCV 来实现摄像头的打开、读取等操作。

打开摄像头（初始化），使用 opencv 的 `cv::VideoCapture` 实现

```

1.  static cv::VideoCapture cap;
2.  static void camera_init(int dev_id)
3.  {
4.      cap.open(dev_id); #open camera using opencv
5.  }

```

读取摄像头，使用 opencv 的 `cv::VideoCapture` 实现。需要注意的是，USB 摄像头所采集的图片通常是 MJPEG 格式，而 `cv::VideoCapture` 函数所获得的图片已经被转换成了 `cv::Mat` 格式，这其中还包含了 MJPEG 解码的过程。

```

1.  static int camera_read_to_cv_mat(int dc, int dh, int dw, cv::Mat& cv_img_orig,  

    cv::Mat& cv_img_resized)
2.  {
3.      cap.read(cv_img_orig); #read camera
4.      if (cv_img_orig.empty())
5.      {
6.          printf("opencv read video failed\n");
7.          return -1;
8.      }

```

```

9.   cv_img_resized = cv_img_orig;
10.  if ( (cv_img_resized.rows != dh) || (cv_img_resized.cols != dw) )
11.  {
12.      cv::resize(cv_img_resized, cv_img_resized, cv::Size(dw, dh));
13.  }
14.  return 0;
15. }

```

(3) 推理计算

推理计算的过程对于大多数程序是相同的

```

1.  ret = tpu->eeptpu_forward(result, eepPixelFormat_CHW);

```

(4) 结果输出

结果输出属于推理计算后处理阶段。在计算结果真正通过某种方式发送出去之前，通常需要某种方式的后处理。

在本实验中，推理计算得到的结果由 N 个条目组成，每个条目代表一个人脸坐标框计算结果，其中包含 6 个内容：类别、置信度、X 坐标、Y 坐标、宽、长。

这里需要特别注意的是，在深度学习检测算法中，计算所得的“X 坐标、Y 坐标、宽、长”并不是以绝对值的形式出现，而是以原始图片比例的形式。这样设计最大的好处是不必规定输入图像的尺寸，训练所得算法可以用于多种尺寸的图片。

正是由于这种“比例”的机制，在结果输出之前必须进行坐标计算后处理，计算过程如下：

```

1.  static int post_process_obj_detect(cv::Mat& cimg, struct EEPTPU_RESULT&
    result, std::vector<Object>& objects)
2.  {
3.      objects.clear();
4.      if ( (result.c * result.h * result.w == 0) || (result.c != 1) )
5.      {
6.          //printf("Nothing detected !\n");
7.          return 0;

```

```

8.     }
9.     for (int i = 0; i < result.h; i++)
10.    {
11.        const float *values = result.data + i * result.w;
12.        Object object;
13.        object.label = values[0]; # (1) 类别
14.        object.prob = values[1]; # (2) 置信度
15.        object.rect.x = values[2] * cvimg.cols; # (3) X坐标
16.        object.rect.y = values[3] * cvimg.rows; # (4) Y坐标
17.        object.rect.width = values[4] * cvimg.cols - object.rect.x; # (5) 宽
18.        object.rect.height = values[5] * cvimg.rows - object.rect.y; # (6) 长
19.        strcpy(object.obj_class_name, class_names[object.label]);
20.        if (object.rect.x < 0) { object.rect.width += object.rect.x; object.rect.x = 0; }
21.        if (object.rect.y < 0) { object.rect.height += object.rect.y; object.rect.y = 0; }
22.        objects.push_back(object); #所有计算结果的集合
23.    }
24.    return 0;
25. }

```

在完成所有计算，获得所有检测人脸的坐标数据后（保存在 `std::vector<Object> objects` 中），我们需要把坐标结果通过网络发送出去。在这里，我们把获得的 `objects` 数据与原始图片 `cvimg_orig` 一起打包发出。

我们使用 UDP 协议来发送数据。UDP 是 User Datagram Protocol 的简称，中文名是用户数据报协议，是 OSI（Open System Inter connect 开放式系统互联）参考模型中一种无连接的传输层协议，提供面向事务的简单不可靠信息传送服务。UDP 协议的优点是快（UDP 没有 TCP 的握手，确认，窗口，重传，拥塞控制等机制，UDP 是一个无状态的传输协议，所以它在传输数据是非常快）。UDP 协议的缺点是不可靠、不稳定，容易发生丢包。在本教程的实验环境中，UDP 协议符合要求。

在使用 UDP 协议通过网络发送结果数据之前，我们需要进行网络初始化操作，如下，其中，`p_socket_addr` 和 `socket_port` 是来自外部的可配置参数：

```

1.  socket_set_addr_port(string(p_socket_addr), socket_port);
2.  socket_udp_test();

```

需要特别注意的是，在图像输入阶段，我们使用 `cv::VideoCapture` 函数读入图片并转成 `cv::Mat` 类型。在最后通过网络发送时，需要把 `cv::Mat` 图像再压缩成 jpeg 格式进行发送（降低数据大小）。图片压缩的代码如下：

```
3.  int socket_send_obj_detect(std::vector<struct Object>& objects, cv::Mat&
    cvimg)
4.  {
5.      if (is_socket_on() == false)
6.      {
7.          return 0;
8.      }
9.      if (cvimg.empty() == true) return -1;
10.     string ext_info = "Face Detection";
11.     std::vector<unsigned char> data_encode;
12.     unsigned char* encoded_data = NULL;
13.     unsigned long encoded_data_len = 0;
14.     int quality = 85; #设定 jpeg 压缩的质量
15.     std::vector<int> params; /*IMWRITE_JPEG_QUALITY For JPEG, it can be a
        quality from 0 to 100 (the higher is the better). Default value is 95 */
16.     params.push_back(CV_IMWRITE_JPEG_QUALITY);
17.     params.push_back(quality);
18.     cv::imencode(".jpg", cvimg, data_encode, params);
19.     params.clear();
20.     encoded_data = (unsigned char*)&data_encode[0]; #新的压缩 jpeg 数据
21.     encoded_data_len = data_encode.size()*sizeof(unsigned char);
```

最后，通过一个自定义协议，使用 UDP 发送出去，在接收端（PC），一个程序被用于接收这些数据并解析，最终显示。在这里，具体的 UDP 发送过程不在本教程的范围内，用户可以自行查找相关的书籍和教程。本教程所附的代码及其配套程序可以被当做一个现成的组件使用。该网络传输组件的作用，是把原始图片以及对应的来自 AI 算法计算所得的 `std::vector<struct Object>& objects` 类型的数据进行打包，并发送给对应的（PC）接收端用于显示。这段基于 UDP 的网络发送程序具备一定的通用性，可以被其他的目标检测应用直接使用。

UDP 发送的代码如下：


```

22.  int objlist_size = 0;
23.  for (unsigned int i = 0; i < objects.size(); i++) // calc the size of objects list.
24.  {
25.      objlist_size += ( 8 + sizeof(struct Object) ); // 8: class_name_len + color
26.  }
27.  unsigned char* retbuf;
28.  retbuf = (unsigned char*)malloc(4 + objlist_size); // obj list len + list content
29.  if (retbuf == NULL) return -1;
30.  unsigned char* ptrbuf = retbuf;
31.  uint32_to_buf(ptrbuf, objects.size()); ptrbuf += 4;
32.  for (unsigned int i = 0; i < objects.size(); i++)
33.  {
34.      uint32_to_buf(ptrbuf, objects[i].label); ptrbuf += 4;
35.      uint32_to_buf(ptrbuf, int(objects[i].prob*1000)); ptrbuf += 4;
36.      uint32_to_buf(ptrbuf, int(objects[i].rect.x*1000)); ptrbuf += 4;
37.      uint32_to_buf(ptrbuf, int(objects[i].rect.y*1000)); ptrbuf += 4;
38.      uint32_to_buf(ptrbuf, int(objects[i].rect.width*1000)); ptrbuf += 4;
39.      uint32_to_buf(ptrbuf, int(objects[i].rect.height*1000)); ptrbuf += 4;
40.      uint32_to_buf(ptrbuf, strlen(objects[i].obj_class_name)); ptrbuf += 4;
41.      strcpy((char*)ptrbuf, objects[i].obj_class_name); ptrbuf +=
        strlen(objects[i].obj_class_name);
42.      *ptrbuf++ = 1;
43.      *ptrbuf++ = 0; // r
44.      *ptrbuf++ = 255; // g
45.      *ptrbuf++ = 0; // b
46.  }
47.
48.  int ret = socket_udp_send(CCMD_OBJDET, encoded_data,
        encoded_data_len,
49.      retbuf, ptrbuf-retbuf, forward_ms*1000, (char*)ext_info.c_str(),
50.      NULL, 0
51.      );
52.  if (retbuf != NULL) free(retbuf);
53.  data_encode.clear();
54.  return ret;
55. }

```

4.2.3.3 编译、下载及测试

应用程序开发完成后，需要经过编译、下载及运行过程，才能最终实现应用程序的测试。请用户自行根据第三章的内容完成编译、下载及测试过程。所有的参考代码和脚本位于/home/eep/embedeep/Tutorials/P4_face/reference_design 文件夹内，需要注意的是，应用程序的 main 函数有 4 个参数：bin 文件、图片、PC 机 IP 地址（用于接收数据并显示）、PC 机端口号。

如第三章所述，通过 ssh 命令进入 FPGA 开发板，启动命令示例如下：

```
sudo ./eeptpu_face_inference ./eeptpu_face.bin /dev/video0 "192.168.0.111" 60003
```

其中，蓝色部分是端口号，必须与 PC 端的显示程序的端口号一致，默认为 60003，如无特殊则不需要修改。红色的 IP 地址是 PC 端的地址，如果网络环境允许广播，则可以把该地址去掉，这样本网段下的所有 PC 都可以接收到数据，如下：

```
sudo ./eeptpu_face_inference ./eeptpu_face.bin /dev/video0 "" 60003
```

在 PC 端，双击显示程序 eep_panel64.exe（暂时仅提供 64 位 Windows7 以上系统的程序），可以看到 FPGA 计算结果

4.3 基于 USB 摄像头的客流统计实验

4.3.1 实验目的

1. 掌握人工智能应用开发方法；
2. 掌握多线程人工智能应用的开发方法；
3. 掌握 EEP-TPU 人工智能应用开发流程；
4. 掌握 EEP-TPU 人工智能应用部署流程；

4.3.2 实验内容

采用 YOLO-V3 经典网络，基于已训练好的人脸检测算法，完成人脸检测应用开发，并最终部署在 ZYNQ 开发板上，实现摄像头读取图像的人脸检测与显示。该实验的工程目录位于/home/eep/embedeep/Tutorials/P4_ppcount/

4.3.3 实验步骤

4.3.3.1 实验环境搭建

与 3.2.3.1 所描述的步骤相同，请参考该章节。

实验环境已预先搭建完毕，打开虚拟机后，进入 /home/eep/embedeep/Tutorials/P4_face/目录，按照后续步骤可以完成本实验。

4.3.3.2 应用程序开发

在上一节，我们使用**人脸检测算法**来实现人脸检测应用，在本节，我们将使用**人头检测算法**来实现客流统计应用。我们直接使用已经开发好的人头检测算法，具体算法训练流程可以参考上一章节，该算法以 eeptpu.bin 文件的形式呈现，在应用开发中，可以直接使用。

回顾上一章的内容，我们知道 EEP-TPU 最基本的推理流程为：**初始化->图像输入->推理计算->结果输出**。本章的客流统计应用与人脸检测的推理流程基本相同，不同的是，本章我们将使用多线程并行的方式来实现客流统计应用，也即把上一节串行的图像输入、推理计算、结果输出在本节中使用多线程的方式来实现。

(1) EEP-TPU 的初始化

尽管使用多线程的方式来实现，但其初始化流程与单线程的人脸检测是完全相同的，具体参考上一节，这里不再复述。

(2) 多线程并行

多线程并行是为了进一步提升计算速度而采用的应用开发方法。我们简单的把人工智能应用划分为三大部分：预处理、推理、后处理。在单线程程序中，每次任务需要顺序的完成三部分计算，任务时长由三部分计算的时间的总和决定。在多线程程序中，三部分计算并行完成，并以流水线的方式处理：在同一时间内，任务 n、任务 n+1 和任务 n+2 同时在计算流水线中，单个任务的时长由预处理、推理和后处理中最慢的那部分计算决定。

按照上述划分的多线程并行程程序的代码如下：

```
1.  int run_threads()
2.  {
3.      pthread_t pid_forward;
4.      pthread_t pid_post_process;
5.      pthread_t pid_pre_process;
6.
7.      pthread_create(&pid_forward, NULL, thread_forward, NULL);
8.      pthread_create(&pid_post_process, NULL, thread_post_process, NULL);
9.      pthread_create(&pid_pre_process, NULL, thread_pre_process, NULL);
10.
11.     int *ret_pre, *ret_forward, *ret_post;
12.     pthread_join ( pid_forward, (void**)&ret_forward );
13.     pthread_join ( pid_post_process, (void**)&ret_post );
14.     pthread_join ( pid_pre_process, (void**)&ret_pre );
15.
```

```

16.  if (*ret_forward < 0) return *ret_forward;
17.  if (*ret_post < 0) return *ret_post;
18.  if (*ret_pre < 0) return *ret_pre;
19.  return 0;
20. }

```

(3) thread_pre_process 预处理线程

在本节所实现的客流统计应用中，预处理线程的主要作用是图像的获取，在这之前，我们需要定义一个全局的 FIFO，用于存储原始图像数据、resize 图像数据、同步标志位、推理计算结果、推理时间等信息：

```

(1) #define STAGE_EMPTY                0
(2) #define STAGE_WAIT_FORWARD        1
(3) #define STAGE_WAIT_POST_PROCESS  2
(4) struct st_fifo_data
(5) {
(6)     volatile int stage;
(7)     cv::Mat cv_in_orig;
(8)     cv::Mat cv_in_resized;
(9)     bool b_sync;
(10)     int result_idx;
(11)     vector<struct EEPTPU_RESULT> blobs_out;
(12)     unsigned long forward_us_hws;
(13)
(14)     st_fifo_data():
(15)         stage(STAGE_EMPTY),
(16)         b_sync(false),
(17)         result_idx(0)
(18)     {}
(19) };
(20) #define FIFO_DEPTH    10
(21) struct st_fifo_data fifo_mats[FIFO_DEPTH];

```

预处理线程的主要作用是从 USB 摄像头中读取图像数据，并（根据写指针 `write_index`）向全局 FIFO 中写入该图像数据，最后向 EEP-TPU 的缓存中写入计算数据。其中，图像获取的代码与上一节的人脸检测应用的 USB 摄像头获取完全相同，使用 OPENCV 的 `cv::VideoCapture` 函数来实现。

特别需要注意，与单线程串行执行不同，由于多线程任务之间的执行顺序不确定，且各自的执行时长也不确定，因此需要一定的同步机制来实现不同线程之间的协同操作。在这里，我们使用 `tpu->eeptpu_wait_input_writable(delay)` 函数来完成 EEP-TPU 推理的输入同步。

EEP-TPU 处理器拥有自己专享的缓存，计算发生前，需要向缓存中写入待计算的数据。而在写入之前则需要等待该输入缓存变为可写入状态。如果输入缓冲区的数据还未被使用，则不允许写入数据的发生，否则会导致输入数据错乱。对于“写输入数据”与“推理”为异步执行的情况，因为输入数据写入后，难以简单地判断当前输入数据是否进行了推理操作，为了防止该输入数据还未进行推理却被新数据覆盖的问题，需要在输入数据前执行 `tpu->eeptpu_wait_input_writable(delay)` 函数，等待输入可写。而对于“写输入数据”与“推理”为同步串行执行的情况（如上节的人脸检测），则无需调用本函数进行等待。

最后，写入 EEP-TPU 缓存的操作完成后，需要把全局 FIFO 中写指针（`write_index`）所对应条目的状态由原来的 `STAGE_EMPTY`，更改为 `STAGE_WAIT_FORWARD`，从而指定该条目的内容已完成数据准备操作，可以执行推理计算。

该预处理线程的代码如下：

```
1. void* thread_pre_process(void* argv)
2. {
3.     static int ret = 0;
4.
5.     while(1)
6.     {
7.         if (fg_exit) { ret = -1; break; }
8.         if (fifo_mats[write_index].stage == STAGE_EMPTY)
9.         {
10.            if (fifo_mats[write_index].cv_in_orig.empty() == false)
                fifo_mats[write_index].cv_in_orig.release();
```

```

11.         if (fifo_mats[write_index].cv_in_resized.empty() == false)
            fifo_mats[write_index].cv_in_resized.release();
12.         ret = camera_read_to_cv_mat(tpu->in_c, tpu->in_h, tpu->in_w,
            fifo_mats[write_index].cv_in_orig, fifo_mats[write_index].cv_in_resized);
13.         if (ret < 0) {
14.             printf("Read video fail\n");
15.             break;
16.         }
17.         ret = tpu->eeptpu_wait_input_writable(2000); // 2s
18.         ret = tpu->eeptpu_set_input(fifo_mats[write_index].cv_in_resized.data,
19.                                     fifo_mats[write_index].cv_in_resized.channels(),
20.                                     fifo_mats[write_index].cv_in_resized.rows,
21.                                     fifo_mats[write_index].cv_in_resized.cols);
22.         if (ret < 0) {
23.             printf("Set EEPTPU input fail\n");
24.             break;
25.         }
26.
27.         fifo_mats[write_index].stage = STAGE_WAIT_FORWARD;
28.         //printf("fifo[%d] set to STAGE_WAIT_FORWARD\n", write_index);
29.
30.         write_index++;
31.         if (write_index >= FIFO_DEPTH) write_index = 0;
32.     }
33.     else
34.     {
35.         wait_cond_us(500);
36.     }
37. }
38. return ((void*)&ret);
39. }

```


(4) thread_forward 推理线程

推理线程的主要作用是启动 EEP-TPU 处理器执行深度学习推理操作。该线程执行时，需要预处理线程完成数据准备工作才能执行推理操作，而这个同步等待，是通过全局 FIFO 中读指针（read_index）所对应条目的状态信息的判断来实现的。

在上述的预处理线程中，在任务的最终阶段，会把全局 FIFO 中写指针（write_index）所对应条目的状态由原来的 STAGE_EMPTY，更改为 STAGE_WAIT_FORWARD。而在推理线程，则需要判断这个状态为 STAGE_WAIT_FORWARD 时，才会执行真正的推理计算任务，否则进行等待。

需要特别注意的是，除了输入准备好，输出缓存的数据也需要被取出，才能正确执行推理操作。由于是异步推理方式。若输出缓冲区的数据还未被取出，而又进行推理的话，输出缓冲区的推理数据会被新的数据所覆盖，可能导致应用程序读取的推理结果错乱。在进行推理前，需要使用 eeptpu_wait_result_fetched（delay）函数等待一段时间，以便应用程序及时读出数据。

推理计算完成后，如果计算返回的数据有效（内容 >0），则设置相应的状态为 STAGE_WAIT_POST_PROCESS，如果返回的数据无效（内容 =0），则设置相应的状态为 STAGE_EMPTY，代表该条目处理完毕，可以被覆盖。

该推理线程的代码如下：

```
1. void* thread_forward(void* argv)
2. {
3.     static int ret = 0;
4.     while(1)
5.     {
6.         if (fg_exit) { ret = -1; break; }
7.         if (fifo_mats[read_index].stage == STAGE_WAIT_FORWARD)
8.         {
9.             double st,et;
10.            st = get_current_time();
11.
12.            fifo_mats[read_index].b_sync = false;
13.            //fifo_mats[read_index].b_sync = true;
14.
15.            if (fifo_mats[read_index].b_sync == true)
```

```

16.     {
17.         // Synchronous mode
18.         clear_results(fifo_mats[read_index].blobs_out);
19.         ret = tpu->eeptpu_forward(fifo_mats[read_index].blobs_out,
eepPixelType_CHW);
20.     }
21.     else
22.     {
23.         // Asynchronous mode
24.         tpu->eeptpu_wait_result_fetched(2000);
25.         ret =
tpu->eeptpu_forward_async(&fifo_mats[read_index].result_idx);
26.         // we will call eeptpu_forward_post() in post_process thread.
27.     }
28.     if (ret < 0)
29.     {
30.         fifo_mats[read_index].stage = STAGE_EMPTY;
31.         printf("eeptpu forward fail, ret = %d\n", ret);
32.         break;
33.     }
34.     else
35.     {
36.         et = get_current_time();
37.         forward_ms = et-st;
38.         fifo_mats[read_index].stage = STAGE_WAIT_POST_PROCESS;
39.     }
40.     read_index++;
41.     if (read_index >= FIFO_DEPTH) read_index = 0;
42. }
43. else
44. {
45.     wait_cond_us(500);
46. }
47. }
48. return ((void*)&ret);
49. }

```

(5) thread_post_process 后处理线程

在本实验所实现的客流统计应用中，EEP-TPU 使用深度学习算法实现人头检测功能。与上一节的人脸检测类似，人头检测算法将得到人头的坐标信息，其中的坐标信息后处理步骤（post_process_obj_detect 函数）完全相同，具体可以参考人脸检测一节。

另外，后处理线程需要与推理线程进行同步，也即推理线程完成计算并更新状态为 STAGE_WAIT_POST_PROCESS，后处理线在每次执行时需要判断 post_index 指针对应的全局 FIFO 中的条目的状态是否为 STAGE_WAIT_POST_PROCESS，若是则执行正常的计算流程，若不是则进行等待。

在获取人头坐标后，我们还无法实现客流统计功能。因为到这里为止，我们仅仅能知道类别为人头的目标及其坐标，而无法知道连续两帧所检测出的目标之间的关联。要实现客流统计的功能，我们必须要为每一个视频中所出现的不同的人赋予独特的 ID，并跟踪独立个体之间的运动轨迹（进或出），从而统计进、出某个电子边界的数量。

在本实验室中，我们设计了一种跟踪算法，该算法基于坐标进行聚类，输入类别为“Head”的坐标框集合，输出类别为“ID 号”的坐标集合，并且该 ID 号具备一定的时间属性，也即连续的图像，对于相同位置属性为目标赋予相同的 ID 号，从而实现跟踪功能。有关跟踪算法具体内容不在本教程范围内，可以直接使用参考设计中的代码，并参考相应的书籍和教程来理解其具体含义。

最终，与人脸检测相同，我们通过 socket_send_obj_detect 函数，把图像和相应的“跟踪修正”的坐标集合，通过网络发送出去。

该后处理线程的代码如下：

```
1. void* thread_post_process(void* argv)
2. {
3.     static int ret = 0;
4.     while(1)
5.     {
6.         if (fg_exit) { ret = -1; break; }
7.         if (fifo_mats[post_index].stage == STAGE_WAIT_POST_PROCESS)
8.         {
9.             if (fifo_mats[post_index].b_sync == false)
10.            {
```

```

11.         clear_results(fifo_mats[post_index].blobs_out);
12.         ret = tpu->eeptpu_forward_post(fifo_mats[post_index].blobs_out,
        fifo_mats[post_index].result_idx, eepPixelFormat_CHW);
13.         if (ret < 0) break;
14.     }
15.
16.     //printf("fifo[%d] post processing\n", post_index);
17.     std::vector<Object> objects;
18.     ret = post_process_obj_detect(fifo_mats[post_index].cv_in_orig,
        fifo_mats[post_index].blobs_out[0], objects);
19.     if (ret < 0) break;
20.
21.     ret = tracker_process(objects, fifo_mats[post_index].cv_in_orig);
22.     if (ret < 0) break;
23.
24.     socket_send_obj_detect(objects, fifo_mats[post_index].cv_in_orig);
25.
26.     // draw object
27.     //cv::Mat final_image = draw_objects(cvimg_orig, objects);
28.
29.     //cv::imwrite("./objdet.jpg", final_image);
30.     //printf("Saved result image to: ./objdet.jpg\n");
31.
32.     // show final image.
33.     //cv::imshow("EEPTPU Object Detection Example", final_image); // can
        not show in terminal command line.
34.     //cv::waitKey(0);
35.
36.     //if (final_image.empty() == false) final_image.release();
37.
38.     objects.clear();
39.
40.     fifo_mats[post_index].stage = STAGE_EMPTY;
41.
42.     post_index ++;
43.     if (post_index >= FIFO_DEPTH) post_index = 0;

```

```

44.
45.     fps_update();
46. }
47. else
48. {
49.     wait_cond_us(500);
50. }
51. }
52.
53. return ((void*)&ret);
54. }

```

4.3.3.3 编译、下载及测试

应用程序开发完成后，需要经过编译、下载及运行过程，才能最终实现应用程序的测试。请用户自行根据第三章的内容完成编译、下载及测试过程。所有的参考代码和脚本位于/home/eep/embedeep/Tutorials/P4_ppcount/reference_design 文件夹内，需要注意的是，应用程序的 main 函数有 4 个参数：bin 文件、图片、PC 机 IP 地址（用于接收数据并显示）、PC 机端口号。

如第三章所述，通过 ssh 命令进入 FPGA 开发板，启动命令示例如下：

```

sudo ./eeptpu_ppcount_inference ./eeptpu_pp.bin /dev/video0 "192.168.0.1111"
60003

```

其中，蓝色部分是端口号，必须与 PC 端的显示程序的端口号一致，默认为 60003，如无特殊则不需要修改。红色的 IP 地址是 PC 端的地址，如果网络环境允许广播，则可以把该地址去掉，这样本网段下的所有 PC 都可以接收到数据，如下：

```

sudo ./eeptpu_ppcount_inference ./eeptpu_pp.bin /dev/video0 "" 60003

```

在 PC 端，双击显示程序 eep_panel64.exe（暂时仅提供 64 位 Windows7 以上系统的程序），可以看到 FPGA 计算结果

4.4 基于 USB 摄像头的 HDMI 显示

4.4.1 实验目的

1. 掌握人工智能应用开发方法；
2. 掌握 USB 摄像头读取方法；
3. 掌握通用目标检测的基本原理；
4. 掌握 HDMI 显示开发方法；
4. 掌握 EEP-TPU 人工智能应用开发流程；
5. 掌握 EEP-TPU 人工智能应用部署流程；

4.4.2 实验内容

采用 YOLO-V3 经典的 20 类目标检测网络，基于已训练好的算法模型，完成通用目标检测应用开发，实现 HDMI 显示输出，并最终部署在 ZYNQ 开发板上。该实验的工程目录位于/home/eep/embedeep/Tutorials/P4_HDMI/

4.4.3 实验步骤

4.4.3.1 实验环境搭建

与 3.2.3.1 所描述的步骤相同，请参考该章节。

实验环境已预先搭建完毕，打开虚拟机后，进入 /home/eep/embedeep/Tutorials/P4_HDMI/ 目录，按照后续步骤可以完成本实验。

4.4.3.2 应用程序开发

我们直接使用已经开发好的 20 类通用目标检测算法，该算法可识别热 20 种类别为："aeroplane", "bicycle", "bird", "boat", "bottle", "bus", "car", "cat", "chair", "cow", "diningtable", "dog", "horse", "motorbike", "person", "pottedplant", "sheep", "sofa", "train", "tvmonitor"，具体算法训练流程可以参考前述章节，该算法以 eeptpu.bin 文件的形式呈现，在应用开发中，可以直接使用。

回顾上一章的内容，我们知道 EEP-TPU 最基本的推理流程为：**初始化->图像输入->推理计算->结果输出**。在本章的前面两小节中，我们使用网络的方式实现结果输出，这是 AI 端末计算的一种输出方式。本章，我们将尝试另一种方式：**本地显示**，也即通过端末 AI 装置所连接的本地显示设备进行结果的展示。在本实验中，我们通过 FPGA 开发板的 HDMI 端口外接一台显示器，并通过该显示器实现结果显示。

1. EEP-TPU 的初始化

本实验采用单线程的方式实现，其初始化流程与人脸检测实验完全相同，具体参考相应章节，这里不再复述。

2. USB 摄像头输入

本实验使用 OPENCV 来实现摄像头的打开、读取等操作，其具体方法和人脸检测实验完全相同，具体可参考相应章节，这里不再复述。

3. 推理计算

推理计算的过程对于大多数程序是相同的

```
(1) ret = tpu->eeptpu_forward(result, eepPixelFormat_CHW);
```

4. 结果输出

本实验使用本地输出的方式，通过 HDMI 显示器实现结果的显示。在 HDMI 显示时，应用程序必须运行在 AI 终端的嵌入式 Linux 系统中。在这样的环境下，通过 OPENCV 的 cv::imshow 函数即可实现计算结果的显示：

```
1. cv::imshow("EEPTPU Object Detection Example", final_image);
```


这里需要注意的是，使用 `imshow` 的应用程序，必须是在桌面环境下运行才会有效果，如果采用前两节网络终端方式连接，那么该 `imshow` 显示函数将不会起作用。

4.4.3.3 编译、下载及测试

应用程序开发完成后，需要经过编译、下载及运行过程，才能最终实现应用程序的测试。

在前面的实验章节中，所有应用程序都是在 PC 的 Linux 环境下编译、然后把编译所得的二进制文件下载到 FPGA，最后通过远程连接 FPGA，并在终端环境下执行相应程序，这种开发方式属于**交叉编译**。

在本节，我们将采用另一种开发方式：在嵌入式设备上本地编译运行。本地编译的核心是编译开发环境和最终运行环境都在同一个体系下。在本实验中，编译和运行都在基于 ARM 的嵌入式环境中完成。

所有的参考代码和脚本位于 `/home/eep/embedeep/Tutorials/P4_HDMI/reference_design` 文件夹内，请将该文件夹内的代码和脚本下载到设备上的任意文件夹内（请根据前述的方法完成），终端运行命令

32 位系统下

```
bash arm-cpu-compile32.sh
```

64 位系统下

```
bash arm-cpu-compile64.sh
```

编译完成后，在相应目录打开一个终端，输入如下命令开启程序：

```
sudo ./eeptpu_face_inference ./eeptpu_yolo.bin /dev/video0
```

如果一切正常，则可在开发板所连接的显示器上，看到对应的窗口显示

附录 A. NNAPI

1. 编译

32 位交叉编译工具链: arm-linux-gnueabi-g++, gcc 版本 6.3.1 20170404 (Linaro GCC 6.3-2017.05)。交叉编译工具链下载地址:

http://releases.linaro.org/components/toolchain/binaries/6.3-2017.05/arm-linux-gnueabi-gcc-linaro-6.3.1-2017.05-i686_arm-linux-gnueabi.tar.xz

64 位交叉编译工具链: aarch64-linux-gnu-g++, gcc 版本 6.3.1 20170404 (Linaro GCC 6.3-2017.05)。交叉编译工具链下载地址:

http://releases.linaro.org/components/toolchain/binaries/6.3-2017.05/aarch64-linux-gnu-gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu.tar.xz

程序使用的交叉编译工具链需与上述版本一致, 否则可能会出现一些兼容性问题。

库文件 libeeptpu.so v0.6.2 版本, 需配套使用 eeptpu_compiler v0.6.4 及以上、EEP-TPU 硬件版本 v0.7.1 及以上。桌面版 Ubuntu 18.04 LTS。

2. API 接口

2.1 初始化

初始化函数在程序中只需调用一次

2.1.1 初始化库文件调用接口

函数	EEPTPU* init();
功能	初始化库文件调用接口。在使用所有类成员函数前需先调用本始化函数。
参数	无
返回	返回 EEPDETECT 类指针
示例	EEPTPU *tpu = NULL; // 可声明为全局变量

	if (tpu == NULL) tpu = tpu->init(); // 可放置于 main 函数开头进行初始化
--	--

2.1.2 设置接口类型

函数	int eeptpu_set_interface(int interface_type);
功能	设置数据交互接口类型。 在 init()函数执行后，加载 EEPTPU BIN 文件之前，需调用本函数来设置接口类型。
参数	interface_type: 接口类型 可用接口类型有 eeplInterfaceType_SOC 和 eeplInterfaceType_PCIE。 根据开发板实际情况进行选择。若使用 SOC 形态设计，则设置为 eeplInterfaceType_SOC；若使用 PCIE 插槽，程序在主机端通过 PCIE 与 EEP-TPU 进行交互，则设置为 eeplInterfaceType_PCIE。 enum { eeplInterfaceType_NONE = 0, eeplInterfaceType_SOC = 1, eeplInterfaceType_PCIE, eeplInterfaceType_EOF, /* dummy data. */ };
返回	0: 成功 < 0: 失败。(错误码参考附件 1)
示例	tpu->eeptpu_set_interface(eeplInterfaceType_SOC);

2.1.3 设置 PCIE 设备名

函数	int eeptpu_set_interface_info_pcie(const char* dev_reg, const char* dev_h2c, const char* dev_c2h);
功能	设置 PCIE 设备名。适用于 Xilinx 的板卡。
参数	dev_reg: PCIE 寄存器设备名 dev_h2c: PCIE host to card 的设备名

	dev_c2h: PCIE card to host 的设备名
返回	0: 成功 < 0: 失败。(错误码参考附件 1)
示例	

2.1.4 设置 PCIE 基址

函数	int eeptpu_set_interface_addr_pcie(unsigned int mem_base_addr, unsigned int reg_base_addr);
功能	设置 PCIE 基址。适用于 Xilinx 的板卡。
参数	mem_base_addr: 内存基址 reg_base_addr: 寄存器基址
返回	0: 成功 < 0: 失败。
示例	

2.1.5 设置 EEP-TPU 的内存基址

函数	int eeptpu_set_base_address(unsigned int base0, unsigned int base1);
功能	仅针对 EEP-TPU 为软核的情况。 设置 EEP-TPU 的内存基址。与 boot 内核文件的配置有关，不同开发板可能有不同的内存大小，需根据实际情况进行配置，请在获取 boot 文件时咨询可用的基址。 一般情况下，可不使用这个函数。若 EEP-TPU 为软核且没有设置内存基址时，在加载 bin 文件时会有错误提示。
参数	

返回	0: 成功 < 0: 失败。
示例	tpu->eeptpu_set_base_address(0x30000000, 0x30000000);

2.1.6 加载 EEP-TPU BIN 文件

函数	int eeptpu_load_bin(const char* path_bin, int fg_multi=0);
功能	加载 eeptpu.bin 文件。 注意： 一个 EEPTPU 对象只能加载一个 EEPTPU BIN 文件。如果需要使 用多个 EEPTPU BIN 文件，则需定义多个 EEPTPU 对象。
参数	path_bin: eeptpu.bin 文件的路径 fg_multi: 是否已加载多个 bin 文件。程序中第一次调用本函数时， fg_multi 必须设置为 0；若一个程序中需要加载多个 bin 文件，则第 2 个开始及之后的 fg_multi 必须设置为 1。且第 2 个及之后不需要再设置 eeptpu_set_base_address 函数。
返回	0: 成功 < 0: 失败。
示例	tpu->eeptpu_load_bin(path_bin); 若需加载多个 bin 文件： tpu1->eeptpu_load_bin(path_bin); // 第 2 个参数不填的话，默认 =0 tpu2->eeptpu_load_bin(path_bin, 1); tpu3->eeptpu_load_bin(path_bin, 1);

2.1.7 获取 libeeptpu.so 版本

函数	char* eeptpu_get_lib_version();
功能	获取 libeeptpu.so 版本信息。

参数	
返回	版本字符串
示例	<pre>char* ptr = tpu->eeptpu_get_lib_version(); printf("EEPTPU library version: %s\n", ptr);</pre>

2.1.8 获取 EEP-TPU 硬件版本

函数	char* eeptpu_get_tpu_version();
功能	获取 EEP-TPU 硬件版本信息。
参数	
返回	版本字符串
示例	<pre>char* ptr = tpu->eeptpu_get_tpu_version(); printf("EEPTPU hardware version: %s\n", ptr);</pre>

2.1.9 获取 EEP-TPU 硬件配置信息

函数	char* eeptpu_get_tpu_info();
功能	获取 EEP-TPU 硬件配置信息。
参数	
返回	字符串
示例	<pre>char* ptr = tpu->eeptpu_get_tpu_info(); printf("EEPTPU hardware info : %s\n", ptr);</pre>

2.1.10 获取 EEP-TPU BIN 文件扩展信息

函数	char* eeptpu_get_extinfo();
功能	获取 EEP-TPU BIN 文件自定义的扩展信息字符串。该字符串在编译器编译时传入，可由用户自定义格式并自行解析。
参数	

返回	字符串
示例	<pre>char* ptr = tpu->eeptpu_get_extinfo(); printf("extinfo: %s\n", ptr);</pre>

2.2 输入数据

2.2.1 等待输入缓存可写

函数	<code>int eeptpu_wait_input_writable(unsigned int timeout_ms = 2000);</code>
功能	等待输入缓存变为可写入状态。当输入缓冲区的数据还未被使用时，是不能再继续写入数据的，否则会导致输入数据错乱。使用本函数可以等待输入缓存转变为可写入状态。 # 对于“写输入数据”与“推理”为异步执行的情况（例如分别在两个线程里），因为输入数据写入后，难以简单地判断当前输入数据是否进行了推理操作，为了防止该输入数据还未进行推理却被新数据覆盖的问题，需要在输入数据前执行本函数，等待输入可写。 # 对于“写输入数据”与“推理”为同步串行执行的情况，无需调用本函数进行等待。
参数	timeout_ms: 等待的超时时间，单位：毫秒。（默认值 2000ms）
返回	0: 成功。可以继续写入数据。 < 0: 失败。（错误码参考附件 1）。当前数据缓冲区的数据还未使用完，最好设置更长点的超时时间。如果强行写入的话，可能造成输入数据错乱，影响推理结果。
示例	<code>ret = tpu->eeptpu_wait_input_writable(2000);</code>

2.2.2 设置输入图像

函数	<code>int eeptpu_set_input(unsigned char* cvdata_resized, int channel, int height, int width);</code>
功能	设置输入的图像数据。

参数	cvdata_resized: 经缩放的图像数据。图像大小需与神经网络输入大小一致, 且为 Opencv 的 Mat 格式数据指针。 channel: 图像通道值 height: 图像高度 width: 图像宽度 神经网络输入图像的维度可在加载完 bin 文件后, 通过 tpu->in_c, tpu->in_h, tpu->in_w 来获取。
返回	0: 成功 < 0: 失败。(错误码参考附件 1)
示例	<pre>cv::Mat cvimg_resized; >> read&process image to cvimg_resized tpu->eeptpu_set_input(cvimg_resized.data,tpu->in_c,tpu->in_h, tpu->in_w);</pre>

2.3 推理

2.3.1 推理 (同步方式)

函数	<pre>int eeptpu_forward(std::vector<struct EEPTPU_RESULT> & result, int pixel_type);</pre>
功能	进行图像推理。
参数	result: 推理结果 <pre>struct EEPTPU_RESULT { float* data; // 最终数据 (3 维数组) int c; // 数据的通道值 int h; // 数据的高度 int w; // 数据的宽度 int pixel_type; // 数据排列方式</pre>

	<pre>}; pixel_type: 推理结果中数据的排列方式 enum { eepPixelType_CHW = 1, // 数据按照 C->H->W 的顺序排列 eepPixelType_HWC, // 数据按照 H->W->C 的顺序排列 };</pre>
返回	0: 成功 < 0: 失败。(错误码参考附件 1)
示例	<pre>std::vector<struct EEPTPU_RESULT> results; int ret = tpu->eeptpu_forward(results, eepPixelType_CHW);</pre>

2.3.2 推理（异步方式）

有些神经网络，后面有一些目前 EEP-TPU 暂不支持硬件加速的层类型，这些 EEP-TPU 不支持的层可自动归为软件计算方式进行计算。

异步方式的推理，适用于神经网络后面有连续的软件计算层的情况。这种情况下，一个神经网络先经 EEP-TPU 进行硬件计算，然后再进行后续的软件计算。可将这两部分计算放在不同线程，例如：一张图像在线程 1 进行 EEP-TPU 硬件推理后，可由线程 2 接着进行后续软件部分的推理，与此同时线程 1 可继续下一张图像的硬件推理，而不必等待整个神经网络推理完毕。这样可以充分利用系统资源，节省一部分时间，适用于对时间性能有较高要求的场合。

EEP-TPU 对于输入输出数据设有双缓冲，异步推理方式下，这种双缓冲机制可保证当前推理进行时，前一推理的输出数据不被覆盖。

函数	<code>int eeptpu_forward_async(int* cache_result_id);</code>
功能	异步推理方式的前半部分推理（硬件推理部分）
参数	cache_result_id: 接收输出数据所在的缓冲区 id 号。
返回	0: 成功

	< 0: 失败。(错误码参考附件 1)
示例	<pre>int res_id; ret = tpu->eeptpu_forward_async(&res_id);</pre>

函数	<pre>int eeptpu_forward_post(std::vector<struct EEPTPU_RESULT>& result, int cache_res_id, int pixel_type);</pre>
功能	异步推理方式的后半部分推理（软件推理部分）
参数	result: 推理结果 cache_result_id: 输出数据所在的缓冲区 id 号。 pixel_type: 推理结果中数据的排列方式
返回	0: 成功 < 0: 失败。(错误码参考附件 1)
示例	<pre>int res_id; ret = tpu->eeptpu_forward_async(&res_id); std::vector<struct EEPTPU_RESULT> results; ret = tpu->eeptpu_forward_post(results, res_id, eepPixelFormat_CHW);</pre>

2.3.3 获取 EEP-TPU 硬件部分推理时间

函数	<pre>unsigned int eeptpu_get_tpu_forward_time();</pre>
功能	获取 EEP-TPU 硬件部分推理时间。（单位：微秒）
参数	
返回	硬件部分推理时间（单位：微秒）
示例	<pre>unsigned int hwus = tpu->eeptpu_get_tpu_forward_time(); printf("EEPTPU hw cost: %.3f ms\n", (float)hwus/1000);</pre>

2.3.4 等待推理数据被取出

函数	<code>int eeptpu_wait_result_fetched(int timeout_ms);</code>
功能	适用于异步推理方式。 若输出缓冲区的数据还未被取出，而又进行推理的话，输出缓冲区的推理数据会被新的数据所覆盖，可能导致应用程序读取的推理结果错乱。在进行推理前，使用本函数等待一段时间，以便应用程序及时读出数据。
参数	<code>timeout_ms</code> ：超时退出时间，单位：毫秒。在这段时间内，若数据被取出，则会立即退出本函数。
返回	0: 成功 < 0: 失败。(错误码参考附件 1)
示例	<pre>tpu->eeptpu_wait_result_fetched(2000); ret = tpu->eeptpu_forward_async(&result_idx);</pre>

2.4 关闭

2.4.1 中止 EEP-TPU 运行

函数	<code>void eeptpu_terminate();</code>
功能	中止 EEP-TPU 运行。
参数	无
返回	无
示例	<pre>tpu->eeptpu_terminate();</pre> 示例场景： 在应用程序中捕捉到 Ctrl+C 等中断信号后，可调用本函数来及时中止并退出 EEP-TPU 的推理函数。

2.4.2 关闭 EEPTPU

函数	<code>void eeptpu_close();</code>
功能	关闭 EEPTPU。

参数	无
返回	无
示例	tpu->eeptpu_close(); 在应用程序关闭时，需调用此函数来关闭 EEPTPU。

2.5 其他

2.5.1 重置输入输出缓存

函数	void eeptpu_reset_iobuf();
功能	当出现用户写入了输入数据，却由于某些原因而停止推理操作的情况时，若下次继续进行推理，则会出现推理使用的是一段之前设置的输入图像数据，或者由于输入数据还未被使用而无法覆盖写入，这可能是用户不希望这样的。因此，利用本函数可重置输入输出缓存，使用户可继续写入新数据进行推理。
参数	无
返回	无
示例	tpu->eeptpu_reset_iobuf();

3. 推荐的多线程方案

- # 线程 1：图像预处理线程
 - 读取图像
 - 图像预处理
 - 图像写入到 EEP-TPU 输入缓存
- # 线程 2：EEP-TPU 推理线程
 - 同步或异步推理（eeptpu_forward）
- # 线程 3：后处理线程
 - 如果采用异步推理，则需要额外计算 eeptpu_forward_post
 - 对输出缓存的推理结果进行后处理。

注意：采用多线程方式时，需做好数据同步问题。图像预处理线程与推理线程要做好数据同步，防止出现推理过程还没结束而又继续写入多张输入图像的问题（导致一些图像还未推理就被新的覆盖）。采用异步推理方式时，要防止出现 eeptpu_forward_async 执行多次而 eeptpu_forward_post 速度跟不上导致推理结果被覆盖的问题。

4. 同一程序中多个神经网络算法交替使用

EEP-TPU API 支持在同一个程序中交替使用多个神经网络（eeptpu.bin）。

如下所示代码完成初始化操作后，可通过 tpu1/tpu2/tpu3 来调用不同算法：

```
EEPTPU *tpu1 = NULL;
EEPTPU *tpu2 = NULL;
EEPTPU *tpu3 = NULL;

char path_bin1[] = (char*) "./eeptpu1.bin" ;
char path_bin2[] = (char*) "./eeptpu2.bin" ;
char path_bin3[] = (char*) "./eeptpu3.bin" ;

int eeptpu_init(int interface_type)
{
    int ret;
    if (tpu1 == NULL) tpu1 = tpu1->init();
    if (tpu2 == NULL) tpu2 = tpu2->init();
    if (tpu3 == NULL) tpu3 = tpu3->init();

    // 设置基址的函数只需在第 1 个 EEPTPU 对象调用即可。
    tpu1->eeptpu_set_base_address(0x30000000, 0x30000000);

    // 配置第 1 个 EEPTPU 对象
    ret = tpu1->eeptpu_set_interface(interface_type);
    if (ret < 0) return ret;
    ret = tpu1->eeptpu_load_bin(path_bin1);
```

```

    if (ret < 0) return ret;

    // 配置第 2 个 EEPTPU 对象
    ret = tpu2->eeptpu_set_interface(interface_type);
    if (ret < 0) return ret;
    ret = tpu2->eeptpu_load_bin(path_bin2, 1);
    if (ret < 0) return ret;

    // 配置第 3 个 EEPTPU 对象
    ret = tpu3->eeptpu_set_interface(interface_type);
    if (ret < 0) return ret;
    ret = tpu3->eeptpu_load_bin(path_bin3, 1);
    if (ret < 0) return ret;

    return 0;
}

```

5. 错误码列表

错误码	错误描述
0	成功
-1	失败
-2	参数错误
-3	不支持的操作
-4	文件打开失败
-5	文件读取失败
-6	文件写入错误
-7	内存分配失败
-8	超时
-11	加载 Bin 文件失败

-12	EEP-TPU 初始化失败
-13	错误的图像类型
-14	不支持的像素类型
-15	均值设置失败
-16	错误的 Blob 输入
-17	错误的 Blob 格式
-18	错误的 Blob 大小
-19	错误的 Blob 数据
-20	错误的 Blob 输出
-21	内存地址错误
-22	内存数据写入失败
-23	内存数据读取失败
-24	内存映射失败
-25	设备打开失败
-26	设备未初始化
-27	EEP-TPU 接口类型设置失败
-28	EEP-TPU 接口初始化失败
-29	EEP-TPU 接口操作失败
-40	EEP-TPU Bin 文件版本过旧，请用新版本编译器重新生成 Bin 文件
-41	EEP-TPU 库文件版本过旧，请使用新版本的库文件
-42	EEP-TPU 硬件版本过旧，请升级 EEP-TPU

附录 B. EEP-TPU Compiler

1. 编译和运行环境

桌面版 Ubuntu 18.04 LTS。

2. 编译器版本

目前的 EEP-TPU Compiler，根据 EEP-TPU IP 配置的不同，分为 C8 和 C16 两个版本的编译器。C8 编译器生成的 bin 文件可通用于 C16 平台；C16 编译器生成的 bin 文件只能在 C16 平台的 EEP-TPU 上使用。一般来说，C16 生成的 bin 文件，在 EEP-TPU 上运行的速度比 C8 快。C8 和 C16 编译器的使用方式是一样的。

3. 使用

在 Ubuntu 系统的命令行终端执行

a) 命令行参数：

```
./eeptpu_compiler_c8 -h
```

Usage:

`./eeptpu_compiler [options]`

options:

`--help(-h)` # print this help message

`--prototxt <path>` # caffe prototxt file path

`--caffemodel <path>` # caffe caffemodel file path

`--image <path>` # One typical image for this neural network.
(Format: jpg or pgm)

`--mean <values>` # mean values. Format: float array string, such as
"103.94,116.78,123.68"

`--norm <values>` # normalize values. Format: float array string,
such as "0.017,0.017,0.017"

--extinfo <ext> # extend info. mainly for object detection setting.
--output <folder> # output eeptpu.bin to folder

b) 参数说明:

--prototxt: Caffe 的 prototxt 文件路径。

--caffemodel: Caffe 的 caffemodel 文件路径。

--image: 适用于目前这个神经网络的一张典型的图片的路径。支持 jpg 和 pgm 格式的图片。

--mean: 均值 (加) 。

--norm: 均值 (乘) 。

均值计算公式: $X' = (X - \text{mean}) * \text{norm}$ 其中: X 为输入的值; X' 为均值计算后的值, 作为神经网络的输入。均值的输入需与神经网络输入的通道数一致。例如: 输入通道数是 3, 则均值也需要是一个 3 个数值的数组, 每个数值对应一个通道。目前暂不支持均值文件。均值的获取: 均值一般可以从该神经网络的训练文件 (例如: train.prototxt) 中获取。

--extinfo: 自定义的扩展配置字符串。用户可自定义写入的格式, 可在用户程序中通过 API 函数读出并由用户自行解析。

--output: eeptpu.bin 文件的输出文件夹。输出文件始终以 eeptpu.bin 命名, 请注意使用不同文件夹区分不同的神经网络。

4. 兼容性

EEP-TPU 编译器 V0.6.4 版本生成的 eeptpu.bin 文件, 需要搭配使用的 EEPTPU API 版本为: V0.6.2 及以后; 需要搭配使用的 EEP-TPU 硬件版本为: 0.7.1-R14 及以后。EEP-TPU 编译器支持 Caffe 格式的 prototxt 和 caffemodel 文件。其他平台 (例如 Mxnet、PyTorch 等) 生成的模型文件, 可通过模型转换工具转换成 caffe 格式。

5. EEP-TPU 支持的层列表

在当前版本下，EEP-TPU 编译器支持如下图所示的 LS-V2.0 所包含的层类型

	Layer type	Limitation		Layer type	Limitation
LS-V1.0	Convolution	kernel<=32,stride<=8	LS-V3.0	ALL layers in LS-V2.0	
	Depthwise Convolution	Same as CONV		Transposed Convolution	Same as CONV
	Dilated Convolution	Same as CONV		Transposed Depthwise Convolution	Same as CONV
	Dilated Depthwise Convolution	Same as CONV		Concatenation	Any size channel
	Fully connected			Split, Tile, Slice, Relu6, Absval	
	(Global) MAX/AVE Pooling	Same as CONV		Layer type	Limitation
	SUM/MUL/MAX Element-wise		LS-V4.0	ALL layers in LS-V3.0	
	Concatenation	Channel must be integral multiple of 16		Sigmoid, Tanh, Exp, Log, Power, Sqrt, Elu	
	Relu, Prelu, LeakyRelu, Input, Dropout			Layer type	Limitation
LS-V2.0	Layer type	Limitation	LS-Vx	ALL layers in LS-V4.0	
	ALL layers in LS-V1.0			Permute, Flatten, Normalize, InstanceNorm, LRN, Softmax, Crop, ROI Pooling, Reshape, PSROI Pooling, ChannelShuffle, RNN, LSTM	
	Interp	$0 \leq \text{scale} \leq 5.0$			
	upsample	$\text{scale} \leq 5$			
	ArgMax, Scale, BN				

6. 异构计算特性

EEP-TPU 编译器支持算法自动划分的异构计算，也即自动根据上述层列表（目前是 LS-V2.0），TPU 支持的层计算用 TPU 来执行，其余的层计算用 CPU 来执行。

这种 TPU/CPU 计算划分有两种情况：

- (1) TPU->CPU->TPU 交替执行。这种情况下，TPU 和 CPU 需要频繁的交替切换。由于这种切换会损失额外的时间，因此不推荐在该模式下运行算法。如果用户遇到这样的情况，请提升硬件版本是的相应的层被硬件支持，或者更改算法，把相应的层用支持类型的层来替换。
- (2) TPU-CPU 顺序执行。可以实现高效的 TPU/CPU 异构并行计算

另外，EEP-TPU 编译器还支持多个输出层，不局限于单个输出。通过 EEPTPU API 函数，可将多个层的推理结果数据合并读取。具体请参考附录 A. NNAPI